



首都经济贸易大学

Capital University of Economics and Business

博士学位论文

Dissertation for Degree of Doctor

论文题目: 随机森林算法优化研究

专 业: 统计学

学 号: 12010120053

作 者: 曹正凤

指导教师: 纪宏教授 谢邦昌教授

完成时间: 2014 年 3 月

独创性声明

本人郑重声明：今所呈交的《 随机森林算法优化研究 》论文是我个人在导师指导下进行的研究工作及取得的科研成果。尽我所知，文中除了特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写的内容及科研成果，也不包含为获得首都经济贸易大学或其它教育机构的学位或证书所使用过的材料。

作者签名： 曹凤 日期： 2014 年 3 月 20 日

关于论文使用授权的说明

本人完全了解首都经济贸易大学有关保留、使用学位论文的有关规定，即：学校有权保留送交论文的复印件，允许论文被查阅、借阅或网络索引；学校可以公布论文的全部或部分内容，可以采取影印、缩印或其它复制手段保存论文。（保密的论文在解密后应遵守此规定）

作者签名： 曹凤 导师签名： 王乙 日期： 2014 年 3 月 20 日

首都经济贸易大学

博 士 学 位 论 文

Dissertation for Degree of Doctor

论文题目： 随机森林算法优化研究

Study on optimization of random forests algorithm

院 系： 统计学院

专 业： 统计学

学 号： 12010120053

作 者： 曹正凤

指导教师： 纪宏教授 谢邦昌教授

完成时间： 2014.3

摘要

随机森林算法(Random Forests)是一种基于统计学习理论的组合分类器,它将 bootstrap 重抽样方法和决策树算法相结合,该算法的本质是构建一个树型分类器 $\{h_k(x), k=1, \dots\}$ 的集合,然后使用该集合通过投票进行分类和预测。由于该算法较好地解决了单分类器在性能上无法提升的瓶颈,因此具有较好的性能,能应用于各种分类筛选和预测中。当然,该算法也存在一些有待完善的地方,针对这些不足,理论界主要集中在三个方面进行优化研究,一是引进新的算法,二是对将数据预处理融入到算法中,三是针对算法自身构建过程进行优化。本文在充分查阅国内外相关资料的基础上,对后两个方面开展了优化研究。

一、在数据预处理方面,提出了两种改进随机森林的优化算法。

首先,针对随机森林不能很好地处理非平衡数据的问题,根据聚类算法思想和物理学的重心理论,本文提出了 C_SMOTE 算法,该算法能较好地降低数据集的非平衡性,从而提升了随机森林算法的分类性能。该算法针对 SMOTE 算法在选取“人造”样本时,存在一定的盲目性现象和容易产生边缘化的问题,提出了从负类样本的重心出发,有目的构造“人造”样本的新思路,使得在“人造”负类样本的过程中,新产生的样本有向重心汇聚的趋势,这样就可以有效地解决了 SMOTE 算法的缺陷,从而实现了既保留原有数据集的信息,又较好地解决数据集的非平衡性问题,从而在很大的程度上提升了随机森林算法在非平衡数据集上的分类性能。

其次,随机森林算法在进行节点分裂时常采用 C4.5 算法,但 C4.5 算法在处理连续变量时,采用二分离散化的方法,该方法运行效率依赖于连续变量取值的数量,该数量越大,随机森林算法执行时间越长。针对此现象,本文提出了一种降低连续变量取值的数量的新算法,该算法可以很好地为 C4.5 算法提供简约的数据集,从而提升 C4.5 算法的执行效率。新算法是在借鉴 CHI2 系列算法思想的基础上,针对 CHI2 系列算法没有考虑 χ^2 统计量和真实值之间存在偏差的问题而提出的。该算法使用 χ^2 矫正公式较好地处理了 CHI2 系列算法中的偏差问题。文中通过使用三种通用的 UCI 数据集,将新算法和没有解决偏差问题 CHI2 系列算法,在改善随机森林算法性能方面进行了比较分析。实证数据表明,和 CHI2 系列算法相比,新算法能更有效地约简数据集中的冗余信息,使连续变量取值的数量很大程度地减少,从而提升随机森林算法的执行效率。

二、在随机森林自身构建过程优化方面,本文通过分析随机森林算法分类性能的影响因素,针对随机森林在生成过程中,节点分裂算法不同引起的随机森林分类性能不同的现象,提出了一种基于线性组合的节点分裂混合算法。该算法将 C4.5 算法和 CART 算法在节点分裂时的函数进行线性组合,通过变换组合函数中的系数,充分发

挥了这两种算法优势,较好地实现了随机森林算法分类性能的优化。同时,还详细分析了混合算法的稳定性、相关度和强度。首先,通过构造 F 统计量进行方差分析,对该混合算法的稳定性进行了检验。统计结果表明,该随机森林的混合算法随着森林中树木个数的变化虽然存在一定的不稳定性,但当森林中树木达到 800 棵时,算法可以达到稳定的状态。然后对混合算法的相关度和强度进行了理论上的推导和论述,同时实现了随机森林的平均相关度和强度的计算,并使用实证分析的办法,验证了平均相关度和算法分类精度存在负相关,森林的平均强度和算法的分类精度存在正相关的关系,并得了出混合算法对提升森林的平均强度和降低平均相关度较有其他算法具有明显的优势,也从另一个方面验证了混合算法的优越性。

在优质股票池选择的实际应用中,该应用的数据集存在大量的连续变量,且该应用对分类算法的精度要求严格。本研究提出的随机森林优化算法,可以很好地处理连续变量及提升随机森林的分类精度。本文在价值成长投资策略的选股指标体系的基础上,通过小波分析和 COR_CHI2 算法进行数据预处理,使用节点分裂混合算法形成的随机森林成功地实现了优质股票池的选择,可为投资者进行有针对性的投资组合提供统计支持。

关键词:随机森林 非平衡数据集 连续变量离散化 节点分裂 优化研究

Abstract

Random forests are a combined classifier based on statistical learning theory. Through combining bootstrap re-sampling method and decision tree algorithm, its essence is to build a collection tree classifier $\{h_k(x), k=1, \dots\}$, and then use the set by voting for classification and prediction. Because the algorithm breaks through the bottleneck that the single classifier cannot improve in performance, it has good performance and can be applied to various kinds of classification filtering and prediction. Certainly there are also some points needing to be improved in the algorithm. Aiming at these deficiencies, theoretical research is mainly conducted in three aspects. One is the introduction of new algorithm, second is to blend in data preprocessing in the algorithm, and the third is the optimization of the algorithm structuring process. On the basis of full access to relevant information from both China and abroad, this paper concentrates on the study of latter two aspects optimization.

In terms of data preprocessing, this paper presents two optimization algorithms to improve the random forests.

First, in view of the random forest's inability to well deal with the issue of unbalanced data, and in line with clustering algorithm and the center of gravity in physics, this paper puts forward the C_SMOTE algorithm, which can reduce the data set unbalance, so as to improve the random forest classification performance. Aiming at SMOTE algorithm having certain blindness and prone to the problem of marginalization in the selection of "artificial" sample, the algorithm put forward the starting from the gravity center of the negative samples and with the new thoughts to purposely structure "synthetic" samples, which makes new samples have the trend of convergence to the gravity center in the process of structuring "synthetic" negative samples and effectively solves the defects of the SMOTE algorithm. It not only keeps the information of original data set, but also better solves the problem of unbalanced data sets, which to a large extent, improves the random forest algorithm in classification performance of unbalanced data sets.

Second, random forest often uses C4.5 algorithm for node split, but in dealing with continuous variables, C4.5 algorithm uses dichotomy discretization method with its operation efficiency depending on the number of continuous variable values. The larger is the number, and the longer is the execution time of random forests. Aiming at this problem, this paper puts forward a new algorithm to reduce the number of continuous variable values. This algorithm can provide simple data set for C4.5 algorithm, so as to improve the

execution efficiency of C4.5 algorithm. It uses χ^2 correction formula to deal with the deviation in CHI2 series algorithm. By using three kinds of common UCI data sets, this paper comparatively analyzes the new algorithm and the CHI2 series algorithm in terms of improving the performance of random forest. Empirical data show that compared with CHI2 series algorithm, the new algorithm can reduce the redundant information of data set more effectively, making the number of continuous value greatly reduced and thus to improve the execution efficiency of random forests.

In terms of random forests structuring process optimization, through analyzing the factors affecting the classification performance of random forests and aiming at node splitting method difference causing random forest classification performance difference in random forests generating process, this paper proposes a node split hybrid algorithm based on linear combination. The algorithm brings the function of C4.5 algorithm and CART algorithm in the node split and forms a linear combination function. Through the conversion of combination function coefficient, it gives full play to the advantages of these two algorithms and realizes the random forest classification performance optimization. In the mean time it is also analyzed in detail the stability, relevancy, and strength of the hybrid algorithm. First of all, by constructing F statistic variance analysis, the stability of the hybrid algorithm is inspected. Statistical results show that the hybrid algorithm of random forest has certain instability as the change of the number of trees in the forest, but when there are more than 800 trees in the forest, the algorithm can achieve the stable state. Then the correlation and intensity of hybrid algorithm are theoretically derived and discussed, and meanwhile the average correlation and strength of random forests are calculated. Furthermore, empirical analysis is used to verify that there is negative relationship between the average correlation and algorithm classification accuracy, the average intensity of forest and classification accuracy of algorithm are positively related, and comparing with other algorithms, the hybrid algorithm has obvious advantages in improving average intensity and reducing average correlation of the forest, and also from another aspect, the superiority of the hybrid algorithm is verified.

In practical application of selecting high quality stock pool, there are a large number of continuous variables in the application of data sets, and it has a high accuracy requirement for classification algorithm. The optimization algorithm proposed in this paper is a good way to deal with continuous variables and improve the classification precision of random forests. Based on screening stock index system with value growth investment strategy, this paper uses wavelet analysis and COR_CHI2 algorithm for data preprocessing,

using random forests from node split hybrid algorithm to successfully realize the choice of high quality stock pool, so as to provide statistical support to investors for targeted investment portfolios.

Key Words :Random forests unbalanced data continuous variables discretization
node split the study of optimization

目 录

摘 要	3
Abstract	5
目 录	I
1 引 言	1
1.1 研究背景和意义	1
1.2 国内外随机森林优化方法综述	2
1.2.1 引入新的算法进行优化的方法综述	2
1.2.2 将数据预处理融入随机森林算法的优化方法综述	3
1.2.3 针对随机森林算法自身构建过程进行优化的方法综述	3
1.3 本文的主要工作与结构安排	4
1.3.1 本文的主要工作	4
1.3.2 本文的结构	5
2 随机森林研究综述	6
2.1 随机森林的基分类器——决策树	6
2.1.1 决策树概述	6
2.1.2 决策树生成过程中的节点分裂算法	6
2.1.3 决策树分类中存在的问题	12
2.2 随机森林的构建过程	12
2.2.1 为每棵决策树抽样产生训练集	12
2.2.2 构建每棵决策树	14
2.2.3 森林的形成及算法的执行	15
2.3 随机森林的随机性分析	15
2.3.1 训练集的随机选取	16
2.3.2 随机特征变量的随机性	16
2.3.3 随机森林的随机性在实证分析中的表现及解决办法	16
2.4 随机森林的理论概述	17
2.4.1 随机森林的数学定义	17
2.4.2 随机森林的性质	19
2.5 随机森林的性能指标	19
2.5.1 分类效果系列指标	19
2.5.2 泛化误差与 OOB 估计	21
2.5.3 随机森林算法运行效率指标	22
2.6 几种实现随机森林算法的软件介绍	23

2.6.1	Weka 软件.....	23
2.6.2	R 软件.....	24
2.6.3	MATLAB 软件.....	25
2.7	随机森林算法存在的问题.....	25
2.7.1	不能很好地处理非平衡数据.....	25
2.7.2	对连续性变量的处理还需要进行离散化.....	26
2.7.3	随机森林算法的分类精度还需要进一步的提升.....	26
2.8	本章小结.....	27
3	随机森林处理非平衡数据集的优化研究.....	29
3.1	非平衡数据简介.....	29
3.2	非平衡数据集分类困难的原因分析.....	29
3.2.1	数据稀少问题.....	30
3.2.2	数据碎片问题.....	30
3.2.3	归纳偏差问题.....	31
3.2.4	噪声问题.....	31
3.2.5	评价指标选择问题.....	31
3.2.6	数据分布问题.....	32
3.3	非平衡数据集分类问题的解决办法.....	32
3.3.1	通过改进算法解决非平衡问题的方法简介.....	32
3.3.2	通过改造数据解决非平衡问题的方法简介.....	33
3.4	随机森林处理非平衡数据分类问题的优化--C_SMOTE 算法.....	35
3.4.1	C_SOMTE 算法设计.....	35
3.4.2	C_SMOTE 算法流程图.....	37
3.5	使用 C_SMOTE 算法提升随机森林算法性能实证分析.....	38
3.5.1	数据集介绍.....	38
3.5.2	实验环境.....	38
3.5.3	实验结果及分析.....	39
3.6	本章小结.....	45
4	随机森林处理连续变量的优化研究.....	46
4.1	连续变量离散化概述.....	46
4.1.1	连续变量离散化的概念.....	46
4.1.2	连续变量离散化的衡量标准.....	46
4.1.3	随机森林算法中连续变量离散化方法.....	47
4.2	连续变量离散化的方法简介.....	48
4.2.1	简单的连续变量离散化算法.....	48
4.2.2	基于信息熵的 CADD 系列算法.....	50
4.2.3	基于统计学的 CHI2 系列算法简介.....	51

4.3	随机森林处理连续变量的优化—COR_CHI2 算法.....	52
4.3.1	COR_CHI2 算法设计.....	53
4.3.2	COR_CHI2 算法程序流程图.....	54
4.4	COR_CHI2 算法对随机森林算法性能优化实证分析.....	54
4.4.1	实证分析使用的数据集介绍.....	54
4.4.3	COR_CHI2 算法对数据集的约简效果分析.....	55
4.4.4	COR_CHI2 算法约简后的数据集对随机森林算法运行效率的影响分析.....	58
4.5	本章小结	59
5	随机森林算法分类性能的优化研究.....	60
5.1	随机森林算法分类性能的影响因素分析.....	60
5.1.1	森林中决策树的数量.....	60
5.1.2	随机特征变量 F 的值	62
5.1.3	不同的节点分裂算法比较.....	64
5.2	随机森林的分类性能优化--节点分裂混合算法	67
5.2.1	节点分裂混合算法描述.....	68
5.2.2	节点分裂算法的程序流程.....	68
5.2.3	混合算法分类性能实证分析.....	69
5.2.4	混合算法与 R 软件中随机森林算法的比较	71
5.3	混合算法稳定性分析.....	73
5.3.1	混合算法稳定性估计的模型分析.....	73
5.3.2	混合算法稳定性估计.....	74
5.3.3	混合算法稳定状态分析.....	76
5.4	混合算法形成的森林中相关度及强度分析.....	78
5.4.1	随机森林中相关度及强度估计.....	78
5.4.2	混合算法形成的森林中强度分析.....	78
5.4.3	混合算法形成的森林中相关度分析.....	79
5.5	本章小结	80
6	优化后的随机森林算法对优质股票池的选择.....	82
6.1	量化投资中选股研究综述.....	82
6.1.1	朴素贝叶斯分类方法在选股分析中的应用	82
6.1.2	决策树分类方法在选股分析中的应用	83
6.1.3	神经网络分类方法在选股分析中的应用	83
6.1.4	支持向量机分类方法在选股分析中的应用	83
6.1.5	使用优化后的随机森林算法进行选股分析的原因	84
6.2	股票数据预处理方法简介.....	84
6.2.1	数据标准化处理.....	84
6.2.2	数据噪声处理.....	85

6.3	选股模型指标体系的简介.....	88
6.3.1	传统的价值型投资策略.....	88
6.3.2	积极的成长型投资策略.....	90
6.3.3	价值成长投资策略 (GARP).....	90
6.3.4	以价值成长投资策略为指导的选股模型指标体系的构建.....	91
6.4	优化后的随机森林算法对优质股票池的选择.....	94
6.4.1	样本的选择.....	94
6.4.2	指标数据的财报匹配.....	95
6.4.3	数据预处理.....	95
6.4.4	当月实际表现分析.....	96
6.5	本章小结.....	97
7	总结与展望.....	98
7.1	本文的创新点.....	98
7.2	今后的研究思路.....	98
8	参考文献和注释.....	100
9	附 录.....	107
10	在学期间发表的学术论文和研究成果.....	120
11	后 记.....	121

1 引言

1.1 研究背景和意义

在数据和信息急剧膨胀的今天,如何获取数据和信息已经不是什么难事,比较棘手的是如何获取有效的对数据进行统计分析的工具,从而实现对这些数据和信息进行有效的加工和处理。因此,如何在原有统计分析和数据挖掘工具的基础上,进一步进行优化研究是该研究领域的热门话题之一。

在数据挖掘领域中,分类技术是较为重要且应用广泛的技术之一。它根据训练集建立合适的分类器,然后通过这个分类器对用户需要分类的数据给出预测的分类结果。根据分类器的多少,分类技术分为单分类器技术和多分类器技术。单分类器技术中比较有代表性的是贝叶斯和决策树,这些算法都在一定程度上促进了分类技术的发展,各方面的研究和应用都得到了全面的开展,但是单分类器由于自身的限制,其性能提升达到了无法超越的瓶颈,于是,人们开始提出多分类器组合的思想。多分类器组合使用多个基分类器进行分类,并综合所有分类结果形成一个最终结果。随机森林就是在这个背景下产生的一种多分类器组合。

多分类器组合思想起源于集成学习算法。集成学习算法是机器学习的一种新的学习思想,该学习算法把同一个问题分解到多个不同的模块中,由多个学习器一起参与学习,共同解决目标问题,从而提高分类器的泛化能力^[1]。

将集成学习算法应用到数据挖掘的数据分类领域,最早先的是 Boosting 方法^[2]和 Bagging 方法^[3],这两个方法针对单分类器性能无法提升的现实问题,参照集成学习算法的中心内容,提出了通过产生多个分类器降低算法的泛化误差,从而提升算法性能的思想,并将该思想应用到了数据分类领域。

随着集成学习方法的发展,Tin Kam Ho 在 1995 年提出了随机决策森林(Random Decision Forests)的概念^[4],三年后,他又提出了随机子空间的集成方法^[5],在 2001 年 Leo Breiman 将随机森林方法^[6]进行系统的阐述,随机森林算法正式成为数据挖掘分类算法的重要组成部分。在过去的 10 多年中,由于随机森林算法的优良表现,因此该算法在实际中得到广泛应用,成为数据分析和挖掘、知识管理、模式识别等众多领域的研究人员和技术人员共同关注的一个热点研究话题。

在生物信息学方面,Parkhurst 等人使用随机森林算法研究了沙滩细菌密度与其它变量的影响关系,Smith 等人用随机森林算法对细菌追踪数据进行了研究,并和判别分析方法进行了比较 Alonso 等人利用生物标记寄生虫进行鱼类种群判别^[7-9];在生态学方面,Gislason 等人利用随机森林算法方法对土地的覆盖面积进行了研究,并发现随机森林算法与其它组合算法相比训练更快,Jan 等人基于随机森林算法和 Logistic 模

型建立了方法生态水文分布模型,比较发现随机森林算法的预测误差小于 Logistic 模型^[10-11];在医学上, Lee 等人利用随机森林技术对肺 CT 图像进行肺结节的自动检测,同时还在随机森林算法中加入了(CAC)^[12];在遗传学上, Diaz – Uriate 等人利用随机森林算法方法进行基因识别, Chen 和 Liu 进行了蛋白质相互关系的研究^{[13] [14]};在遥感地理学上, Pal、Ham 和 Gislason 等人都分别利用随机森林分类器进行了遥感研究^{[15] [16] [17]}。此外 Xu 等还进行语言的自动学习, Auret 等人还将随机森林应用到时间序列中,对时间序列的变化点进行检测^{[18] [19]}。另外,随机森林算法在空间特征识别、网络完全检测^[20]、信息抽取^[21]、生物芯片^{[22] [23] [24]}等领域,也都取得了比较好的效果。在国内研究人员关注的重点是随机森林算法的应用,而在随机森林算法优化和改进方面的研究的还不是太多。

虽然该算法应用越来越广泛,功能也越来越强大,但是随机森林算法并不是十全十美的,在算法处理不同数据集的性能和算法分类精度的提升方面,还存在着一定的空间。这也就是本篇文章研究的背景和出发点。

1.2 国内外随机森林优化方法综述

由于随机森林算法的优良特性,在很多领域都得到了广泛的应用,但正如上一节所述,算法的分类精度还存在优化的需要,因此理论界就随机森林算法的整体性能的优化提出了很多很好的方法,细分起来,它们可以分成以下三个主要的方面。

1.2.1 引入新的算法进行优化的方法综述

Breiman 在 2001 年提出随机森林算法,并从理论和实验两方面进行了详细和明确的阐述,从本质上讲,该算法是 Breiman 的 Bagging 方法和 Tin Kam Ho 的 Random Subspace 方法的组合。从这以后,不同随机森林的改进,大部分都采用将其他优秀的算法融合到 Breiman 的随机森林算法中来,实现算法的优化组合。

Gall^[25]等人将霍夫变换引入到随机森林的投票机制中,得出霍夫森林算法,该算法被广泛应用于各类目标跟踪和行为识别^{[26] [27]}。霍夫森林针对随机森林的投票过程进行优化改进,使用霍夫投票来代替多数投票机制。在霍夫森林的叶子节点除了进行类别的测试外,还利用广义霍夫变换在霍夫空间中进行最大后验概率估计,以此检测偏移量的不纯度。

Ishwaran^[28]等人引入生存树的概念,对随机森林构建过程进行改进,得出随机生存森林算法。该算法在抽样时也采用 bootstrap 重抽样方法进行 N 个训练集的选取,不同的是对每个训练集都要建立一棵生成分析树,在进行投票时,计算每棵树的生存函数,森林的组合值是平均生存函数,这样就将生成分析树的内容和预测结果进行了综合。该算法在计算生存函数时,采用的是 KM 估计的方法。Ishwaran 等人^[29]还从理论上证明了随机生存森林的一致性,并认为随机生存森林(RSF)在处理高维数据时,要显著优于其他生存分析方法。

Nicolai^[30]等人引入分位数回归的理论,将分位数回归应用于决策树的生成和决策的过程中,提出了分位数回归森林算法。他从数学上证明了分位数回归森林的一致性。该算法认为对于每一个属性变量的值都可以得到原始 n 个观察值权重集合 $\omega_i(x), i=1, 2, \dots, n$ 。单棵决策树的预测通过因变量的观测值 $Y_i (i=1, \dots, n)$ 的加权平均得到。单棵决策树的预测值为 $\mu(x) = \sum_{i=1}^n \omega_i(x, \theta) Y_i$ 。

马景义^[31]等人在随机森林算法中引入 Adaboost 算法,通过分析 Adaboost 算法在自适应自助抽样权重和自适应投票权重设置方面的优势,设计拟自适应分类随机森林算法,该新算法把 Adaboost 算法和随机森林算法的优点集中在一起,得到了较好的实验结果。Dirk Van den Poel 和 Anita Prinzie^[32]等人引入多项 Logit 模型,将该模型和随机森林算法进行结合,实现了一种新的多分类器的随机多项式 Logit 模型,记为 RMNL。

1.2.2 将数据预处理融入随机森林算法的优化方法综述

针对随机森林算法在处理非平衡数据和连续性变量方面存在的问题,研究者们将数据预处理也纳入到随机森林算法优化的范畴中来,通过对数据进行预处理后,算法的性能得到了改善,一些不能处理的数据,和一些不能应用的领域也得到了突破。

吴琼^[33]等人提出将 NCL 技术引入随机森林算法,主要是针对非平衡训练集,对数据进行 NCL (Neighborhood Cleaning Rule) 技术处理,然后将处理过的数据结合随机森林 (Random Forest) 算法进行分类,试验结果表明,改进后的随机森林算法分类效果更好。

杜均^[34]等人从应用代价敏感学习算法解决非平衡数据集分类问题的角度,研究出一种新的代价敏感随机森林算法。代价敏感随机森林算法首先通过对原始训练集进行可替代的随机采样办法,形成多个袋(bag);然后在每个袋中随机选择部分属性的来剪枝,从而产生代价敏感决策树;最后将多个袋组合起来,形成集成算法。

1.2.3 针对随机森林算法自身构建过程进行优化的方法综述

雍凯^[35]等人通过分析随机森林中各棵树的强度和相关系数之间的关系,提出了一种新的随机森林特征选择算法。该算法的主要思想是通过随机森林的泛化误差的上界的分析发现:在构建随机森林时,增大森林中决策树的强度可以实现降低随机森林的泛化误差的目标。但是他们还发现,在增加强度的同时,应最小限度的减少决策树之间的相关性。

在这个思路的指导下,他们提出了对属性进行权重评估的办法,具体解决方法是:首先根据卡方检验进行属性的相关性评估。然后通过给每个属性赋予权重的方法,这些权重就表示这个属性和目标属性的关联度,这些权重通过分析每个属性和目标属性之间的关联度计算得出。最后,通过对这些计算出来的权值进行排序,在选择属性时,取排好顺序的序列前几位,这样属性选择就有了限定,从而使得森林中各个决策树之

间的强度就会增大,同时降低了决策树之间的相关系数,达到提升单分类器分类精度的目的。

孙丽丽^[36]等人针对传统决策树算法每个结点只有一个属性,在生成和决策时过程均比较复杂的情况,提出在每个决策树的结点生成多个属性的随机森林算法。该算法采用较少的、有针对性的对每一个数据聚类堆进行随机选择的属性,生成多棵多变量决策树。最后,根据每一个聚类堆构建的多棵多变量决策树形成随机森林来进行分类器的加权集成,从而试图覆盖数据集中的所有概念,进一步保证了随机森林算法的分类精度。

从上述三种优化方面看,随机森林算法分类性能的提升,主要集中对各种算法的组合研究上,这些组合的研究一般都应用在某个特定的领域;第二个方面侧重于对于特殊数据集在随机森林中的应用优化;而第三个方面的优化方式主要通过对算法本身的改进,使其具有一定的通用性,可以应用在不同的领域。本文拟从第二和第三个方面入手,进行随机森林算法的优化。

1.3 本文的主要工作与结构安排

1.3.1 本文的主要工作

作为集成学习的多分类器中的一种,随机森林算法充分利用了单个分类器和统计抽样技术,解决了单分类器在性能上无法提升的问题,但其本身也存在一些有待完善的地方和一些应用上的缺陷,本文在充分查阅国内外相关资料的基础上,针对随机森林在理论和应用方面存在的问题,主要进行了如下两方面的优化工作。

1. 在数据预处理方面,提出了两种改进随机森林性能的优化算法。

首先,充分研究了非平衡问题的研究现状和几个优秀的非平衡数据处理算法,在分析 SMOTE 算法自身存在问题的基础上,提出一种新 C_SMOTE 算法,并对该算法进行了 JAVA 程序实现,使用不同的数据集进行测试新算法和 SMOTE 算法,分析两个算法对随机森林算法分类性能的改进效果。

其次,在对连续变量离散化的研究现状进行阐述后,分析了基于统计的和基于信息熵的两种连续变量离散化算法,探讨了基于统计 CHI2 系列算法自身存在的问题,并针对该问题提出了一种新的连续变量离散化算法,在对该算法进行了 JAVA 程序实现后,使用不同的 UCI 数据集进行了算法的性能测试,得出了新算法能更有效地约简数据集中的冗余信息,而且提升了数据集在随机森林算法上的执行性能和效率的结论。

2. 在随机森林自身优化方面对随机森林算法分类性能影响因素进行了详细的分析,针对随机森林在生成过程中,节点分裂算法不同引起的随机森林性能不同的现象,提出了一种基于线性变换的随机森林优化算法,在将上述两种数据预处理的算法融合到新算法中后,进行了相应的 JAVA 程序实现,使用通用的 UCI 数据集,较好地实现

了随机森林算法的分类性能优化。

在完成上述两方面的优化后,本文就量化投资领域如何选择优质股票的应用进行了积极的探索,分析了选股问题的本质和完成股票选择所需要考虑的问题,使用价值成长投资决策的选股指标体系,将优化后的随机森林算法对股票数据进行了实证分析。

1.3.2 本文的结构

为了完成上述工作的内容,本文拟进行如下结构安排:

第一章 引言。这一章是本文的开头部分,介绍了随机森林的产生和发展历史背景等,并详细介绍了国内外三种随机森林算法优化研究方法,最后对本文的研究工作进行了较简要的概述。

第二章 随机森林研究综述。在这一章中,对随机森林算法的基本理论进行简单介绍,这些内容主要包括:随机森林算法的基础分类器——决策树,详细介绍了决策树的生成算法;分析了随机森林的构建过程和随机性;概述了随机森林的理论基础;详细分析了随机森林的性能指标,提出了随机森林算法存在的理论和应用上的问题,并介绍了几个实现随机森林的软件。

第三章 随机森林在处理非平衡数据集的优化研究。通过对非平衡数据现象的分析和对现有非平衡数据集解决方法的介绍,提出了一种改进的 SMOTE 抽样算法,并对该算法在随机森林算法分类精度的优化方面进行了实证分析。

第四章 随机森林处理连续变量的优化研究。本章通过对连续变量离散化问题的讨论和现有连续变量离散化的介绍,提出了一种改进的连续变量离散化方法,并对该算法在随机森林算法运行效率的优化方面进行了实证分析。

第五章 随机森林的分类性能优化研究。通过分析影响随机森林分类性能的主要因素,在此基础上提出了一种新的随机森林优化节点分裂算法,并对该算法在随机森林算法分类性能的优化方面进行了实证分析。

第六章 使用优化后的随机森林算法实现优质股票池的选择。将前三章提到的优化算法集合到一起,构成一个较完善的随机森林混合算法。使用该优化后的随机森林算法实现了优质股票池的选择。

2 随机森林研究综述

随机森林(RF)是一种组合分类器,它利用 bootstrap 重抽样方法从原始样本中抽取多个样本,对每个 bootstrap 样本进行决策树建模,然后将这些决策树组合在一起,通过投票得出最终分类或预测的结果。大量的理论和实证研究都证明了随机森林算法具有较高的预测准确率,对异常值和噪声具有很好的容忍度,且不容易出现过拟合。本章拟对随机森林的研究做一个系统的总结和整理,以利于后续的优化研究。

2.1 随机森林的基分类器——决策树

研究随机森林,就必须涉及到决策树,因为随机森林的基分类器就是没有剪枝的决策树。决策树是一种单分类器的分类技术。

2.1.1 决策树概述

决策树是一种典型的单分类器,该分类器的生成和决策过程分为三个部分,首先是通过训练集进行递归分析,生成一棵状如倒立的树状结构;然后分析这棵树从根节点到叶子节点的路径,产生一系列规则;最后,根据这些规则后,对新数据进行分类或预测。从本质上讲,决策树的分类思想其实是通过产生一系列规则,然后通过这些规则进行数据分析的数据挖掘过程。

决策树可视为一个树状模型,树中包括三种节点:根节点、中间节点、叶子节点。树中每个节点表示对象的属性,而从每个节点出发的分叉路径则代表的某个可能的属性值,而每个叶子节点则对应从根节点到该叶子节点所经历的路径所表示的对象的值。从根节点出发,经过若干中间节点后,到达叶子节点的路径表示某个规则,整个树表示由训练样本决定的规则的集合。决策树仅有单一输出,也就是从根节点出发,只能到达唯一的叶子节点,即规则是唯一的,这样就可以用于数据的分类和预测。

2.1.2 决策树生成过程中的节点分裂算法

决策树的生成时,主要采用一种程序递归的方式,从根节点开始,分成二棵子树,从子树开始,又继续产生根节点和左右子树,每棵子树继续递归生成新的子树,直到到达叶子节点为止。由根节点,产生左右子树时,需要比较不同属性分裂后的结果的优劣,选择最优的属性分裂产生左右子树,这个比较后分裂的过程称之为节点分裂。不同的比较规则就对应不同的决策树生成算法。决策树的生成算法很多,包括 CLS、ID3、C4.5、CART 等节点分裂算法。

1. CLS 算法

CLS 算法是早期决策树生成算法,该算法由 Hunt 等人提出,该算法的主要思想是:将数据集中的属性作为根节点和中间节点,生成一棵树状结构,从根节点到叶子节点就为一个规则,这是最早的决策树思想。在 CLS 算法中,节点分裂过程是随机的。

首先，从数据集所有的属性中随机选择一个属性作为根节点，然后从这个属性出发，开始分裂。分裂的过程根据属性的取值进行，有多少个属性的值域，就对应多少个分支。每个节点都按照这种办法产生新的分支，直到产生叶子结点时算法才终止执行。叶子结点的产生的规则是：当前的数据集是空的，或者是这个数据集中所有的样本都属于同一个类别。

为了详细分析 CLS 算法的执行过程, 分析如下一个例子。假设研究的数据集是人，每个样本的属性为眼睛、头发颜色以及人种，这个数据集的内容如下：

表 2.1 人种分类表

人员编号	眼睛颜色	头发颜色	所属人种
1	黑色	黑色	黄种人
2	蓝色	金色	白种人
3	灰色	金色	白种人
4	蓝色	红色	白种人
5	灰色	红色	白种人
6	黑色	金色	混血儿
7	灰色	黑色	混血儿
8	蓝色	黑色	混血儿

在选择第一个测试属性时，CLS 算法是没有确定给定，而是由程序随机给定，假设这里程序选择了眼睛颜色第一个测试属性，则将该属性标志为根节点，眼睛颜色这个属性共有三个取值，因此从这个根节点出发，分裂出三棵子树，树的结构如下：

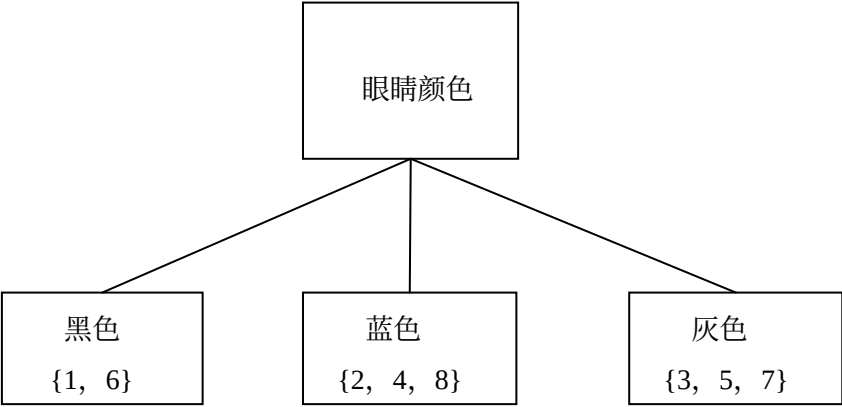


图 2.1 从眼睛颜色属性分裂成三棵子树

该属性将原数据集分裂成三个子集，分别为{样本 1，样本 6}（记为 A 子集）、{样本 2，样本 4，样本 8}（记为 B 子集）和{样本 3，样本 5，样本 7}（记为 C 子集）。逐个分析每个子集发现，A、B、C 三个子集中的样本的目标属性的值取值都不一样，这不符合叶子节点条件，分裂过程要继续下去，也就是说从这三个子集出发，需要继

续产生新的子树。这样程序继续进行上述过程，产生新的分支，最终产生一棵完整的决策树，树的结构如下图所示。

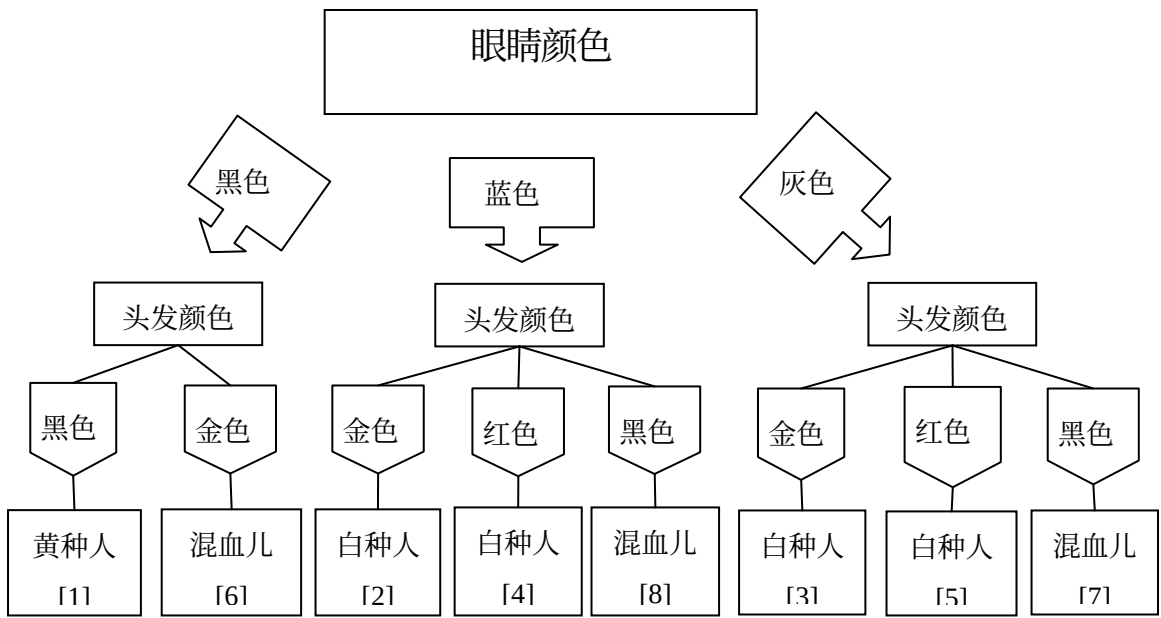


图 2.2 使用 CLS 产生的决策树的结构

从上面的分析可以看出，CLS 算法生成决策树的步骤如下：

Step 1 设置决策树 M 的根节点 (X, Q) ，其中 X 是当前训练集的集合， Q 是当前训练集的样本属性集合；

Step 2 若当前训练集的所有样本都属于相同的类别，或者训练集中样本属性集合为空，则产生叶子节点，算法结束，否则转 Step 3；

Step 3 从 Q 中随机选取一个属性，记为属性 B ，属性 B 的取值有 N 个，则 X 被 B 分为 N 个不相交的子集 $X_i, 1 \leq i \leq N$ ，这样就从属性 B 出发，产生了 N 个分支；

Step 4 设置每个分支的根节点，然后转 Step 2。

从算法的执行过程可以看出，按照 CLS 算法可以产生很多不同的决策树，这主要是因为在选择根节点时，CLS 算法没有一个固定的规则确定应该选择哪个属性作为根节点，这样的做法使得算法存在一定的盲目性，使算法的使用范围受到了限制，但该算法是决策树算法的起源，以后的优秀决策树算法都在它的继承和发展。

2. ID3 算法

针对 CLS 没有规定采用何种测试属性带来算法的不确定性，1986 年 Quinlan 提出了著名的 ID3 算法，和 CLS 算法的显著不同是，ID3 算法在选择属性时都遵守一个固定的规则，这个规则就是通过引入信息熵对属性进行比较，通过比较信息熵的大小，来决定哪个属性作为分裂节点。

信息熵源于信息论，信息论是由香农在 1948 年提出，主要是通过数学的办法解

决了信息通信问题。信息论中最重要的概念是信息熵,对于离散信源而言,信源概率空间的概率分别为 $P(u_i)$, $i=1,2,\dots,r$, 当概率 P 越小,则消息 u_i 出现的概率就越小,一旦概率小的消息 u_i 出现,则所获得的信息量就越大。用自信息量反应消息 u_i 传递的信息量的大小,其公式为:

$$I(u_i) = \log[1/P(u_i)] \quad (2.1)$$

由于自信息量具有随机变量的性质,但自信息量不能表示信源总体的不确定程度,因此引入自信息量的数学期望——信息熵,它是消息 u_i 发生后所含有的信息量(自信息量)的数学期望。由于引入了数学期望的概念,使得信息熵就可以度量全体信源 X 的不确定性。它是在发出信息前信源不确定性的平均值。信息熵的定义公式如下:

$$E(X) = P(u_1)I(u_1) + P(u_2)I(u_2) + \dots + P(u_r)I(u_r) = -\sum_{i=1}^r P(u_i) \log_{\alpha} P(u_i) \quad (2.2)$$

其中: $u_1, u_2, \dots, u_r$ 为每个消息, r 消息的个数;

α 是信息熵的单位,一般取值为 2;

在公式 (2.2) 中,当 $p(u_i) = 0$ 时, $p(u_i) \log_{\alpha} P(u_i) = 0$ 。

ID3 算法是以属性取值对应的信息熵的大小作为选择节点分裂标准的。该算法由信息熵的计算出发,计算每个属性的信息增益率,然后使用信息增益的大小来比较选择哪个属性进行节点的分裂。它的核心思想是,如果一个属性的信息增益最小,那么从该属性产生的节点出发,到达到子树的平均路径是最短的,这可以使产生的决策树的平均深度比随机选择要小得多。因此 ID3 算法的关键是要计算每个属性的信息增益,信息增益的计算过程如下:

(1) 计算节点的信息熵

$$Info(D) = -\sum_{i=1}^m P_i \log_2 P_i \quad (2.3)$$

(2) 基于按 A 划分对 D 的样本分类所需要的期望信息,其公式如下:

$$Info_A(D) = \sum_{j=1}^v \left[\left(\frac{|D_j|}{|D|} \right) * Info(D_j) \right] \quad (2.4)$$

其中, $\frac{|D_j|}{|D|}$ 充当第 j 个划分的权重。 $Info_A(D)$ 越小,划分的纯度越高。

(3) 由公式 (2.3) 和 (2.4) 可得信息增益,定义式为:

$$Gain(A) = Info(D) - Info_A(D) \quad (2.5)$$

$Gain(A)$ 表示 A 的值而导致的信息需求的期望减少,即信息增益。

ID3 算法就是选择使 $Gain(A)$ 最大的属性作为测试属性。此外, ID3 中还引入了窗口方法进行增量学习来解决实例过大的问题。ID3 算法的步骤如下:

Step 1 取得训练集 X , 若当前训练集的所有样本都属于相同的类别,或者训练

集中样本属性集合为空，则产生叶子节点，算法结束，否则转 Step 2；

Step 2 根据训练集 X ，计算 Q 中所有属性的信息增益

Step 3 选择 Q 中信息增益最大的属性 B 为分裂属性，如果属性 B 的取值有 N 个，则 X 被 B 分为 N 个不相交的子集 $X'_i, 1 \leq i \leq N$ ，这样就从属性 B 出发，产生了 N 个分支；

Step 4 设置每个分支的根节点，然后转 Step 1。

ID3 算法弥补了 CLS 算法随机选择属性的不足，使得在生成决策树时，产生的规则具有固定性和可重现性，且使最终产生的决策树的深度也很小。但它也有不可克服的问题，主要体现在二点，一是它不能处理连续性变量，二是在进行属性选择时，选择指标信息增益会偏向取值较多的属性，这个偏离会导致决策树的局部最优解问题。

3.C4.5 算法

针对 ID 算法的不足，Quinlan 在 1993 年提出了 C4.5 算法，该算法是 ID3 算法的改进。C4.5 算法在选择节点分裂属性时，也和 ID3 算法一样，需要按照固定的规则进行，针对 ID3 算法中信息增益会产生偏离的问题，C4.5 算法引入分裂信息指标，计算信息增益率，通过比较信息增益率的大小来确定属性是否被选择。

C4.5 算法针对 ID3 算法的改进，主要包括如下几个方面：

(1) 针对信息增益指标容易产生多值偏向问题，C4.5 算法引入分裂信息比率指标，该指标的计算公式如下：

$$SplitInfo_A(D) = - \sum_{j=1}^v \left[\left(\frac{|D_j|}{|D|} \right) * \log_2 \left(\frac{|D_j|}{|D|} \right) \right] \quad (2.6)$$

其中 v 为属性 A 的取值个数。

$SplitInfo_A(D)$ 是按照属性 A 分裂时，训练集 S 的均匀性的代表，将这个指标和信息增益指标进行对比，就可以使得选择的属性比较均匀，而不会产生偏向性问题。这样，信息增益率的计算公式为：

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)} \quad (2.7)$$

(2) 可以对连续的数值型属性进行处理。C4.5 使用一种二分离散化方法，通过将原有连续属性变量离散化，然后对离散化后的数据集进行计算，其最后还是归结到离散化的角度来解决这个问题，但是在离散化的过程中 C4.5 算法表现有些直接，会影响算法的性能。因此这也是后续要改进的内容。

(3) 能够对不完整数据进行处理。在实际的应用中，可供使用的数据集不可能是完整无缺的，或多或少会有一些属性的值出现错误的或空的值，这种情况称为不完整数据。处理缺失数据在统计学中有很多办法，在 C4.5 算法中，选择使用概率分布

来进行缺失值的补齐。使用补齐后的数据集，就可以进行决策树的生成了。

算法的具体执行步骤如下：

Step 1 遍历初始数据集中的全部数据，进行数据的预处理。预处理包括两个部分，一是连续变量离散化处理，二是缺失数据补齐。如果数据集中没有连续变量，也没缺失数据，则进入 Step 2；

Step 2 判断当前训练集是否符合算法结束的条件，该条件和 ID3 算法结束条件一致。如果符合，则算法结束，否则转 Step 3；

Step 3 分别取出训练集中每个属性，根据属性的取值和目标属性的取值，按照公式 (2.5) 计算每个属性的信息增益 $\text{Gain}(A)$ 和公式 (2.7) 计算每个属性的信息增益率 $\text{GainRatio}(A)$ ；

Step 4 选择信息增益率最大的属性作为分裂节点，将训练集分为多个子集，子集个数由被选择的属性的值的个数决定，然后对每个子集转 Step 2。

C4.5 算法克服了 ID3 算法的诸多缺点，使得决策树的生成过程更加合理，提升了算法的分类精度。但由于该算法在执行时，需要对数据集进行反复遍历，因此使得算法的执行时间较长，另外，算法在比较的过程中，一般都是将数据集全部装入内存，因此算法的空间复杂度也比较高。

4 CART 算法

1984 年，L.Breiman 和 C.Stone 等人提出了 CART 算法，该算法以信息熵为理论基础，继承了 CLS 算法的递归构建决策树产生分类规则的思想。它和 ID3 算法和 C4.5 算法存在一定差距，主要差距表现在它在进行节点分裂时，采用的分裂规则是 Gini 指标最小原则。

CART 算法将分裂属性的取值划分为两个子集，然后从这两个子集出发，计算由训练集决定的 Gini 指标，然后采用二分递归的方式，将当前训练集分成两个子集，从而产生左右两个分枝的子树。当节点发生分裂时，该算法使用 Gini 指标来度量数据划分，其计算过程如下：

(1) 计算样本的 Gini 系数

$$\text{Gini}(S) = 1 - \sum_{i=1}^m P_i^2 \quad (2.8)$$

其中 P_i 代表类别 C_j 在样本集 S 中出现的概率。

(2) 计算每个划分的 Gini 系数

如果 S 被分隔成两个子集 S_1 与 S_2 ，则此次划分的 Gini 系数为

$$\text{Gini}_{\text{split}}(S) = \frac{|S_1|}{|S|} \text{Gini}(S_1) + \frac{|S_2|}{|S|} \text{Gini}(S_2) \quad (2.9)$$

和 C4.5 算法一样，CART 算法可以较好地处理离散属性变量，但在处理连续属性变量时，也存在连续变量离散化的问题，在进行连续变量离散化后，连续变量就转变

为离散变量。对于离散变量，CART 算法在节点分裂时，需要使用公式 (2.9) 计算每个变量的 GINI 指标，然后根据 GINI 指标最小的原则，选择一个变量进行分裂，最后通过递归的形式，进行决策树的构建，从而产生分类规则。

在 CART 算法，叶子节点产生的规则如下：

- (1) 当前数据集的样本个数小于某个给定的值；
- (2) 当前数据集的样本都属于同一类；
- (3) 当前数据集的属性变量为 0；
- (4) 决策树的深度大于用户设定的值。

2.1.3 决策树分类中存在的问题

由于决策树采取单一的分类器决策模式，因此有其不可克服的缺点：

1. 分类规则复杂。决策树算法在产生规则的时候采用了局部的贪婪方法，每次只选取一个属性进行分析构造决策树，所以它们产生的分类规则往往相当复杂，针对此问题一般使用决策树剪枝的办法实现。决策树剪枝是决策树优化的一个重要研究方向，相关的方法也很多，这里不一一赘述。

2. 收敛到非全局的局部最优解。ID3 算法在搜索中不进行回溯(每当在树的某一个层次选择了一个属性进行测试，它不再回溯重新考虑这个选择)，所以它容易受盲人登山中常见的风险影响，仅仅收敛到非全局的局部最优解。

3. 过度拟合。在决策树的学习中，由于分类器过于复杂，则它可能会过于适应噪声，从而导致过拟合的问题。过拟合对于决策树和其他很多学习算法来讲是一个很难克服的难题。

2.2 随机森林的构建过程

为了克服决策树的上述缺点，结合单个分类器组合成多个分类器的思想，一种很容易被想到的办法就是生成多棵决策树，这些决策树不需要都有很高的分类精度，并让所有的决策树通过投票的形式进行决策，这就象多个专家一起开会讨论，最后举手表决一样，这就是随机森林核心的思想。构建随机森林主要包括以下三个步骤：

2.2.1 为每棵决策树抽样产生训练集

每一棵决策树都对应一个训练集，要构建 N 棵决策树，那就需要产生对应数量的训练集，从原始训练集中产生 N 个训练子集就涉及到统计抽样技术。现有的统计抽样技术有很多，按照抽样是否放回主要包括以下二种：

1. 不放回抽样

设一个总体含有 N 个个体，从中不放回地抽取 n 个个体作为样本 ($n \leq N$)，叫不放回抽样。如果每次抽取使总体内的各个个体被抽到的机会都相等，就把这种抽样方法叫做简单随机抽样。在产生随机数时，包括抽签法和随机数法两种。一是抽签法。该方法就是把总体中的 N 个个体编号，把号码写在号签上，将号签放在一个容器中，

搅拌均匀后，每次从中抽取一个号签，连续抽取 n 次，就得到一个容量为 n 的样本。抽签法简单易行，适用于总体中的个数不多的情况。当总体中的个体数较多时，将总体“搅拌均匀”就比较困难，用抽签法产生的样本代表性差的可能性很大。二是随机数法。即利用随机数表、随机数骰子或计算机产生的随机数进行抽样。采用不放回抽样的特点是每次抽样时，原数据集的数量会减少，同时产生的抽样样本不会重复。

2. 有放回抽样

有放回抽样即在抽取样本时，采用抽取后再将抽取结果放回的抽样方法，这样每次抽样时数据集的大小不变，在抽取的样本集中出现重复样本的情况。根据抽样时是否设置权重又分为无权重抽样和有权重抽样。

(1) 无权重抽样，也叫 bagging 方法，是一种用来提高学习算法准确度的方法。该方法于 1996 年由 Breiman^[37] 根据 Boosting 技术^{[38] [39]} 提出的。bagging 法是以可重复的随机抽样为基础的，每个样本是初始数据集有放回抽样。在可重复抽样生成多个训练子集时，存在于初始训练集 D 中的所有的样本都有被抽取的可能，但在重复多次后，总有一些样本是不能被抽取的，每个样本不能被抽取的概率为 $(1-1/N)^N$ ，这里 N 的是原始训练集中样本的个数。

(2) 更新权重抽样

还有一种方法就是 boosting 方法，也叫更新权重抽样。Boosting 方法进行抽样时，首先随机抽样产生一组训练集，然后对这组训练集中的每一个训练集设定权重为 $1/n$ ， n 为训练集中样本的个数，在设定权重后，对每个带权重的训练集进行测试，在每次测试结束后，对分类性能差的训练集的权重进行提升，从而产生一个新的权重系列，经过多次训练后，每个训练集就有一个和其相对应的权重，在投票时，这些权重就可以对投票的结果产生影响，从而影响最终的决策结果。

Bagging 方法与 Boosting 方法都是可放回抽样的抽样方法，但两者间存在很大的区别，主要体现在两个方面：一是 Bagging 方法在训练的过程中采用独立随机的方式，而 Boosting 方法在训练的过程中，每一次训练都是在前一次的基础上进行的，因此是一种串行的关系，这对算法的执行过程是一个很大的挑战，因为每次执行都需要等待上次的结果才能继续，而 Bagging 方法就不存在这个问题，这为算法的并行处理提供了很好的支持；二是的 Bagging 方抽取出来的训练集都是没有权重的，各训练集的待遇是相同的，而 Boosting 方法在抽取的过程中，对每个训练集都设定权重，使得抽取结束后，每个训练集的待遇是不一致的。

随机森林算法在生成的过程中，主要采用 bagging 抽样技术从原始训练集中产生 N 个训练子集，每个训练子集的大小约为原始训练集的三分之二，每次抽样均为随机且放回抽样，这样使得训练子集中的样本存在一定的重复，这样做的目的是为了森林中的决策树不至于产生局部最优解。

2.2.2 构建每棵决策树

算法为每一个训练子集分别建立一棵决策树,生成 N 棵决策树从而形成“森林”,每棵决策树任其生长,不需要剪枝处理。其中涉及两个重要过程:

1. 节点分裂

节点分裂是算法的核心步骤,通过节点分裂才能产生一棵完整的决策树。每棵树的分支的生成,都是按照某种分裂规则选择属性,这些规则主要包括信息增益最大、信息增益率最大和 Gini 系数最小等原则,不同的规则对应不同的分裂算法。在节点分裂时,将每个属性的所有划分按照规则指标进行排序,然后按照规则选择某个属性作为分裂属性,并按照其划分实现决策树的分支生长。

在第一节的小节中,已经给出了决策树的生成算法,对于不同的算法,节点分裂的规则计算的指标也不尽相同,比如 CART 算法采用的是 Gini 系数, C4.5 算法采用的是信息增益率。不同的节点分裂算法产生的随机森林的性质也不尽相同,现行的统计软件在选择节点分裂原则时,一般都是在 CART 算法和 C4.5 算法中选择其中一个来进行节点分裂。

2. 随机特征变量的随机选取

随机特征变量是指随机森林算法在生成的过程中,参与节点分裂属性比较的属性个数。由于随机森林在进行节点分裂时,不是所有的属性都参与属性指标的计算,而是随机地选择某几个属性参与比较,参与的属性个数就称之为随机特征变量。随机特征变量是为了使每棵决策树之间的相关性减少,同时提升每棵决策树的分类精度,从而提升整个森林的性能而引入的。其基本思想是,在进行节点分裂时,让所有的属性按照某种概率分布随机选择其中某几个属性参与节点的分裂过程。在随机森林算法中,随机特征变量的产生方法主要有两种办法,一个是随机选择输入变量(Forest-RI),另一个是随机组合输入变量(Forest-RC)。

Forest-RI 是对于输入变量随机分组(每组变量的个数 F 是一个定值),然后对于每组变量,利用 CART 方法产生一棵树,并让其充分生长,不进行剪枝。在每个节点上,对输入该节点的变量,重复前面的随机分组,再重复 CART 方法,直到将所有节点均为叶子节点为止。一般 F 有两种选择,首先是 $F=1$,其次取 F 为小于 $\log_2 M + 1$ 的最大正整数,其中 M 是输入变量的个数。假如只有很少的输入变量,比如 M 值不大,用 Forest-RI 法从 M 中随机选择 F 个作为随机特征,这样可能提高模型的强度,但同时也扩大了相关系数。

Forest-RC 是先将随机特征进行线性组合,然后再作为输入变量来构建随机森林的方法。随机选择 L 个输入变量进行线性组合得到新的特征(不同的 L 值对应不同特征)。在每个节点上,随机选出 L 个变量 $v_1, v_2, \dots, v_L$ 及 L 个随机数 k_i , 做线性组合得到变量 v , 公式如下:

$$v = \sum_{i=1}^L k_i v_i, k_i \in [-1, 1] \quad (2.10)$$

一般地，对于给定的集合 M ，具有 $O(M^L)$ 种不同的输入变量的组合。

最常用的随机森林算法都是使用 Forestes-RI 方法构建，在每棵子树的生长过程中，不是将全部 M 个输入变量参与节点分裂，而是随机抽取指定 F ($F \leq M$) 个随机特征变量， F 的取值一般为 $\log_2 M + 1$ ，以这 F 个属性上最好的分裂方式对节点进行分裂，从而达到节点分裂的随机性。

2.2.3 森林的形成及算法的执行

重复上述两个步骤建立大量的决策树，就生成了随机森林。算法最终的输出结果采取大多数投票法实现。根据随机构建的 N 棵决策子树将对某测试样本进行分类，将每棵子树的结果汇总，所得票数最多的分类结果，将作为算法最终的输出结果。

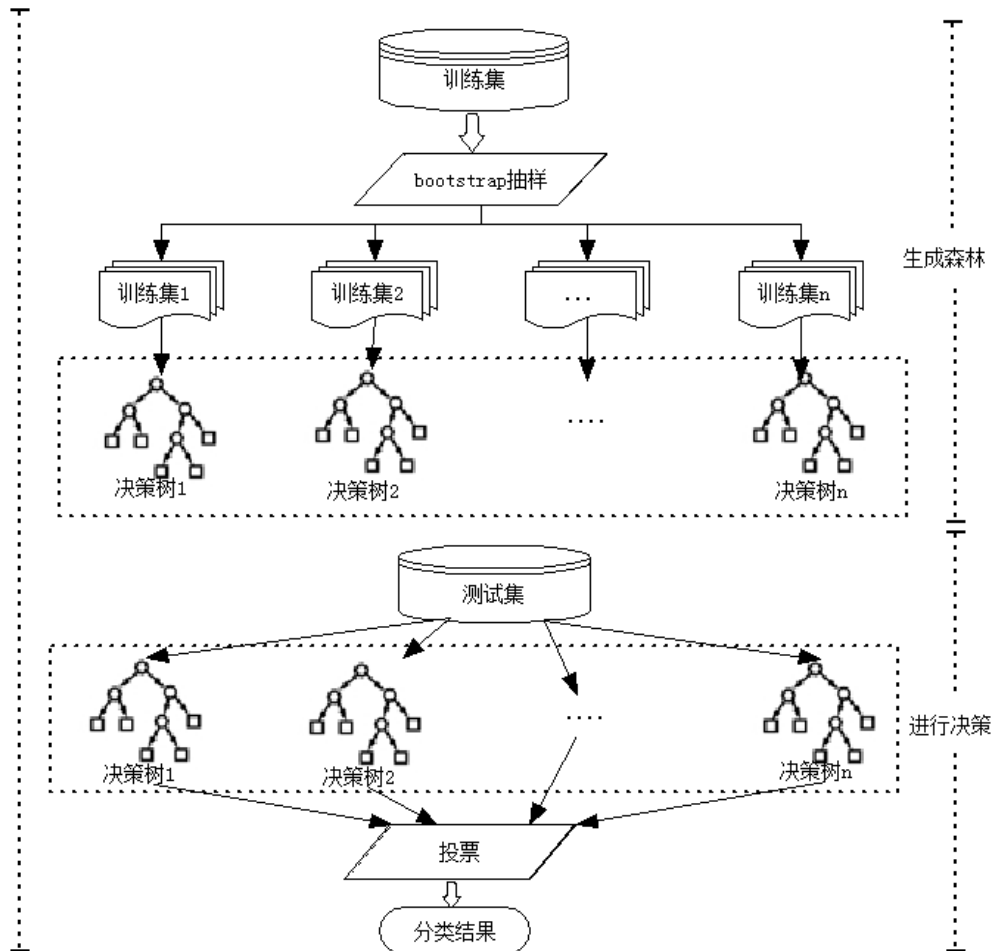


图 2.3 随机森林算法示意图

2.3 随机森林的随机性分析

从随机森林的构建过程可以看出，该算法的核心思想是：将存在过度拟合和局部收敛问题的单个分类器决策树，组合成多个分类器森林。那么，如何构建由多棵决策

树组成的森林，命名该森林能有效地避免了决策树会陷入过度拟合和局部最优的缺点，这就需要在构建森林时，处处体现随机性。从森林的构建过程可以归纳出以下随机性：

2.3.1 训练集的随机选取

根据 Breiman 的观点，随机森林在抽样生成训练集时，采用 bagging 方法，该方法是一种有放回的随机抽样，这种抽样方式产生的训练集较初始训练集存在一些不同。最大的不同体现在新的数据集不是原始数据集的简单复制，而是通过自身样本的重复而达到样本空间的重构，由于新产生的数据集都是原始数据集的子集，因此各个新的数据集间也存在着差异，从而使产生的森林也存在着一定的差异，这种差异最后体现在森林中决策树生长过程中的随机性。而这种随机性又是有条件的，可以看作是一种有条件的概率抽样，从而使得到的随机森林不至于过于发散，也不至于趋向局部的最优解。

随机森林使用 bagging 方法的有两个原因。一方面可以提高随机森林算法的精度。由于在训练集上采用放回抽样法采样，平均 37% 的样本不会出现在采集的样本集合中，这样做就可以有效地避免“异常点”出现在样本集合中。因此，与在原始数据上构建分类器相比，使用 bagging 方法采集的样本集合上更容易得到好的分类器。另一方面还可以使用这些没有被抽取的数据集来估计森林中决策树的泛化误差 (PE^*) 以及强度 (s) 和相关系数 (ρ)，使算法分类精度可以量化度量。

2.3.2 随机特征变量的随机性

随机森林算法为每一个训练子集分别建立一棵决策树，生成 N 棵决策树从而形成“森林”，每棵决策树任其生长，不需要剪枝处理。其中涉及的一个重要过程就是随机特征变量 F 的选取，这个过程增加了决策树构建时的随机性。

在进行节点属性选择时，并不是全体属性都参与选择的过程，而是从全体属性中随机的选择 F 个属性进行节点属性的选择过程，这里 F 的值即为随机特征变量。随机特征变量是随机森林算法为了提高分类精度，减小森林中树之间的相关系数，而引入的一种随机性。由于为随机特征变量的引入，使在每棵子树的生长过程中，不是将样本的全部 M 个属性参与节点分裂，而是随机抽取 F ($F \leq M$) 个属性，比较这 F 个属性上分裂规则最优的属性对节点进行分裂。由于抽取的 F 个属性具有随机性，从而使得每棵决策树在生成的过程中，都有区别的其他的决策树，这就为随机森林引入了另一个随机因素。

2.3.3 随机森林的随机性在实证分析中的表现及解决办法

从上面的两个随机性可以看出，随机森林的随机性体现在产生决策树的训练样本是随机的，树中节点分裂时选择的最优属性也是随机的。有了这两点随机性，保证了随机森林就不会产生过拟合的现象了，但是这两种随机性这也使得算法的分类精度在同样的参数下表现出一定的波动性，然而这种波动由于算法的收敛性使得分类精度仅

在很小的范围内变动。正是由于这种微小波动的存在,使得在进行随机森林实证分析时,每次观察的结果存在一定的差异,为了解决这个问题,在实验的过程,均要在相同的参数下重复随机森林算法的执行次数,然后取分类精度的平均值进行观察。在本文的后续内容中,为了解决随机森林的内在随机性现象,在分析随机森林算法分类精度时,均采用重复多次实验,取多次实验的算法分类精度的均值进行比较。

2.4 随机森林的理论概述

2.4.1 随机森林的数学定义

设一系列决策树 $h_1(x), h_2(x), \dots, h_k(x)$ 构成的森林和两个随机向量 X, Y 。

定义 1 边缘函数 (Margin Function)

$$mg(X, Y) = av_k(I(h_k(X) = Y)) - \max_{j \neq Y} av_k(I(h_k(X) = j)) \quad (2.11)$$

其中: $I(\cdot)$ 为示性函数;

Y 为正确的分类向量;

j 为不正确的分类向量;

$av_k(\cdot)$ 表示取平均。

边际函数表示在正确分类的得票数超过不正确分类的最大得票数的程度。显然,边缘函数越大,表明分类器的置信度越高。

定义 2 泛化误差

$$PE^* = P_{X,Y}(mg(X, Y) < 0) \quad (2.12)$$

其中,下标 X, Y 表明了概率的定义空间。

定义 3 随机森林边缘函数为

$$mr(X, Y) = P(h_k(X) = Y) - \max_{\substack{j \neq Y \\ j=1}}^c P(h_k(X) = j) \quad (2.13)$$

其中: $P(h_k(X) = y)$ 为判断正确的分类的概率

$\max_{\substack{j \neq Y \\ j=1}}^c P(h_k(X) = j)$ 为判断错误的其他分类的概率的最大值,即目标变量具有二

个或二个以上的分类。

对于每一棵决策树,在随机森林构建的过程中,对应一个初始数据集和一个没有抽取的数据集,记没有抽取的数据集为 $O_k(x)$ 。 $Q_k(x, y_j)$ 为对于输入的随机向量 x 在

$O_k(x)$ 中投票的分类类别为 y_j 的比例,则

$$Q(x, y_j) = \frac{\sum_k I(h_k(x) = y_j, (x, y) \in O_k(x))}{\sum_k I(h_k(x), (x, y) \in O_k(x))} \quad (2.14)$$

其中：

分子为每棵决策树和其对应的没有抽取的数据集的正确分类个数之和

分母为所有没有抽取的数据集的样本个数之和

将 $Q(x, y_j)$ 作为随机森林正确分类的概率的估计，则就可以考察随机森林强度和
相关度两个变量。

定义 4 随机森林 $\{h(X)\}$ 的强度为随机森林的边缘函数的期望，即：

$$s = E(mr(X, Y)) = E(P(h_k(X) = Y) - \max_{\substack{j \neq Y \\ j=1}}^c P(h_k(X) = j)) \quad (2.15)$$

将公式 (2.14) 代入到公式 (2.15)，则得到分类器的强度估计为：

$$s = \frac{1}{n} \sum_{i=1}^n \left(Q(x_i, y) - \max_{\substack{j \neq Y \\ j=1}}^c Q(x_i, j) \right) \quad (2.16)$$

定义 5 随机森林中各棵树之间的平均相关度定义为边缘函数的方差除以森林的
标准差的平方，即：

$$\bar{\rho} = \frac{\text{var}(mr)}{sd(h(*))^2} = \frac{\frac{1}{n} \sum_{i=1}^n \left(Q(x_i, y) - \max_{\substack{j \neq Y \\ j=1}}^c Q(x_i, j) \right)^2 - s^2}{\left(\frac{1}{k} \sum_{u=1}^k \sqrt{p_u + \bar{p}_u + (p_u - \bar{p}_u)^2} \right)^2} \quad (2.17)$$

其中；

p_u 作为 $p(k_u(x_i) = y)$ 的 OOB 估计；

$\bar{p}_u$ 作为 $p(k_u(x_i) = \bar{y}_j)$ 的 OOB 估计；

它们的公式如下：

$$p_u = \frac{\sum_{(x_i, y) \in O_x} I(k_u(x_i) = y)}{\sum_{(x_i, y) \in O_x} I(k_u(x_i))} \quad (2.18)$$

$$\bar{p}_u = \frac{\sum_{(x_i, y) \in O_x} I(k_u(x_i) = \bar{y}_j)}{\sum_{(x_i, y) \in O_x} I(k_u(x_i))} \quad (2.19)$$

公式 (2.19) 中 $\bar{y}_j$ 为在训练集中使 $Q(x, y_j)$ 估计值最大的与 y 类别不同的类别。

$$\bar{y}_j = \arg \max_{\substack{c \\ j \neq y}} Q(x, y_j) \quad (2.20)$$

2.4.2 随机森林的性质

1. 随机森林的收敛性

定理 1 根据定义 1、定义 2、大数定律和决策树的结构可知，所有森林中的决策树的泛化误差都收敛于如下表达式^[6]

$$\lim_{n \rightarrow \infty} PE^* = P_{xy} (P_{\Theta} (k(X, \Theta) = Y) - \max_{j \neq Y} P_{\Theta} (k(X, \Theta) = j) < 0) \quad (2.21)$$

其中 n 为森林中树的数目。

定理 1 表明随着树的增加，泛化误差 PE 将趋向一个上界，这表明随机森林对未知的实例有很好的扩展性，也就是说具有良好的防止过拟合的能力。这说明了为什么随机森林算法不会随着决策树的增加而产生过度拟合的问题，但要注意的是可能会产生一定限度内的泛化误差。

2. 森林中决策树的相关度和强度影响算法的泛化误差

定理 2 泛化误差的一个上界^[6]

$$PE^* \leq \frac{\bar{\rho}(1-s^2)}{s^2} \quad (2.22)$$

其中： $\bar{\rho}$ 为决策树之间的相关度的平均值；

s 是决策树的平均强度。

从定理 2 可以看出要使随机森林的泛化性能好，则应该尽量减小决策树之间的相关性，增大单棵树的分类性能。虽然这个上界比较松散，但是在评价一个随机森林的性能时可以据此作为参考依据。相关度和强度之间的相互影响也可以通过这个上界得到体现。由决策树形成森林后，完成了单分类器到多分类器的转变，整个森林的性能自然受每棵决策树的性能决定，每棵树的分类强度越大，则随机森林的分类性能越好。森林中树之间的相关度（Correlation）越大，则随机森林的分类性能越差。

2.5 随机森林的性能指标

随机森林分类性能受内外两方面因素影响，从外部因素看，主要来自训练样本的情况，包括训练样本的正负类样本分布，即训练样本的平衡；训练样本的规模，即样本的大小、样本的变量个数及变量的类型。从内部因素看，主要包括单棵树的分类强度和树之间的相关度。衡量随机森林性能的指标三种，一是分类效果指标，二是泛化误差，三是运行效率。

2.5.1 分类效果系列指标

随机森林算法主要用于分类和预测，那么分类效果就是其主要的考核项目，为了较好地分析随机森林的分类效果，该类指标的内容较多。为了描述这些指标，考虑如下二分类数据的混淆矩阵。

表 2.2 二分类数据的混淆矩阵

	Classified positive	Classified negative
Positive	TP	FN
negative	FP	TN

表 2.2 是两分类数据集的混淆矩阵。假设数据集中有两个分类，分别叫正类 (positive) 和负类 (negative)，对应表中的列坐标，TP 和 TN 分别代表正确分类的正类和负类的样本数量；FN 和 FP 分别是错误分类的正类和负类的样本数量。

随机森林分类器的分类效果的评价指标由以下定义给出。

定义 6: 分类精度 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.23)$$

这个指标是用来衡量随机森林对测试集的总体分类精度，一般而言总体的分类精度越高，算法的分类效果越好。

定义 7: 灵敏度 (Sensitivity)，记为 Acc (+)。代表分类模型对正类的分类精度，计算公式为：

$$\text{Sensitivity} = TP / (TP + FN) \quad (2.24)$$

定义 8: 特异度 (Specificity)，记为 Acc (-)。代表分类模型对负类的分类精度，计算公式为：

$$\text{Specificity} = TN / (FP + TN) \quad (2.25)$$

定义 9: 几何均值 G-mean

$$G - \text{mean} = \sqrt{[TP / (TP + FN)] \times [TN / (FP + TN)]} \quad (2.26)$$

G-均值 (G-mean) 是 Acc (+) 灵敏度和 Acc (-) 特异度的几何平均，其设计准则是保持正负两类分类精度平衡条件下的总体精度的最大化。若正类的分类精度较高但负类较低，则 G-均值较低。只有当两者都较高时，G-均值才会较高。几何均值 G-mean 是非平衡数据集学习中常用的评价指标。

定义 10: 负类的查全率，记为 Recall，表示被正确分类的负类样本占有所有真实负类样本的比例。公式如下：

$$\text{Recall} = TP / (TP + FN) \quad (2.27)$$

定义 11: 负类的查准率，记为 Precision，表示被正确分类的负类占有所有预测为负类的样本的比例。公式如下：

$$\text{Precision} = TP / (TP + FP) \quad (2.28)$$

定义 12:负类检验值 (F-Value), 是从负类的角度出发综合评价随机森林性能的又一个综合评价指标。它是负类的查全率 (Recall) 和查准率 (Precision) 的组合。其公式如下:

$$F-value = \frac{(1 + \beta^2) \cdot recall \cdot precision}{\beta^2 \cdot recall + precision} \quad (2.29)$$

其中 β 的取值范围是 (0, 1], 其值根据实际情况取值, 通常取值为 1。负类检验值 (F-Value) 也是非平衡数据集学习中有效的评价指标。

2.5.2 泛化误差与 OOB 估计

1. 泛化能力

机器学习的目的是学到隐含在数据背后的规律, 对具有同一规律的学习集以外的数据, 经过训练的分类器也能给出合适的输出, 该能力称为泛化能力。泛化能力是指经过训练后的模型对未在训练集中出现的 (但来自同一分布的) 样本作出正确反映的能力, 概括地说, 所谓泛化能力 (generalization ability) 是指机器学习算法对新鲜样本的适应能力。

2. 泛化误差

泛化误差 (Generalization Error) 是反应泛化能力的一个指标, 泛化误差越小, 则该学习性能越好, 反之则性能越差。按照定义 2, 随机森林的泛化误差从理论上是可以计算出来的, 然而, 在实际环境中, 样本的期望输出和分布情况都是未知的, 无法直接通过计算泛化误差来评估随机森林的泛化能力。目前, 主要有两种方法用于估计分类器的泛化误差, 一种是分析模型 (Analytical Model), 而另一种就是交叉验证 (Cross-Validation, CV) 的方法。分析模型的一个主要问题是很难对随机森林的有效参数个数做出很好的估计, 因此只能对一些简单的单一性线性分类问题比较有用^[42]。

而交叉验证的方法是通过将标注的样本划分为训练集和验证集, 由于验证集中的样本目标变量的输出值已经给出, 因此可以通过验证集来估计泛化误差。而在交叉验证中, 一般采用 k-fold 交叉验证的方法, 如果样本集合比较大, 那将需要很大的时间开销。

3. OOB 估计

在随机森林算法中, 有一种比较好的估计泛化误差的方式, 那就是使用 OOB 估计。如前所述, 随机森林是使用 bagging 方法进行训练集的生成的, 在生成这些数据集时, 初始的训练集中有一些样本是不能被抽取的, 这些样本的个数是初始数据集的 $(1 - 1/N)^N$ (其中 N 为初始训练集中样本的个数)。可以证明, 当 N 足够大时, $(1 - 1/N)^N$ 将收敛于 $1/e \approx 0.368$, 这个数据说明, 有将近 37% 的样本不会被抽出来, 这些不能从初始数据集中抽取出来的样本组成的集合, 称之为袋外数据, 简记为 OOB。使用 OOB 数据来估计随机森林算法的泛化能力, 称之为 OOB 估计。以每一棵决策树为单位, 利用未被该森林选中的所有的训练样本点的集合, 统计该树的 OOB 误分率; 将所有树

的误分率取平均得到随机森林的 OOB 误分率，就可以得到一个 OOB 误差估计。

交叉验证(CV)也可以用来估计随机森林的泛化误差，但是交叉验证进行估计时，因为要进行数据集的划分和合并处理，这会导致在算法的运行过程中，需要进行很大的运算，使得算法的时间复杂度和空间复杂度会过高，从而使算法的运行效率降低。这一点在使用 OOB 估计时是不会产生的，因为 OOB 数据是在随机森林算法生成的过程中产生的，因此 OOB 估计就可以在每棵决策树生成时同时计算出 OOB 误差率，最终只需要使用很少的资源就可以得到随机森林算法的泛化误差估计。因此，和交叉验证方式相比，OOB 估计的效率是很高的，大量的实验数据表明，两者的估计效果是差不多的，因此 Wolpert 等人在文献中也建议，随机森林泛化误差的估计使用 OOB 估计方法就可以了。在本文的后续章节中，如果没有特别的说明，随机森林算法的性能都使用 OOB 估计来评价，OOB 估计越小，说明算法的性能越好。

Breiman 通过实验已经证明，OOB 估计是随机森林的泛化误差的一个无偏估计。Breiman 研究了装袋分类模型(bagged classifier)的误差估计，利用实际例子证明了使用 OOB 估计和使用相同样本容量的测试集的精度一样，因此他认为使用 OOB 估计的话，就没有必要再使用测试集。使用 OOB 数据还可以估计强度和相关系数。它提供了一个内部估计，从而有助于理解分类精度以及如何提高精度。

2.5.3 随机森林算法运行效率指标

算法运行效率一般使用算法复杂度来考量，算法的复杂性体现在运行该算法时的计算机所需资源的多少上，计算机资源最重要的是时间和空间（即寄存器）资源，因此算法复杂度分为时间和空间复杂度。

1. 时间复杂度

时间复杂度，它是指执行算法所需要的计算工作量，是算法在执行的过程中计算机所花费的时间。从通常情况下，算法运行过程中所花费的时间，是无法计算的，必须通过在计算机上运行才能计算出来。但如果知道算法的详细代码，也可以通过分析算法中循环语句的多少来进行衡量，从理论上讲，算法运行耗费的时间与该算法中程序语句的重复执行次数成正比，也就是说在算法的语句需要重复执行次数越多，那么该算法所耗用的时间就越多。

算法的时间复杂度记做： $T(n)=O(f(n))$ ，考虑如下的程序代码

```
for (i=1;i<=n;++i)
{
    for (j=1;j<=n;++j)
    {
        c[i][j]=0;
        for (k=1;k<=n;++k)
            c[i][j] += a[i][k]*b[k][j];
```

```
}  
}
```

程序中，有三次循环，每次循环的次数为 n ，因此最里层的语句 $c[i][j] += a[i][k]*b[k][j]$ 循环的次数最多，其循环次数为 n 的 3 次方，算法的执行时间就是这个语句的重复次数，因此该算法的时间复杂度： $T(n) = O(n^3)$ 。通过比较 $T(n)$ 的数量级，就可以比较不同算法的时间复杂度。当然在实际应用中，也可以比较在同一台机器上，算法的执行前后的时间差，来衡量算法的时间复杂度。

2. 空间复杂度

空间复杂度，它是指执行这个算法所需要的内存空间。利用这个指标，可以预先估计程序运行时，占用内存的大小。程序在执行的过程中，除了使用程序本身的常量、变量和字符外，还需要 CPU 和内存存在执行程序的过程中，临时产生的一些辅助存储空间，这些空间的产生可能比程序本身所需要的空间更大一些，这是在分析一个算法是需要客观考虑的问题。一般来讲，算法的空间复杂度包括两个部分。

(1) 算法固定不变的部分。这部分空间由算法本身的代码行数（或者叫指令空间），数据存储的单元，这些单元主要是一些变量和常量占据。这部分空间的大小与用户的需要没有太多的关系，有时也叫静态空间。

(2) 运行算法时操作系统产生的可变空间。这部分空间是变化的，和操作系统的特点有关，不同的操作系统在执行算法时，其调用的辅助空间大小是不一致的，这和操作系统执行算法所调用的辅助算法有关，比如线性表、堆栈和树等存储结构有关，这部分空间和算法本身的关系不是太大。

在现阶段，硬件技术发展飞速的情况下，随机森林算法的空间复杂一般不需要太多的考虑，而随机森林算法的时间复杂度是需要考虑的问题，因为在有些特殊的应用中，时间是决定成败的关键。

2.6 几种实现随机森林算法的软件介绍

2.6.1 Weka 软件

Weka 的全名是怀卡托智能分析环境（Waikato Environment for Knowledge Analysis），是一款免费的，非商业化（与之对应的是 SPSS 公司商业数据挖掘产品 --Clementine）的，基于 JAVA 环境下开源的机器学习（machine learning）以及数据挖掘（data mining）软件。它的强项主要在分类（classification）领域，在这个领域几乎包含了所有的机器学习的方法，而且它还集成了如回归、关联规则、clustering 等算法。Weka 软件是由 Java 编写的数据挖掘工具，所有的数据挖掘程序都包含在软件中，用户只需要到网站上下载最新的软件即可^[43]。该软件采用窗口可视化操作方式，用户只需要组织好数据，然后以数据表的形式引入到软件中，点击相应的数据挖掘方法，勾选好相应的设置，运行就可以得到所需要的统计结果。

使用 Weka 软件可以构建决策树或是随机森林，也可以使用其进行预测和回归，但是用户不能对这些算法进行二次开发，也不能将自己开发的软件组合到该软件中，也就是说 Weka 软件的开放性还不够好，而且有些随机森林的重要性质如变量的相互作用、资料相似性分析、找出异常点和错误标签数据等等还没有较完美的实现。

2.6.2 R 软件

1980 年左右，Auckland 大学的 Robert Gentleman 和 Ross Ihaka 及其他志愿人员开发了 R 软件。R 是一套完整的数据处理、计算和制图软件系统。其功能包括：数据存储和处理系统；数组运算工具（其向量、矩阵运算方面功能尤其强大）；完整连贯的统计分析工具；优秀的统计制图功能；简便而强大的编程语言：可操纵数据的输入和输出，可实现分支、循环，用户可自定义功能。

R 软件因其开源性、强大的统计计算等功能而受到统计人员的青睐。R 软件具备高效的数据处理和存储功能，擅长数据矩阵操作，提供了大量适用于数据分析的工具，支持各种数据可视化输出。R 软件的一大优势是分析人员可利用简单的 R 程序语言描述处理过程，以构建强大的分析功能。

但 R 软件具有自身难以克服的困难，主要包括如下几个方面：

1. 界面友好性差。R 软件采用命令式输入方式，使得有些统计人员或非统计人员以为 R 软件需要高深的程序设计能力，极大地阻碍了 R 软件的推广。
2. 运算速度慢。当前统计方法和模型的更新速度非常快，统计学科的发展日新月异，数据规模也越来越大，单机版的 R 软件无法支持庞大的计算量，需要利用 Internet 来扩展其运算速度。
3. 维护麻烦。R 软件经常性的更新而降低了使用 R 软件的意愿。迄今为止，R 源代码已经经历了近 70 次主要更新，经常性的更新使得有些统计人员放弃对 R 软件的使用。
4. 可扩展性差。R 软件具有大量的软件包，这些软件包实现了各种各样的功能，但其使用却没有一个统一的方式，需要使用者去查阅大量的相关文献才能做到，增大了 R 软件使用的难度。另外，R 软件包的开发也需要使用特定的语言，提高了 R 软件开发的门槛，不利于 R 软件的推广。

R 软件中实现随机森林算法，一般使用通过以下几个步骤完成。一是读入随机森林算法程序包 `library(randomForest)`，该程序包是基于 Breiman 随机森林理论的 R 语言软件包 `randomForest 4.6-6`；二是导入数据集，一般需要把数据集的缺失数据补齐；三是设置好随机森林算法的参数；四是运行并分析结果。

四个步骤中，最难完成的是第三个步骤，在 R 软件的随机森林算法中需要设置二个主要的参数：森林中决策树的数量（`ntree`）和内部节点随机选择属性的个数（`mtry`）。一般来讲，决策树的数量越多算法的精度越高，但程序的速度会有下降；

内部节点随机选择属性的个数 (mtry) 是影响算法精度的关键因子, 算法的强度和相关性和 mtry 的大小相关, 如果 mtry 足够小, 树的相关性趋向于减弱; 另一方面, 分类模型的强度随着 mtry 的增加而提高。在本文的后续章节中, 如没有特别说明, 在使用随机森林算法进行实验时, 均采用上述 R 软件的随机森林算法。

2.6.3 MATLAB 软件

MATLAB 是由美国 MathWorks 公司生产的商品化应用软件, 该软件具有良好的用户界面和实时的人机交互环境, 使用该软件可以进行程序设计、统计分析和数据挖掘, 一般的统计分析功能都可以在 MATLAB 软件中实现, 当然有的时候要适当进行代码的设计。在 MATLAB 软件中, 有一个很大的特色是提供了众多的应用函数, 这些函数丰富了软件的功能, 也方便了用户。经过多年的改版和更新, MATLAB 的用户界面越来越接近 Windows 的标准界面, 操作也越来越简单, 编程环境也更人性化, 开发者编写的程序可以不用编译也能运行, 同时具有良好的程序调试和纠错功能, 这些都为软件的广泛的使用提供了重要的支持。

虽然 MATLAB 功能强大, 且界面有好, 但由于其商业性质, 不同于 weka 和 R 的开源性质, 用户使用正版 MATLAB 软件时需要支付一定的费用。MATLAB 软件中, 提供了随机森林算法的接口, 和 R 软件一样, 用户需要对其参数进行设置, 有些应用还需要进行适当的编程才能使用该算法。

2.7 随机森林算法存在的问题

从随机森林的构建过程和性质可以看出, 随机森林算法在分类精度、泛化误差、算法强度等性能方面较决策树等单分类器较大的提升, 随机森林也在医学、蛋白质组学^[44]、生态学研究^[45]等众多领域得到了广泛的应用。随着研究的深入, 随机森林算法自身存在的问题也暴露出来, 主要表现在以下三个方面。

2.7.1 不能很好地处理非平衡数据

非平衡数据是指数据集中某一类的样本数量明显少于其他类样本的数目, 其中占数量最多的一类样本被称为多数类, 而占数量最少的一类则称为少数类。对于非平衡分布的数据集, Weiss G M 在文献^[46]中深入分析了其引发的数据稀缺、极端值和噪声等一系列问题, 并指出这是导致传统分类方法性能下降的原因。

在不平衡分类问题上, 黄衍等人^[47]通过比较在 20 个 UCI 数据集上, 随机森林和支持向量机的性能, 得出随机森林在不平衡分类数据的处理显著逊色于支持向量机。李建更等通过实验数据对比加权随机森林 (Weighted Random Forest, WRF) 和普通的随机森林在非平衡数据集上的分类性能, 结果表明前者在大多数情况下能取得优于后者的结果^[48]。

随机森林算法对非平衡数据的处理不够理想的主要原因有二点。一是由于随机森林在构建过程中, 训练集是随机选取的, 使用 Bagging 随机选取时, 由于原训练集中

的少数类本身的数据量就比较少，因此被选中的概率就很低，这使得 N 个随机选取的训练集中含有的少数类的数量比原有的数据集更少或者没有，这反而加剧了数据集的不平衡性，使得基于此数据集训练出来的决策树的规则就没有代表性；二是由于数据集本身少数类占有的比例就比较低，使得训练出来的决策树不能很好地体现占有数量少的少数类的特点，只有将少数类的数量加大，使数据集中的数据达到一定程度的平衡，才能使得算法更稳定。

针对随机森林算法不能很好地处理非平衡性数据集的情况，本文拟设计一种新的非平衡性数据集优化算法，用于优化随机森林算法的分类精度。

2.7.2 对连续性变量的处理还需要进行离散化

连续变量离散化是机器学习和数据挖掘研究和应用中的一个重要方面。连续变量的离散化可以降低系统的粒度数，如果粒度数过高，则每一个等价类中所含的个体数过少，进而使得从该信息系统产生的规则的支持度降低，使系统中含有更多的冗余信息；如果粒度过低，即每个属性过于离散化，则每一个等价类中所含的个体数过多，进而使得可以正确分类的样本数所占整个样本空间的比例过低，从而导致不相容信息的增加。因此，属性的离散化存在一个度的问题。已经证明，连续变量的最优离散化问题是一个 NP 问题。

在随机森林中，如果存在连续型的变量，通用的做法是把这些连续型变量的值分成不同的区间，即“离散化”。这种方法虽然能将连续型变量转化为离散型变量，但由于其离散化后算法复杂度和数据集的约简率存在很大的关系，使得算法在分析计算节点分裂标准时，需要花费很多的时间，极大地影响了算法的执行速度，因此在随机森林算法中，连续变量的离散化是一项需要优化的内容。

当前，连续变量离散化的方法有很多，最主要的方法是基于统计学思想的 CHI2 相关算法。本文拟在借鉴和改进 CHI2 算法的基础上，提出一种新的连续变量离散化方法，用于优化随机森林算法的运行效率。

2.7.3 随机森林算法的分类精度还需要进一步的提升

根据第四节内容，随机森林分类算法一个重要的性能指标是分类精度，亦称准确度(accuracy of measurement)，是指使用算法得出的分类结果与真实值之间的接近程度。分类精度的提高是分类算法优化研究中永恒的主题，虽然随机森林是众多分类算法中分类精度较高的算法，但在现有的研究文献中，其分类精度大约在 70%—90% 之间，这个比例在某些数据集上表现得更低。

在黄衍等人^[47]的论文中，测试了 20 个 UCI 数据集随机森林算法交叉验证误差和分类精度（分类精度 $\approx 1 - \text{交叉验证误差}$ ）如下表所示：

表 2.3 随机森林算法分类精度分析表

序号	二分类			三分类		
	数据集名称	交叉验证 误差	分类 精度	数据集名称	交叉验证 误差	分类 精度
1	Breast Cancer	4.80%	95.20%	Wine	2.10%	97.90%
2	Vertebral Column	15.50%	84.50%	Iris	6.90%	93.10%
3	Blood Transfusion	21.60%	78.40%	Cardiotocography	7.60%	92.40%
4	Haberman's Survival	30.00%	70.00%	Vehicle Silhouettes	22.10%	77.90%
5	Ionosphere	6.60%	93.40%	Glass	22.80%	77.20%
6	Parkinsons	7.20%	92.80%	Breast Tissue	23.60%	76.40%
7	MAGIC	19.70%	80.30%	Image Segmentation	4.80%	95.20%
8	Pima Indians Diabetes	23.00%	77.00%	Segmentation	4.80%	95.20%
9	Connectionist Bench	19.70%	80.30%	Ecoli	16.00%	84.00%
10	Heart	12.80%	87.20%	Libras Movement	24.90%	75.10%
11	平均值	16.09%	83.91%	平均值	13.56%	86.44%
12	标准差		0.08	标准差		0.09

从上表可以看出，在二分类数据集中，随机森林的分类精度平均只有 83.91%，表现最差的是 Haberman's Survival 数据集，其分类精度只有 70%，有五个数据集的分类精度刚刚达到 80%或低于这个值，占总二分类数据集总数的 50%。在三分类数据集中，也有 4 个数据集是低于 80%的。因此随机森林算法的分类精度还有提升的需求。

随机森林算法在不同数据集上的分类精度存在问题的主要原因有二个方面。一是数据集本身的原因，有些数据集的属性变量较少，在决策树的形成过程中分类规则过于简单；二是数据集的非平衡性导致的。这两种情况都需要对数据集进行预处理。二是算法本身的问题，算法在分裂的过程中选择分裂规则，或者是随机抽样时存在一些需要改进的方面。另外，在大数据时代，随着数据量的增加，数据噪声更加严重，在某些需要更高精度的领域应用时，该算法还具有进一步优化的现实需求。本文考虑设计一个新的算法，该算法通过优化随机森林中单棵决策树的节点分裂算法，达到提高算法分类精度的目的。

2.8 本章小结

本章是本文后期所有研究工作的基础理论部分，详细的介绍了决策树算法的生成算法，并对 CLS 算法、ID3 算法、C4.5 算法和 CART 算法展开了详细的讨论。在对决策树算法存在的问题进行分析后，详细描述了随机森林算法的构建过程，同时给出了随机森林算法的理论基础。

本章的另一个重要内容是详细介绍了衡量随机森林的三个性能指标，为后续的性能优化提供了可比较的依据，并详细论述了泛化误差与 OOB 估计的关系。文中还介

绍了几种实现随机森林的软件，这些软件是使用随机森林时必不可少的工具。

在本章的最后，提出了随机森林算法在处理非平衡数据、连续性数据等方面存在的问题及算法分类精度提升的现实需求，为后续章节的研究确定了方向，文章后面的内容都是依据这些问题和应用展开的。

3 随机森林处理非平衡数据集的优化研究

在第二章的分析中可以看出,随机森林算法对非平衡数据集的分类效果较差,因此有很多文献进行了通过改善数据集的非平衡性来优化随机森林算法的性能的探讨,本章通过讨论非平衡数据集的分类问题的解决办法,提出一种新的算法来改善数据集的非平衡性,从而达到提升随机森林算法分类性能的目的。

3.1 非平衡数据简介

非平衡数据是指数据集中某一类的样本数量明显少于其他类样本的数目,其中占数量最多的一类样本被称为多数类,而占数量最少的一类则称为少数类。在二分类数据集中,多数类简称为正类,少数类简称为负类。对于非平衡分布的数据集,Weiss G M 在文献^[49]中深入分析了其引发的数据稀缺、极端值和噪声等一系列问题,并指出这是导致随机森林分类方法性能下降的主要原因。

在实际应用中,非平衡数据是经常存在的,特别是在网络安全、邮件处理等领域。以网络安全为例,在大量的网络数据中,不安全或恶意的数据信息所占的份额很少,这就使数据集出现了非平衡的现象。而且在当今大数据时代,面对海量数据,非平衡数据集的出现就成为了一种常态。如何解决非平衡数据在分类研究中所产生的不好的问题,已经成为数据分析和挖掘研究领域的热点。

本章在详细分析非平衡数据集 SMOTE 再抽样算法的基础上,结合欠抽样的原理,提出一种新的非平衡数据集的抽样办法,有效地解决随机森林算法在非平衡数据集上进行分类时性能较差的问题。实验数据表明,使用随机森林算法对经过该处理的非平衡数据集进行分类时,可获得较为理想的分类效果。

3.2 非平衡数据集分类困难的原因分析

当使用传统的分类算法对非平衡数据集进行分类时,经常会大面积出现负类的预测准确率远远低于正类,或者负类的预测准确率远远低于正类的现象,使得分类算法得到的分类器具有很大的偏向性,从而导致分类器性能的大幅度下降。由于偏向性的问题存在,使得在随机森林算法的分类精度很高的情况下,也会出现大面积的错分现象,这使得很多实际问题难以得到有效解决。

按照是否是数据集自身原因导致的不平衡,非平衡数据集可分为二种,一是本质非平衡数据集;二是非本质非平衡数据集。本质非平衡数据集是指原始数据集在一开始时,自身的分布就存在不平衡,这些数据集的一个重要特点是,其实际应用的关注点都在少数类数据上。比如对信用卡非法交易的检测,非法交易记录是监测的目标,但训练数据中包含大量正常的信用卡交易记录,只有特别少的一部分属于非法交易,

使用普通的模式分类方法,非法交易记录的检测率很低。非本质非平衡数据集的不平衡问题的来源是人为因素,不是数据集自身原因。最容易理解的就是个人隐私的调查,在这些调查中,有些人不愿意把自己的个人隐私告诉给调查者,因此使得调查结果形成的数据集中负类很少,另外还有一种情况就是采集数据时,负类数据的采集需要高昂的费用。除此之外,还有一些非本质非平衡数据集是根据某些特殊实验的需要而产生的,比如由于支持向量机无法直接解决多分类问题,这时实验者将原始问题分解成一系列二分类问题才能够间接地解决该问题,这样很容易导致原来平衡分类问题变得不平衡了。

针对少数类样本识别比多数类样本识别更为困难的问题,Weiss G M 深入分析了非平衡数据集中导致传统分类方法性能下降的主要原因^[49]。

3.2.1 数据稀少问题

在非平衡数据集的定义中可以看出,该数据集有一个典型的特点就是负类样本的数据较正类样本要稀少得多,显然,这是导致分类器性能急剧下降的最直接的原因。当然,负类样本比正类样本的稀少,既包括相对数量的稀少和绝对数量的稀少两个概念。具体来讲,负类样本绝对数量稀少,是指在非平衡数据集中,我们所关注的负类样本的数量较少,从而使得在训练分类器时,负类样本的特征无法完整地在分类器中得到训练,使得分类器在预测和分类时不能取到一个好的分类效果。

负类样本相对数量稀少,是指在非平衡数据集中,我们所关注的负类样本的绝对数量是不少的,但相对于正负样本,负类样本在整个数据集中所占的比重过小。对于这种现象,已经有学者通过实验证明,通过改变训练集规模,或者改变抽样的办法,就可以使这种相对不平衡的现象得到比较好的消除,而且只要总样本足够多,相对数量较少并不一定会导致分类性能下降。

因此在实际的应用中,可以区分这两种非平衡数据集,然后根据具体应用的特点,有针对性地提出解决办法。对于相对数量稀少的非平衡数据集,可以采用欠抽样的办法解决,而对于绝对数量稀少的非平衡数据集,则要采用其他的办法解决,从而提升分类器的性能。

3.2.2 数据碎片问题

数据碎片是指在算法的执行过程中,原来的数据集被划分成多个小的数据集,这些小的数据集的样本量越来越小,从而使得原本平衡的数据集变成了非平衡的数据集。这种现象在决策树和随机森林算法中普遍存在。比如以 CART 算法为分裂算法的决策树中,当选择好一个属性进行节点分裂时,每来的数据集就会一分为二,在一分为二的过程中,就会产生数据集变小的情况,从而使下一步的节点分裂带到很严重的问题,就是在计算过程中数据集的非平衡性问题。

由于数据碎片的存在,使得决策树在分裂的过程中,丢弃了原有数据集的规律,使得数据的原有信息在产生数据碎片的过程中逐渐消失,从而使得分类问题陷入一种

自身难以解决的怪局，这也是传统分类方法不适用于非平衡数据集的另一个重要原因。

3.2.3 归纳偏差问题

归纳偏差是在分类算法在学习的过程中，为了有效避免过学习，而采用的一种消除办法，比如在贝叶斯分类器中，分类准确率依赖于概率分布的准确估计。为了让分类算法在学习的过程中不出现过学习，必须允许算法存在一定的归纳偏差。正是由于这个原因的存在，使得负类样本在数据量少的情况下，其概率估计的准确性就要比正类样本差很多，从而使分类器在对负类样本进行分类时其准确率大大下降。因此归纳偏差加剧了负类样本的不正确率，因此归纳偏差问题也影响一些传统分类器分类性能的一个原因。当然，归纳偏差只是少数分类器算法的问题，在本文讨论的决策树和随机森林算法中，这个问题所产生的负面作用不是很明显。

3.2.4 噪声问题

噪声数据 (noisy data) 就指是无意义的的数据 (meaningless data)。这个词通常作为损坏数据 (corrupt data) 的同义词使用。但是，现在它的意义已经扩展到包含所有难以被机器正确理解和翻译的数据，如非结构化文本。任何不可被创造它的源程序读取和运用的数据，不管是已经接收的、存储的还是改变的，都被称为噪声。从统计学的角度讲，噪声 (noise) 是被测量的变量的随机误差或方差

在非平衡的数据集中，负类样本的数量是比较少的，但负类样本也是存在噪声的，如果为了防止过学习对数据集进行噪声处理，在使不合适的数据处理工具的情况下，会使本来就数量较少的负类样本，在处理时丢弃一些有意义的负类样本，这样做的结果会使负类样本的数量更进一步地减少，从而从某种程度上进一步地加剧了数据集的不平衡性，使得分类器的分类性能效果更差。如果不进行噪声处理，又可能会产生过学习问题，这样也会产生分类器分类性能差的现象。因此由于噪声数据的存在，使得非平衡数据集在进行分类时，分类性能会进一步下降。

3.2.5 评价指标选择问题

分类器评价指标是指在对分类器性能评价时，所采用的衡量标准。在第二章的第四节中，已经对随机森林算法的性能指标已经做了定义，这些指标的如何选择也是有一定区别的。分类器分类性能的评价指标选择，在非平衡数据集中也会出现一些问题。一般的分类器常用的评价指标都是分类精度，而在非平衡数据集中，分类精度指标不一定能准确地反应算法的性能。例如，对数量比为 1:99 的两类样本，在最极端情况下，分类器可将所有样本都分到多数类样本中，但依然能得到 99% 的准确率，但此分类器却不具实际意义。

因此单一的指标选择对于非平衡数据集来讲是不够的，必须加入更多的指标进行分析，在第二章提到的负类检验值 (F-Value)、G 均值，都是衡量非平衡数据集比较好的指标，而这些指标在传统的分类器中比较少，一般需要用户自己计算。

3.2.6 数据分布问题

数据集中正负类样本的分布,也是引起分类器分类性能下降的一个重要原因。由于正负类样本会有类重叠现象,某些数据集的部分两类样本在数据空间中重叠在一起难以区分开来,使得部分负类样本的识别变得几乎不可能。由于负类样本本来数量较少,出现这种重叠后,可用的负类样本数量更少了,因此在训练的过程中,分类器可以得到改进的空间变小,这也从另一方面加重了非平衡数据集分类的困难程度。

此外,还有类内不平衡也是非平衡数据集分类困难的重要原因。类内不平衡,即小析取项,从概念学习的角度来说,它反映了同一类的若干子概念之间学习样本分布的非平衡性。小析取项即那些学习样本数量偏少的子概念,它们容易被错误学习进而影响分类器的整体分类性能。类间和类内不平衡是非平衡数据集的两个不同侧面,它们可能同时出现,均会影响分类器的性能

3.3 非平衡数据集分类问题的解决办法

非平衡数据集分类问题的研究已得到国内外学者的广泛关注。目前,解决非平衡数据集分类问题大致分为两大类:一类是改进算法,另一类是改造数据。

3.3.1 通过改进算法解决非平衡问题的方法简介

改进算法的主要思路是结合非平衡数据集的特点,通过对传统分类算法的改进或设计新的分类算法,以适应非平衡数据的分类需要。主要包括代价敏感(Cost-Sensitive)学习、特征选择和单类学习等方法^[34]。

1. 代价敏感学习方法

代价敏感学习基于一个基本假设,即非平衡类问题中识别负类比正类具有更大的现实意义。比如识别数量少的恶意的攻击性网络包,比识别数量多的善意的网络包更具有意义。其基本思想是赋予负类样本更大的权重,也叫错分代价,迫使分类器更加关注于提高负类样本的分类准确率,从这个思路上看,它能够很好地解决非平衡数据集分类问题。以二分类问题为例,给予负类更高的权重,那么分类器在进行训练时,会对错分少数类样本的行为做出更大的惩罚,这样就迫使最终得到的分类器对少数类样本具有更高的预测准确率。

2. 特征选择方法

在实际的问题中,如果数据集现在不平衡现象,那么在样本的属性特征分布也会出现一种与之对应的非平衡现象,尤其在文本分类问题当中,负类样本记录的某些特征,也许在正类样本里根本不会出现。特征选择方法正是基于此现象产生的。该方法一般都是将属性特征和数据集的不平衡结合起来,通过打破属性特征的不平衡性,改变数据集的不平衡性。该方法的应用范围一般较小,只适合特定的数据集。

3. 单类学习方法

单类学习方法是指在数据非平衡状态很严重的情况下,通过重点关注负类样本,

而忽略对正类样本的关心,在进行分类器训练时,使少数正类样本参与学习,而产生分类规则。在单类学习方法中,目前的研究都集中在对 SVM 算法的改进。

3.3.2 通过改造数据解决非平衡问题的方法简介

改造数据的主要思路是改变训练集数据的分布,通过人为增加负类样本,提升不平衡率,主要包括随机向上抽样,随机向下抽样或者是对这两种抽样方法的改进的抽样方法。

1. 随机向上抽样和随机向下抽样方法

随机向上抽样^[55]就是通过简单的复制一批少数类,从而使少数类的数量增加,达到数据集平衡化的目的。而随机向下抽样^[56]则是将数据集中的正类样本进行一定程度的删除,通过这样处理后,数据集中正类样本的数量会和负类样本的数量达到一致,从而达到数据集平衡化的目的。

无论随机向上或向下抽样,虽然可以使新的数据达到一定的平衡,且数据的分布能很好地适应一般分类算法的需求,但是由于其随机性,使得新的数据集不能很好的表示目标数据集,而且两者间的差异越来越大,经过实践的检验,这两种方法都达不到理想的效果。人们开始寻求对这两者改进的方法,大量的文献都进行了这方面的探索。

2. SMOTE 算法

针对向上抽样时过于随机性的问题,2002 年 Chawla 等人提出了 SMOTE 算法^[57], SMOTE 是 Synthetic Minority Over-sampling Technique 的缩写,该算法的本质是对随机向上抽样策略的改进。由于随机向上抽样的过程中,只是简单地复制负类,在选择复制对象是存在随机性,且生成的对象在原数据集中也存在重复问题,不能有效地解决问题的本质。

针对随机向上抽样的重复问题, SMOTE 算法提出一种基本假设:相距较近的负类之间的样本仍然是负类。从这个假设出发,该方法为每个负类样本确定其 K 个相距最近的负类,然后在样本与其近邻样本的连线上合成“人造”样本,合成过程依据如下公式完成:

$$P_i = x_i + rand(0,1) \times (y_{ij} - x_i) \quad (3.1)$$

其中: x_i ($i=1,2,\dots,n$) 为负类样本, n 为负类样本总个数;

y_{ij} ($j=1,2,\dots,m$) 为 x_i 相邻的 m 个近邻样本;

p_j ($j=1,2,\dots,m$) 为合成的“人造”样本;

$rand(0,1)$ 表示在 $(0, 1)$ 之间的一个随机数。

通过这种组合产生的“人造”样本就具有一些简单复制样本不能具有的优良特性,比如“人造”样本不会和原数据集中的样本重合,“人造”样本继承了负类样本和相邻样本的某些特性。把这些“人造”的负类样本加入到初始的非平衡的数据集,生成

的新的数据集的非平衡性就大大降低了,非可以较好地提升分类算法在非平衡数据集上的性能。

该算法的执行步骤如下：

Step 1 为每个负类样本挑选出 K 个数量的邻近样本；

Step 2 将负类样本和其邻近样本进行线性组合；

Step 3 将组合产生新的“人造”样本加入到原数据集中；

Step 4 计算数据集的非平衡率,当数据集没有达到目标平衡率时,转 Step 1, 否则程序结束。

算法执行中,用到非平衡数据集考量时一个很重要的指标,就是非不平衡率 (Imbalance Rate),它是少数类占全体样本量的比率。不平衡率越小说明该数据集的不平衡程度越严重。算法还涉及另一个变量 K , K 为算法执行过程中,由 x_i 出发找到的 K 个相距最近的负类,也叫 K 个近邻样本, M 为从 K 个近邻中随机挑选出的近邻样本的个数,且有 $M < K$,一般 K 值由使用者根据实际情况设定。

SMOTE 算法的程序流程图如下：

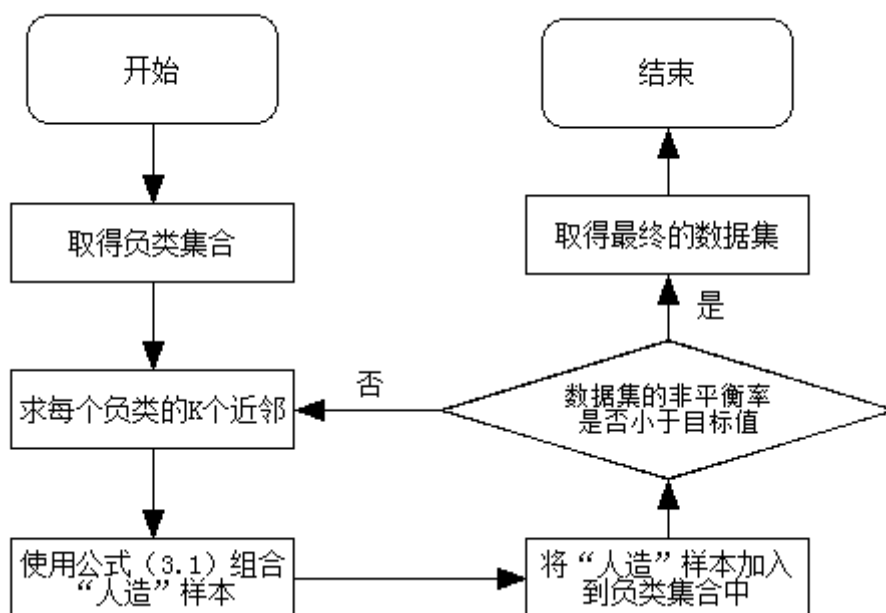


图 3.1 SMOTE 算法流程图

3. SMOTE 算法存在的问题

该算法存在的主要存在两方面的问题：一是在近邻选择时,存在一定的盲目性。从上面的算法流程可以看出,在算法执行过程中,需要确定 K 值,即选择多少个近邻样本,这需要用户自行解决。从 K 值的定义可以看出, K 值的下限是 M 值, M 值的大小可以根据负类样本数量、正类样本数量和数据集最后需要达到的平衡率决定。

但 K 值的上限没有办法确定, 只能根据具体的数据集去反复测试。因此如何确定 K 值, 才能使算法达到最优这是未知的。

另外, 该算法无法克服非平衡数据集的数据分布问题, 容易产生分布边缘化问题。由于负类样本的分布决定了其可选择的近邻, 如果一个负类样本处在负类样本集的分布边缘, 则由此负类样本和相邻样本产生的“人造”样本也会处在这个边缘, 且会越来越边缘化, 从而模糊了正类样本和负类样本的边界, 而且使边界变得越来越模糊。这种边界模糊性, 虽然使数据集的平衡性得到了改善, 但加大了随机森林算法进行分类时的难度。

4. 针对 SMOTE 算法的进一步改进

针对 SMOTE 算法存在的边缘化和盲目性等问题, 很多人纷纷提出了新的改进办法, 在一定程度上改进了算法的性能, 但还存在许多需要解决的问题。

Han 等人^[58] 在 SMOTE 算法基础上进行了改进, 提出了 Borderhne-SMOTE 算法, 该算法在运行的过程中, 查找一个适当的区域, 该区域可以较好地反应数据集的性质, 然后在该区域内进行插值, 以使新增加的“人造”样本更有效。这个适当的区域一般由经验给定, 因此算法的执行的過程中有一定的局限性。

王春玉^[59] 针对非平衡数据引起分类性能下降的问题, 提出了一种改进的算法, 该算法将 boosting 算法和基于错分样本的过抽样技术相结合, 在新算法中, 引入支持向量机进行分类, 每次 boosting 迭代中标记出错分的样本点, 然后根据 SMOTE 思想在错分样本点与其近邻间随机产生一定数量“人造”样本, 将这些新的样本加入初始训练集中, 打破数据集的不平衡状态, 这种办法过程复杂, 且算法最终的效果和 SMOTE 算法相比没有太大的提升。

薛薇^[60] 提出非平衡数据集的改进 SMOTE 再抽样算法, 通过改进 SMOTE 再抽样算法, 将过抽样和欠抽样方式有机结合, 有针对性地选择近邻并采用不同策略合成样本, 较好地解决非平衡数据集的问题。这些基于 SMOTE 算法的改进算法有很多, 但至今没有较完善的办法, 本文拟针对 SMOTE 算法存在的问题, 提出一种改进的 SMOTE 新算法。

3.4 随机森林处理非平衡数据分类问题的优化--C_SMOTE 算法

3.4.1 C_SOMTE 算法设计

针对 SMOTE 算法存在的问题, 本文提出一种改进的 SMOTE 抽样算法, 简记为 C_SMOTE 算法。C_SMOTE 算法的基本假设是: 根据聚类算法的思想和物理学的重心理论, 同一类的样本应有一个共同的重心, 一个类用它的重心(该类样本属性的均值)做代表比较合理^[68]。根据这个假设, 既然每个类都有一个重心, 那么在“人造样本”的过程中, 新产生的样本也应向类的重心靠拢, 这样使得样本的产生过程有一定的方向性, 从而克服 SMOTE 算法在进行合成时的盲目化问题。而且由于“人造”

样本会有目的趋向重心，由处在分布边缘的样本产生的新样本的分布会得到改善，从而克服了 SMOTE 算法在进行合成时的边缘化问题。如何在产生“人造”样本时，使新产生的样本向重心汇聚，这是 C_SMOTE 算法的核心内容。C_SMOTE 算法的具体步骤如下：

第一步，计算负类样本的中心点，记为 X_{center} 。如果将每个负类样本看作空间中的一个点，那么负类样本的中心点就是这些点组成的空间的重心，越趋近于重心的点越具有该负类样本的特征。重心的计算可以有二种方式。

一是可以使用向量的概念来完成该重心的计算，主要是引入向量重心概念。设训练集为 T ，每个样本有 r 个属性，负类和正类的样本量分别为 $n1$ 和 $n2$ ，则负类样本集合可记为 $X: X = \{X_1, X_2, \dots, X_{n1}\}$ ， $X_i = (x_{i1}, x_{i2}, \dots, x_{ir})$ ；正类样本集合可记为 $Y: Y = \{Y_1, Y_2, \dots, Y_{n2}\}$ ， $Y_i = (y_{j1}, y_{j2}, \dots, y_{jr})$ 。

定义 8 负类样本的重心为

$$X_{center} = (\frac{1}{n1} \sum_{i=1}^{n1} x_{i1}, \frac{1}{n1} \sum_{i=1}^{n1} x_{i2}, \dots, \frac{1}{n1} \sum_{i=1}^{n1} x_{ir}) \quad (3.2)$$

定义 9 正类样本的重心为

$$Y_{center} = (\frac{1}{n2} \sum_{j=1}^{n2} y_{j1}, \frac{1}{n2} \sum_{j=1}^{n2} y_{j2}, \dots, \frac{1}{n2} \sum_{j=1}^{n2} y_{jr}) \quad (3.3)$$

根据定义 8 就可以计算出负类样本的重心点，C_SMOTE 算法的核心就是围绕重心点产生“人造”样本。

二是使用统计工具完成数据集的初始聚类，找到负类样本的重心点 X_{center} 。具体做法是，使用 SPSS、R 等统计软件，选择 K 均值聚类算法、系统聚类、两步聚类等常用的聚类办法，将负类样本集输入到软件中，设置需要将负类样本集分为二类。根据软件的运行结果，选择样本点数量多的一类的中心点作为本算法中负类样本的中心点。

第二步，“人造”样本的合成。为了保证合成的样本向类的重心趋近，本算法对 SMOTE 算法的公式进行改造，公式如下：

$$p_j = X_i + rand(0,1) \times (X_{center} - X_i) \dots\dots\dots (3.4)$$

其中：

X_i ($i=1, 2, \dots, n1$) 为负类样本， $n1$ 为负类样本总个数；

X_{center} 为负类样本的中心点；

p_j ($j=1, 2, \dots, m$) 为合成的“人造”样本；

$rand(0,1)$ 表示在 (0, 1) 之间的一个随机数。

从几何角度上看，本算法 (C_SMOTE 算法) 的本质是在负类样本和数据集重心

之间的连线上进行再抽样，这样使得新产生的“人造”样本，永远在中心点和负类样本之间，其所处的位置由随机数确定，不会偏离出负类样本集的几何空间，因此不会产生边缘化的趋势，而是有目的地向中心点趋近，从而减少了随机性和盲目性。另外，和公式（3.1）相比，公式（3.4）不需要取得 k 值，减少了算法的不稳定性。

第三步，欠抽样处理合成后的负类数据集。根据中心点、 M 值和公式（3.4）合成样本，让合成的负类样本数量比实际需要的样本数量多出一部分，然后使用欠抽样的办法删除一些样本，被删除的样本需符合一个基本原则，那就是该样本离中心点的距离较别的样本点要远。负类样本到中心点的距离计算的方式有很多，包括欧氏距离、曼哈顿距离、切比雪夫距离、闵可夫斯基距离、标准化欧氏距离、马氏距离、夹角余弦、汉明距离等，可以根据实际应用来选择具体的方法。最后，使所合成的样本数量达到所需要满足的平衡率，平衡率可以根据实际应用确定。

经过上述三个步骤的处理后，整个再非平衡处理过程结束，将负类样本集和正类样本集组合在一起，就可以作为随机森林算法的训练数据。

3.4.2 C_SMOTE 算法流程图

C_SMOTE 算法在实现时，使用 JAVA 语言进行开发，开发平台为 MyEclipse 7.0+JDK1.6，主要的程序代码见附录 A，算法的流程图如下：

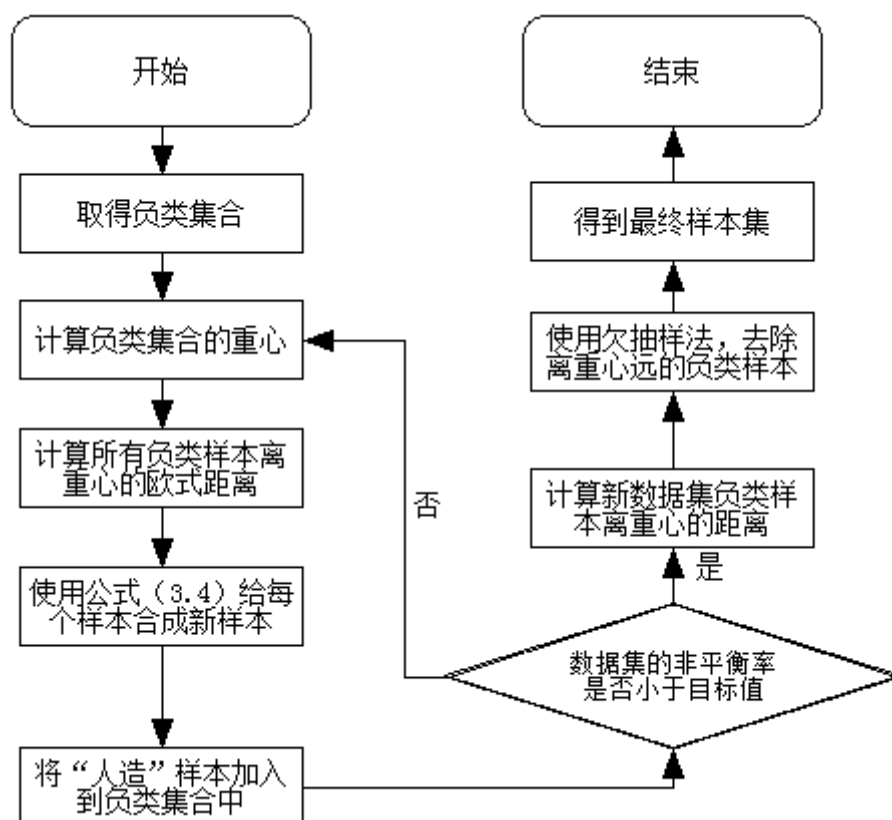


图 3.2 C_SMOTE 算法流程图

3.5 使用 C_SMOTE 算法提升随机森林算法性能实证分析

3.5.1 数据集介绍

实验中所采用的数据集为 Blood, Yeast 和 Pima, 均来自 UCI 机器学习数据库, 数据可以从 <http://archive.ics.uci.edu/ml/> 网址下载得到。表 3.1 描述了实验所使用的初始不平衡数据集的特征, 包括初始数据集中样本个数、数据集变量个数、负类样本数量、正类样本数量和初始的不平衡率。

表 3.1 初始不平衡数据集的特征描述

数据集名称	样本数	变量数	负类样本数	正类样本数	初始不平衡率
Blood	745	5	178	570	31.23%
Yeast	1484	9	51	1433	3.56%
Pima	768	8	268	500	53.5%

3.5.2 实验环境

随机森林算法使用 R 3.0.2 软件的语言软件包 randomForest 4.6-6 来实现, 随机森林算法的二个主要的参数设置如下: 森林中决策树的数量 (ntree=1000), 随机选择属性的个数 (mtry=3)。

在欠抽样计算样本点和聚类中心的距离使用标准化欧氏距离。标准化欧氏距离是针对简单欧氏距离的缺点而作的一种改进方案。下面对不同的距离计算方式做一下简单的介绍:

1. 欧氏距离。欧氏距离是计算两个点之间距离最简单的方法, 也是经常被采用的方法, 它定义的空间可以是二维的, 也可以是 N 维的。

(1) 在二维平面上, 两点 $a = (x_1, y_1)$ 与 $b = (x_2, y_2)$ 间的欧氏距离公式为:

$$d_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad (3.5)$$

(2) 在三维空间中, 两点 $a = (x_1, y_1, z_1)$ 与 $b = (x_2, y_2, z_2)$ 间的欧氏距离公式为:

$$d_{12} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \quad (3.6)$$

(3) 两个 n 维向量 $a = (x_{11}, x_{12}, \dots, x_{1n})$ 与 $b = (x_{21}, x_{22}, \dots, x_{2n})$ 间的欧氏距离公式为:

$$d_{12} = \sqrt{\sum_{k=1}^n (x_{1k} - x_{2k})^2} \quad (3.7)$$

2. 标准欧氏距离

欧氏距离虽然简单易用, 但也存在一些很简单的问题。它将数据的中各指标的之

间的差别忽略不计,或者一视同仁,这样做有的时候很难满足实际的应用。要解决这个问题,比较通用的做法就是将个分量都“标准化”到均值、方差相等的状态。具体做法是,设样本集是由 n 维向量组成的集合,样本个数计为 m ,计算 m 个样本在每个维度的均值(记为 $mean_k$),标准差(记为 sd_k),然后在每个维度上进行标准化,公式为:

$$x_k^* = \frac{x_k - mean_k}{sd_k} \quad (3.8)$$

然后对新生成的两个 n 维向量 $a^* = (x_{11}^*, x_{12}^*, \dots, x_{1n}^*)$ 与 $b^* = (x_{21}^*, x_{22}^*, \dots, x_{2n}^*)$ 使用公式公式(3.4)计算其欧氏距离。经过简单的推导就可以得到两个 n 维向量 $a = (x_{11}, x_{12}, \dots, x_{1n})$ 与 $b = (x_{21}, x_{22}, \dots, x_{2n})$ 间的标准化欧氏距离的公式:

$$d_{12} = \sqrt{\sum_{k=1}^n \left(\frac{x_{1k} - x_{2k}}{sd_k} \right)^2} \quad (3.9)$$

如果将公式(3.9)中向量每个维度的方差 sd_k 的倒数看成是一个权重,这个公式可以看成是一种加权欧氏距离(Weighted Euclidean distance)。

3.5.3 实验结果及分析

1. 两个算法处理的数据集对随机森林性能的影响分析

根据 SMOTE 算法和 C_SMOTE 算法的思想,设计出相应的 JAVA 程序后,将上述三个数据集进行平衡处理,使最终数据集的平衡率为 1:1,即正类样本和负类样本的数量一致。然后使用这些数据集在 R 软件上进行随机森林分类,考察初始数据集和经两个算法处理后的数据集对随机森林性能的影响。随机森林的性能指标在第二章已有介绍,此处使用 G-mean, F-value 和 OOB 估计三个评价指标,评价由于两种非平衡数据集处理算法的不同而使随机森林算法的性能优化的程度。由于随机森林的随机性,某一次实验结果会存在一定的随机误差,为了使实证分析更为严谨,每次实验时均需重复 100 次,即 SMOTE 算法和 C_SMOTE 算法重复运行 100 次,产生 100 个不同的数据集,然后在此 100 个训练集上均重复 100 次随机森林试验,观察每个指标在 100 次重复实验结果的均值。粗体代表实验中比较好的实验结果。

(1) OOB 估计

实验结果如表 3.2 所示。

表 3.2 随机森林算法在不同数据集上的 OOB 估计指标结果表

数据集	Blood	Yeast	Pima
初始数据集	0.2170	0.0330	0.2419
经 SMOTE 算法处理后的数据集	0.1516	0.0577	0.0955
经 C_SMOTE 算法处理后的数据集	0.1026	0.0311	0.0260

(2) G—mean

实验结果如表 3.3 所示。

表 3.3 随机森林算法在不同数据集上的 G—mean 指标结果表

数据集	Blood	Yeast	Pima
初始数据集	0.5263	0.4362	0.7143
经 SMOTE 算法处理后的数据集	0.8631	0.7948	0.9026
经 C_SMOTE 算法处理后的数据集	0.8967	0.8823	0.9753

(3) F—value

实验结果如下表所示。

表 3.4 随机森林算法在不同数据集上的 F—value 指标结果表

数据集	Blood	Yeast	Pima
初始数据集	0.8402	0.9842	0.8261
经 SMOTE 算法处理后的数据集	0.8687	0.9666	0.9008
经 C_SMOTE 算法处理后的数据集	0.8967	0.9826	0.9753

从表 3.2-4 可以看出：

(1) 使用“人造”样本的方法均改善了随机森林算法的性能。无论是 SMOTE 算法还是 C_SMOTE 算法使用“人造”样本的方法，改善了原有数据集的平衡，从而使随机森林算法的 OOB 估计，即随机森林算法的泛化能力得到了提升。在 pima 数据集中表现得最为明显，分别较原始数据集提升了 15 个百分点和 22 个百分点。虽然在 yeast 数据集中，没有加工的数据集的 OOB 估计较 SMOTE 算法平衡后的数据集的 OOB 估计要低，这主要是因为没有加工的数据集中负类样本的比较太少，从而使得随机森林在预测时，没有加工的数据集的准确度反而上升了。但由于算法对于负类样本的识别准确度很差，因此在 G—mean 指标反映时没有加工的数据集的表现就很差，这主要是因为 G—mean 指标综合体现了随机森林算法对正、负类样本的分类能力，而 OOB 估计仅体现算法对所有样本的总体分类能力。

(2) C_SMOTE 算法较 SOMOTE 算法有更好的非平衡处理能力。由于 C_SMOTE 算法在合成“人造”样本时，样本具有向心的趋势，克服了 SMOTE 算法在“人造”过程中的盲目性。从三个表中可以看出三个性能指标在三个不同的数据集中均有很好的表现，C_SMOTE 算法在 3 个数据集下的 OOB 估计、G-mean、F-value 三个评价指标均大于 SMOTE 算法。

也可以采用两两配对 T 检验的显著性检验方法对两个算法的实验结果进行对比验证。两两配对样本 T 检验是一种统计检验方法，通过对配对样本的数据之差组成的数据序列的均值来检验配对样本是否存在显著性差异。配对的过程中，要求配对的两列数据具有相同的个数，同时配对源性质基本相同，比如可以将性别、年龄、体重和种类相同的动物的某项观测值进行配对。也可以观测对同一个数据集，使用不同方法

对数据集的性能的改进变，从而观察这些方法是否存在本质的差异。

两两配对 T 检验的统计量计算公式如下：

$$t = \frac{\frac{1}{n} \sum d}{\sqrt{\frac{\sum d^2 - \frac{1}{n} (\sum d)^2}{n-1}}} \quad (3.10)$$

其中： d 为两样本各对数据之差

根据 $df=n-1$ 查临界 t 值，将计算所得统计量的值与其比较，作出两配对样本所属总体平均数是否存在差异的推断。

此处考虑对三个数据集进行的 100 次实验中，分别使用 SMOTE 和 C_SMOTE 算法处理后，分析不同算法对随机森林分类性能的影响程度。因此该配对数据可以检验不同算法在相同数据集和实验环境下，对随机森林算法 OOB 估计的影响程度。检验结果如下表：

表 3.5 成对样本统计量信息

配对序号	数据集	算法名称	均值	样本数量	标准差	均值的标准误
1	Pima	SMOTE	0.095524	100	0.0046318	0.0014647
		C_SMOTE	0.026017	100	0.0039032	0.0012343
2	Yeart	SMOTE	0.057668	100	0.0042272	0.0013368
		C_SMOTE	0.031081	100	0.0029265	0.0009255
3	Blood	SMOTE	0.15155	100	0.0135689	0.0042909
		C_SMOTE	0.102593	100	0.0080243	0.0025375

表 3.6 成对样本检验结果表

配对序号	数据集	算法名称	均值	标准差	95% 置信区间		t	df	Sig. (双侧)
					下限	上限			
1	Pima	SMOTE	.0695071	.0077671	.06395	.07506	28.299	99	.000
		C_SMOTE							
2	Yeart	SMOTE	.0265873	.0057565	.02247	.03071	14.605	99	.000
		C_SMOTE							
3	Blood	SMOTE	.0489567	.0146376	.03849	.05943	10.576	99	.000
		C_SMOTE							

从表 3.5-6 可以看出：

(1) 在表 3.6 中, 前二列分别体现了差值样本的均值和标准差, 从这些数据可以看出, 三对数据的均值存在差异, 且差异化比较平稳。从最后 Sig. (双侧) 指标都为 0.00, 都小于 0.05, 应该拒绝原假设: 两组数据没有明显差异。因此可以得出结论, 三个数据集在分别使用 SMOTE 算法和 C_SMOTE 算法后, 随机森林算法对两个算法新产生的数据集分类性能有显著差异, 而且这种差异在三个数据集上表现都是一样的, 说明这种现象存在普适性。

(2) 从表 3.5 中, 可以看出三个数据集在分别使用 SMOTE 算法和 C_SMOTE 算法后, 新产生的数据集对随机森林 OOB 估计的均值有较大的差异, 特别是在 PIMA 数据集中表示最为突出, 均值的绝对差值为 0.07, C_SMOTE 算法的均值更大些, 说明 C_SMOTE 算法的非平衡处理更优于 SMOTE 算法, 经 C_SMOTE 算法加工后的非平衡数据集更有利于提升随机森林算法的性能。

2. 数据集的不平衡率对随机森林算法性能影响程度分析

不平衡率达到怎样的水平将严重影响随机森林的分类性能, 或者说, 非平衡数据集经再抽样处理后的不平衡率达到多少时方可获得较好的分类效果, 一直没有明确定论。经过非平衡再抽样后, 数据集的不平衡率就会发生变化, 在不平衡再抽样算法中, 最终数据集的不平衡率仅受 M 值影响。 M 值是 SMOTE 系列算法中对每一个负类样本产生“人造”样本的个数, 随着 M 值的增大, 负类样本的数量将逐渐向正类样本靠拢, 并逐渐超过正类样本的数量, 最后将会导致新的不平衡状态出现, 因此分析 M 值的变化对随机森林的性能影响有很重要的意义。数据集的不平衡率和 M 值的关系如下:

$$nonRate = \frac{(M+1) * neNum}{poNum} \quad (3.11)$$

其中:

$neNum$ 为负类样本数;

$poNum$ 为正类样本数;

$M=1, 2, 3, \dots$ 。

为了简便起见, 此处仅以 yeast 数据集为例, 分析随着 M 值的增加, 不同算法对随机森林算法 G-mean 指标的影响程度。

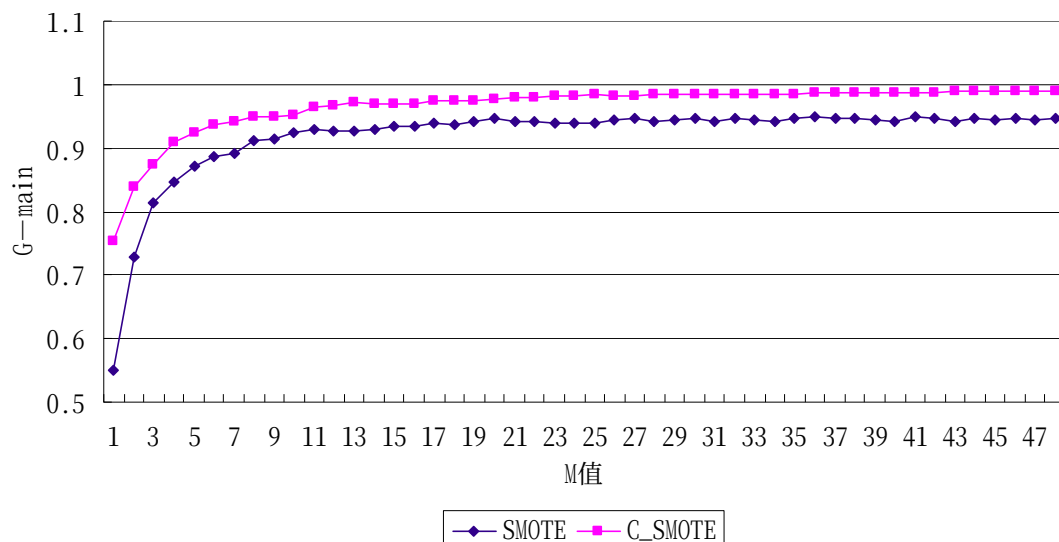


图 3.3 不同算法对 G—mean 指标的影响程度

从图 3.3 可以看出：

(1) 随着 M 值的增大，G—main 的值也在增加，但增大到一定程度后，不再有很大的变化，在 M=27 时，数据集的平衡率为 99.65%，基本达到平衡，此时 G—main 的值也趋于稳定。由于正类样本的数据量较大，要打破平衡需要更多的样本量，因此这个稳定会维持更长区间，由于篇幅的关系，本文不展开论述。

(2) 两种算法均对数据集产生了较好的影响，而且产生影响的效果很迅速，M 值仅从 1 增加到 11，就使 G—mean 指标趋近于 1，说明 SMOTE 算法和 C_SMOTE 算法均对提升随机森林算法的性能有较好的作用。

(3) C_SMOTE 算法对数据集的非平衡处理更有利于提升随机森林算法的性能。从图上可以看出，C_SMOTE 算法处理后的数据集的 G—mean 指标组成的曲线始终在 SMOTE 算法的上方，因此说明经 C_SMOTE 算法处理后的非平衡数据集更有利于提升随机森林算法的性能。

3.不同取得数据集中心方式对 C_SMOTE 算法的影响

为了测试不同取得数据集中心方式对 C_SMOTE 算法的影响，选用 Blood 数据集作为初始集，数据集中心点计算分别采用聚类法和向量重心两种办法。

首先，使用聚类的办法获取数据集的中心点。过程如下：将数据集引入到 SPSS 19.0 软件中，使用 K 均值聚类方法，选择聚为 2 类，最大迭代 100 次，最后得到 171 个样本为一类，其聚类中心向量为 $(-0.49, 0.17, 0.17, -0.16)$ ，记为 Center1。

然后，使用求各维平均值的办法求数据集的中心。过程如下：将初始数据集引入到 EXCEL 中，对每个自变量的按列计算平均值，然后将这些平均值组合在一起，就得到了数据集的中心点。引处得到的负类数据集的中心向量为 $(-0.5005, 0.3910, 0.3910, -0.0641)$ ，记为 Center2。

然后,分别从两个重心出发,对 Blood 数据集进行 100 次 C_SMOTE 算法非平衡抽样处理,将 100 次的数据集引入到 R 软件中,测试这些数据集的 G-main 指标的性能结果如下图所示:

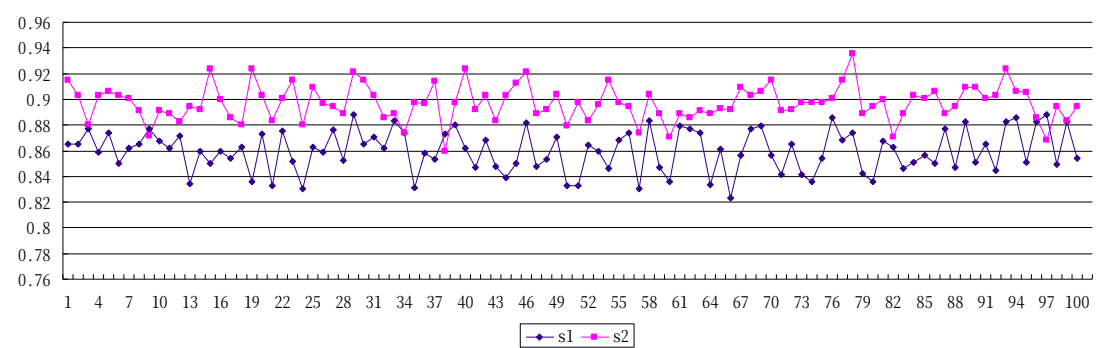


图 3.4 不同取得负类数据集重心方式对 G-main 指标的影响

从上图可以看出,在 100 次试验中,Center2 的 G-main 指标值基本在 Center1 的上方,也就是说 Center2 产生的数据集更优于 Center1,说明求负类数据集的重心时,使用计算向量各维平均值的方式,更优于使用聚类的方式。

也可以采用两两配对 T 检验的显著性检验方法上述实验的结果进行检验,进一步分析不同重心取得方式对算法性能的影响。

表 3.7 成对样本统计量信息

配对序号	重心名称	均值	样本数量	标准差	均值的标准误
1	center1	.859850	100	.0160913	.0016091
	center2	.897047	100	.0136642	.0013664

表 3.8 成对样本检验结果表

配对序号	重心名称	均值	标准差	95% 置信区间		t	df	Sig.(双侧)
				下限	上限			
1	center1	-.0371969	.0201684	-.0411987	-.0331950	-18.443	99	.000
	center2							

(1) 从表 3.7 中可以看出,和重心 center1 相比,重心 center2 取得的数据集产生的随机森林 G-main 的均值要高出 3.72%,也就是说重心 center2 产生的数据集,较好地提升了随机森林算法的分类性能。

(2) 在表 3.8 中,前二列数据分别体现了差值样本的均值差异和标准差,从这些数据可以看出,两个重心存在较大的差异,且差异化比较平稳。从最后 Sig.(双侧)指标为 0.00,可以得出结果,这两组数据的均值存在显著差异。因此可以从统计分析的角度,进一步验证了求负类数据集的重心时,使用计算向量各维平均值的方式,更优于使用聚类的方式。

3.6 本章小结

根据随机森林算法在非平衡数据集上性能较差的问题,本章在对非平衡数据进行描述后,着重介绍了非平衡数据分类问题的解决办法,并将解决这类问题的方法归纳了改造数据和改进算法两个方面,并对 SMOTE 算法进行了详细的描述,分析了 SMOTE 算法存在的问题,针对这些问题,提出了 C_SMOTE 算法,该算法主要针对 SMOTE 算法在“人造”样本的过程中存在的盲目性和边缘化问题,提出了向重心看齐的“人造”样本过程,很好地解决了 SMOTE 算法的问题。

在本章的第四部分,针对三个非平衡集的数据集,进行 C_SMOTE 的实证分析,实验数据表明, C_SMOTE 算法在优化随机森林算法性能方面,具有比 SMOTE 算法有更好的表现。在随机森林算法程序设计的时候,就可以考虑将本算法容纳到程序中,从而提高随机森林算法在处理非平衡数据集是的性能较差的问题。

4 随机森林处理连续变量的优化研究

在第二章的分析中可以看出,由于随机森林算法在处理连续属性变量时,采用二分离散化方法,该方法使得随机森林算法的运行效率受连续变量取值的影响很大,本章通过讨论连续变量离散化问题,研究通过连续变量离散化来降低连续变量取值的数量,从而达到提升随机森林算法运行效率的目的。

4.1 连续变量离散化概述

连续变量离散化是数据预处理重要内容之一。在机器学习和数据挖掘的规则提取、特征分类等算法中,特别是在使用决策树、随机森林等分类算法时,连续变量必须离散化^[61]。

4.1.1 连续变量离散化的概念

连续变量离散化是粗糙集和信息论的范畴。考虑信息系统 $S = \{U, A, V; F\}$, 其中 $U = (X_i)_{i=1}^m$ 由 m 个研究对象组成,称为论域; A 为属性集合; $V = \{V_a : a \in A\}$ 是属性取值集合; F 是 $U \otimes A \rightarrow V$ 的映射,即对于每一个具体的对象 X_i , 及 $\forall_a \in A$, 有 $v_a \in V_a$ 使得 $F_a : X_i \mapsto v_a$ 。若 $A = C \cup D, C \cap D = \Phi$, 其中 C 称为条件属性, D 称为决策属性, 则称 S 为决策表。

设在决策表 S 中有连续变量 $a \in C$, 则 V_a 为连续统。当 U 确定时, V_a 表现为离散的取值形式, 其递增序列可记为 $V_a = (a_1, \dots, a_n)$, 其中 $a_1 < a_2 < \dots < a_n$ 。则称 (a_{i-1}, a_i) 为区间, a_i ($i=1, 2, \dots, n$) 为节点, 简称为第 i 个节点。对于递增序列 V_a , 通过某种规则, 得出 m 个节点, 记为 a_j ($j=1, 2, \dots, m$), 得到新的递增序列 $V'_a = \{a'_1, a'_2, \dots, a'_m\}$, 其中 $m < n$ 。则称从 V_a 到 V'_a 的过程为连续变量离散化。

简而言之, 连续变量离散化就是根据某种规则, 合并一些近似的区间, 使连续变量的节点数减少, 从而实现数据的约简。因此, 离散化的关键在于如何制定规则, 获得最优的序列 V'_a , 该序列既可以简化信息表示, 又可以最大限度地保留原序列 V_a 所表达的信息。

4.1.2 连续变量离散化的衡量标准

连续变量的离散化就是在特定的连续变量的取值的范围内, 设定若干个区间, 最后用不同的符号或整数值代表落在每个子区间中的属性值。如何设定这些区间, 并使设定的区间更为合理, 是连续变量离散化研究的重要内容。如何衡量连续变量离散化算法的合理性, 就涉及连续变量离散化的衡量标准问题。从直观的角度看, 衡量一个离散化算法是否成功, 可以从以下几点出发:

1. 多变量处理能力。

多变量处理能力是指算法能够完成同一个数据集中多个连续变量的离散化过程,

而不需要人为的干预。一般来讲，一个数据集的连续性属性不止一个，因此设计出来的离散化方法不应该只能处理一个连续性属性变量。

2. 信息约简的效率高。

将多个区间的连续变量约简为数量较少的区间，这是连续变量离散化的根本目标，约简的效果越高，算法的效能越好，如何衡量算法的约简效果，需引入约简率，用于衡量算法对原数据集分类数约简的程度，其计算公式为

$$R_i = \frac{\sum_{i=1}^k (V_{ai} - V_{ai}')}{\sum_{i=1}^k V_{ai}} \quad (4.1)$$

其中：

i 为数据集中连续性属性的个数；

V_{ai} 为第 i 个属性的初始节点数；

V_{ai}' 为第 i 个属性使用算法约简后的节点数；

该比率越大，说明约简效果越好。

3. 较好地保持了原有数据集的信息。

一个好的连续变量离散化算法应在约简率高的情况下，使新产生的数据集充分保留原有数据集的信息。一般使用分类算法的分类精度来衡量原有数据集信息减少的程度，如果连续变量离散化算法产生的数据集比原有数据集在同一个分类算法中的精度有所下降，则说明数据集在约简的过程中，信息量也随之减少了，这是一种不好的离散化的结果。

4.1.3 随机森林算法中连续变量离散化方法

在随机森林中，如果存在连续型的属性变量，通用的做法是把这些连续型属性的值分成不同的区间，即“离散化”。在决策树节点分裂算法中，C4.5 算法就需要对连续变量离散化来达到处理连续变量的目的，这也是 C4.5 算法对 ID3 算法的改进之一。C4.5 算法在连续变量离散化时，采用二分离散化的方法，该方法的主要思想是：在训练集中，观察某个连续变量的取值范围，找出最大值和最小值，然后在最大值和最小值之间设置多个等分断点，这些等分断点将数据集分为二个小的集合，计算这些断点所产生的信息增益率，使用 C4.5 算法生成决策树。C4.5 算法中“离散化”的方法步骤是：

Step 1 对连续变量的值进行排序，找到最大值（记为 MAX）和最小值（记为 MIN）；

Step 2 设连续变量共有 N 个取值，取每个值为断点，则将区间【MIN, MAX】分成 $N-1$ 个区间；

Step 3 针对每个断点 A_i ， $i=1, 2, \dots, N$ ，分别计算【MIN, A_i 】和（ A_i , MAX】

($i=1,2,\dots,N$) 作为区间值时的 Gain 值, 并进行比较;

Step 4 选取 Gain 值最大的 A_k 作为该连续型属性的断点, 把属性值设置为【MIN, A_k 】和 (A_k , MAX】两个区间值。

从第二个步骤可以看出, 等分断点的个数由 N 决定, 当连续变量的取值 N 很多时, 使得等分断点的数量也会增加, 从而会产生更多的子集, 这会使得算法的时间复杂度进一步上升, 从而影响算法的执行效率。如果连续变量离散化的方法可以使连续属性变量的取值个数 N 更少, 那么就可以提升随机森林算法的执行效率。

4.2 连续变量离散化的方法简介

从本质上讲, 连续变量离散化的过程就是根据某种规则, 不断地把两个相邻区间进行合并成一个更大的区间, 从而使原来数量众多的区间用少数几个有代表性的区间来表示, 它可看成是对数据的约简过程, 该过程可以有效地降低数据集中数据的粒度。不同的规则就对应了不同的离散化算法, 连续变量离散化算法有很多, 根据分类标准的不同, 可以分为四类^{[62][63][64]}。

1. 全局离散化和局部离散化, 根据离散化时是否利用整个属性空间进行区别。全局离散化是指以整个样本空间为基础进行离散化, 而局部离散化指只在样本空间的部分区域内进行离散化。

2. 监督离散化和非监督离散化, 根据离散化时是否以类别信息做参考进行区别。有监督的方法是指在离散化过程中使用了类别信息, 反之便是无监督的方法。

3. 静态离散化和动态离散化, 根据离散化与决策树的生成是否并行进行区别。静态离散化指在离散化过程中需要指定一个参数作为最大离散类别。而动态离散化方法仅仅是按照统一的离散标准进行, 使得每一个样本的机会均等^[65]。

4. 归并离散化和拆分离散化, 根据是把属性的每个值作为一个离散属性值还是把整个属性值区间当作一个离散的属性值进行划分。在初始阶段把数据集中的每一个样本视作一个区间, 然后按照某种标准选取候选断点, 通过这些候选断点把数据集中的样本重新划分成若干个区间 (个数少于初始状态区间个数), 最后将候选断点划分的区间以某种离散判别式为标准进行合并, 这一个过程就是合并 (自底向上) 式离散化算法; 在初始阶段把数据集中的所有样本看成一个区间, 然后按照某标准选取候选断点, 再从这些候选断点中以某种离散判别式确定断点, 通过这些确定的断点将初始阶段的一个大区间重新划分成若干个较小区间 (个数大于初始状态区间个数), 这一个过程就是拆分 (自顶向下) 式离散化算法。

4.2.1 简单的连续变量离散化算法

1. 等宽算法

等宽算法是最简单的无监督的连续离散化算法。顾名思义, 等宽算法就是指离散化后, 得出的区间中离散属性的值均相等。首先, 它将须离散化的数据序列进行排序,

序列的最大值记为 MIN，最小值记为 MAX，然后根据用户指定的区间数 N，计算每个区间的长度 $W = (MAX - MIN) / N$ ，则区间边界值为 $MIN + W, MIN + 2W, \dots, MIN + (N-1)W$ 。这个算法比较简单，执行方便，但在使用时却存在很大的问题，在 Catlett 的文献中提到，如果序列中的数据存在极为严重的偏斜时，如某一个值在样本中所占的比重较大时，这种离散的方法基本无效。

2. 等频算法

等频算法也是一种无监督的连续离散化算法，该算法和等宽算法类似，离散化结束后，每个区间应包含数量相等的样本，划分的区间数还是由用户指定。两个算法不同之处在于，等频算法不是要求每个区间宽度相等，而是要求位于每个区间的样本数量相等，比如说 $N=10$ ，每个区间应该包含大约 10% 的样本。

无论是等宽算法还是等频算法，均容易实现，但因为这两个算法在划分区间时基本不考虑样本决策属性的分布，因而很难将区间的划分充分保留数据集的原有信息，很难以将区间的边界设置在最合适的位置上。有时还存在一些极端的情况，使得本来不存在相互联系的类，反而被划进了同一个区域，使得后续的数据分类效果比原来的差很多，因此在大多数情况下，这两种连续变量离散化算法的性能均不能达到令人满意的效果。

3. K-means 算法

这个算法的基本原理来源于统计的聚类思想，通过对数据的聚类，找出这些数据的不同类别，然后按照这个类别进行离散化。该算法的具体做法是，首先设定 K 个中心，按照距离最近的原则，把样本一个个地加入到不同的中心中，然后重新计算各个类别的质心，从而确定新的中心，这样一直循环往复，直到中心的距离小于某个阈值算法才结束。选择适当的初始点是该算法的关键。常见的方法有随机选择法、层次聚类法、初始质心法和 Canopy 的聚类算法

一般是目标函数达到最优或者达到最大的迭代次数即可终止。对于不同的距离度量，目标函数往往不同。当采用欧式距离时，目标函数一般为最小化对象到其簇质心的距离的平方和，公式如下：

$$\min \sum_{i=1}^K \sum_{x \in C_i} dist(c_i, x)^2 \quad (4.2)$$

当采用余弦相似度时，目标函数一般为最大化对象到其簇质心的余弦相似度之和，公式如下：

$$\max \sum_{i=1}^K \sum_{x \in C_i} cosine(c_i, x) \quad (4.3)$$

K-means 算法试图找到使平方误差准则函数最小的簇。当潜在的簇形状是凸面的，簇与簇之间区别较明显，且簇大小相近时，其聚类结果较理想。在处理大数据集

合，该算法非常高效，且伸缩性较好。但该算法存在三个难以解决的缺点：一是要事先确定簇数 K，二是初始聚类中心决定了算法的离散化性能，三是经常会陷入局部最优解。

4.2.2 基于信息熵的 CADD 系列算法

在基于信息熵的连续变量离散化算法中，比较有代表性的离散化方法是基于信息熵和互信息思想的 CADD 系列算法，主要包括：CADD 算法^[71]、CAIM 算法^[72]、CACC 算法^[73]。

CADD (Class-Attribute Dependent Discretizer) 算法^[71]由 J.Y. Ching 等人基于信息理论在 1995 年时提出，该算法使用类和属性之间的想到依赖程度来进行连续变量的离散化。该算法使用 CAIR 指标作为连续变量离散化的标准，该指标的公式如下：

$$CAIR(C, D | A) = \frac{I(C, D | A)}{H(C, D | A)} \quad (4.4)$$

其中 $H(C, D | A)$ 和 $I(C, D | A)$ 是第二章提到的信息熵理论的内容。CADD 算法在对训练集中的属性离散化时，就是以 CAIR 作为离散化标准的，按照 CADD 算法的思想，这样选择，就能够选出最好的断点。显然，这个算法是存在不足的，主要有三点，首先是要求训练置信区间，这个区间不能人为给定，其次是置信区间的数据由用户给定，使得算法存在不稳定性因素；第三是初始化是需要使用最大熵离散化法。如果这样的话，会就使得选择的初始断点很差，也会直接影响连续变量离散化的效果。针对这个问题，Lukasz A Kurgan 等人对算法进行了改进，提出了 CAIM 算法^[72]。该算法的离散化的标准指标采用如下公式计算：

$$CAIM(C, D | A) = \frac{\sum_{r=1}^n \frac{\max_r^2}{M_{+r}}}{n} \quad (4.5)$$

其中：n 为目前区间的数目；

$\max_r$ 为第 r 个区间中最大的类别数。

CAIM 指标为了使区间的数目最小，力图找出互相依赖程度最大的类与属性。该算法存在两个主要的缺陷，一是它过分强调最大依赖，没有充分考虑依赖度不高的类和属性；二是最终产生的离散化方案中区间数目不是很合理，有时候竟然和类的个数一样。

Cheng-Jung Tai 等人针对 CAIM 算法存在的问题，提出了 CACC 离散化标准^[73]，该指标的计算公式如下：

$$CACC(C, D | A) = \sqrt{\frac{x}{x + M}} \quad (4.6)$$

其中, $x = M \left[\left(\sum_{i=1}^S \sum_{r=1}^n \frac{q_{ir}^2}{M_i + M_r} \right) - 1 \right] / \log(n)$, $\log(n)$ 为 n 的自然对数。该标准相对

上两种标准, 考虑了数据整体的分布情况。另外, CACC 标准中除以 $\log(n)$ 有两个目的: 一是加速离散化进程, 二是防止在 CAIM 离散化标准中出现连续变量离散化过度现象。

4.2.3 基于统计学的 CHI2 系列算法简介

1. CHI2 系列算法简介

在 1992 年, Kerber^[66] 参照拟合优度的 Pearson 统计量, 提出了 Chi Merge 启发式连续变量离散化算法, 该算法通过构建统计量 x^2 , 来设计离散化的规则。 x^2 统计量的计算公式为:

$$x^2 = \sum_{i=1}^2 \sum_{j=1}^k \frac{(A_{ij} - E_{ij})^2}{E_{ij}} \quad j=1,2,\dots,k, i=1,2; \quad (4.7)$$

其中: i 为两个相邻的区间编号;

k 为决策类别数;

A_{ij} 为相邻两个区间中第 i 个区间中 j 类样本的个数;

E_{ij} 为 A_{ij} 的期望概率, 公式为:

$$E_{ij} = R_i \times C_j / N \quad (4.8)$$

其中: $R_i = \sum_{j=1}^k A_{ij}$ 为 i 区间中样本数;

$C_j = \sum_{i=1}^2 A_{ij}$ 为 j 类样本的个数;

N 为两个相邻区间中样本总个数。

1997 年, Liu 等人^[67] 改进了 CHI Merge 算法, 并在此基础上提出了 CHI2 算法: 通过预先设定一个 α 值, 根据相邻区间决策属性类别的个数减 1 得出自由度, 从而计算 x_a^2 值, 在计算出当前两个相邻区间的 x^2 值后, 计算论域内所有两个相邻区间的 $D = x_a^2 - x^2$ 值, 比较这些 D 值之间的大小, 取 D 值最小的两个相邻区间进行合并。通过动态地提升 α 值, 不断地合并区间, 达到信息约简的目的。合并结束的条件一般为不改变原系统的不一致率 $Incon_rate$ 。不一致率是粗糙集理论中的重要概念, 其公式为:

$$Incon_rate = 1 - \gamma_D \quad (4.9)$$

其中, γ_D 为决策属性 D 在决策表 S 中的近似精度。

2002 年, Tay 等人^[68] 对 CHI2 算法进行了改进, 提出了 Modified CHI2 算法 (以下简称 MCHI2), 他们认为: CHI2 算法中 x_a^2 的自由度应是当前断点相邻的两个区

间中决策类的个数，而不应该是整个论域中决策类的个数。

2005 年, Su 等人^[69]提出了 ECHI2 算法, 该算法认为 MCHI2 算法中计算 D 值时, 应考虑两个区间中决策类的个数, 这样有利于不同自由度的 D 值进行比较, 提出了 D 的计算公式为:

$$D = (x_a^2 - x^2) / \sqrt{2v} \quad (4.10)$$

其中 v 为自由度。

2. CHI2 系列算法分析

CHI2 系列算法的关键过程是使用公式 (4.4) 构造的 x^2 统计量, 其构造过程是模拟皮尔逊的拟合优度检验的统计量, 即 Pearson x^2 统计量:

$$x^2 = \sum_{i=1}^k \frac{(X_i - np_i)^2}{np_i} \quad (4.11)$$

其中第 i 类的概率为 p_i , 第 i 类的期望数即理论数为 T_i , $T_i = np_i$

在公式 (4.7) 中, 如果把 C_j/N (j 类样本在相邻区间样本个数中占的比例) 看作是 j 类样本的概率, 简记为 p_j , 则 $E_{ij} = R_i \times C_j/N$ 就是在 p_j 概率下第 i 个区间中 j 类样本的期望值, 简记为 T_{ij} 。由此可以看出, 公式 (4.7) 的统计量可以看成是在两个相邻区间中, 所有类别分布的均匀程度。该统计量越小, 说明相邻两区间中各类样本分布越均匀, 则节点就越不重要, 由该节点决定的两个区间应被合并。

已经证明, 当 n 充分大时, 由 (4.7) 式所定义的统计量近似服从 x^2 分布。但是, 由于近似性的存在, 在利用此统计量计算时, 常常会比正常值偏低, 特别是当自由度为 1 时, 偏差的程度更大。对此, 杜荣骞 (2003) 认为^[70], 当 $df=1$ 时, (4.7) 式应做连续性矫正, 矫正的 x^2 为

$$x^2 = \sum_{i=1}^k \frac{(|X_i - np_i| - 0.5)^2}{np_i} \quad (4.12)$$

根据 Kerber 的理论, 在公式 (4.12) 中, 统计量的自由度是相邻两个区间的决策属性类别减 1, 当研究的总体的决策属性比较少时, 特别是在总体的决策属性为 2 时, 统计量的自由度为 1 的情况特别多, 因此如果不对统计量进行矫正, 则会使算法的结果出现较大规模的偏差, 计算出来的 D 值也不具有可比较性。针对此问题, 本文设计一种新的连续变量离散化算法, 新算法和 CHI2 系列算法的最大区别在于, 当在自由度为 1 时, 新算法根据公式 (4.12) 的 x^2 矫正公式计算 x^2 值, 这样能使算法避免在计算 D 值时出现大规模的偏差, 使得数据的约简过程更加合理。

4.3 随机森林处理连续变量的优化—COR_CHI2 算法

从上面的分析可以看出, 随机森林算法中 C4.5 算法采用的二分离散化方法, 使

得算法的效率和连续变量的取值有很大关系,因此如果能一种有效地约简数据集的方法,降低连续变量的取值,那么就可以提升随机森林算法的执行效率。约简数据集的方法有很多,在第二节中已经做了介绍,本节拟针对 CHI2 算法的固有问题,提出一种新的连续变量离散化算法,简记为 COR_CHI2 算法,用于随机森林算法的优化,从而提升随机森林算法的执行效率。

4.3.1 COR_CHI2 算法设计

根据杜荣骞(2003)提出的 x^2 矫正公式,本文设计出一种新的基于 x^2 矫正的连续变量离散化算法,简记为 COR_CHI2 算法,其计算过程如下:

1. 计算某一属性取值的两个相邻区间中,决策属性的个数 k
2. 计算理论次数 E_{ij} , 公式为

$$E_{ij} = R_i \times C_j / N \quad (4.13)$$

其中: $R_i = \sum_{j=1}^k A_{ij}$ 为 i 区间中样本数;

$C_j = \sum_{i=1}^2 A_{ij}$ 为 j 类样本的个数;

N 为两个相邻区间中样本总个数。

3. 若某组的理论次数 E_{ij} 小于 5, 则应把它与其相邻的一组或几组合并, 直到理论次数大于 5 或者一个区间只有一组数据为止, 重新计算 k 值。

4. 计算 x^2 统计量的值, 分两种情况

(1) 当 $k < 2$ 时, x^2 统计量的公式为

$$x^2 = \sum_{i=1}^2 \sum_{j=1}^k \frac{(|A_{ij} - E_{ij}| - 0.5)^2}{E_{ij}} \quad j=1; i=1, 2; \quad (4.14)$$

(2) 当 $k \geq 2$ 时, x^2 统计量的公式为

$$x^2 = \sum_{i=1}^2 \sum_{j=1}^k \frac{(A_{ij} - E_{ij})^2}{E_{ij}} \quad j=1, 2, \dots, k; i=1, 2; \quad (4.15)$$

其中:

k 为目标变量类别数;

i 为两个相邻的区间编号;

A_{ij} 为相邻两个区间中, 第 i 个区间中 j 类样本的个数;

5. 区间合并的顺序由 D 值确定, 其公式如下:

$$D = (x_a^2 - x^2) / \sqrt{2v} \quad v \text{ 为自由度} \quad (4.16)$$

6. 选出 D 值最小的区间进行合并。

对数据集中所有的连续属性变量进行上述六个步骤后,就完成了对数据集的约简过程,即实现了连续变量的离散化。

4.3.2 COR_CHI2 算法程序流程图

COR_CHI2 算法的程序流程图如下：

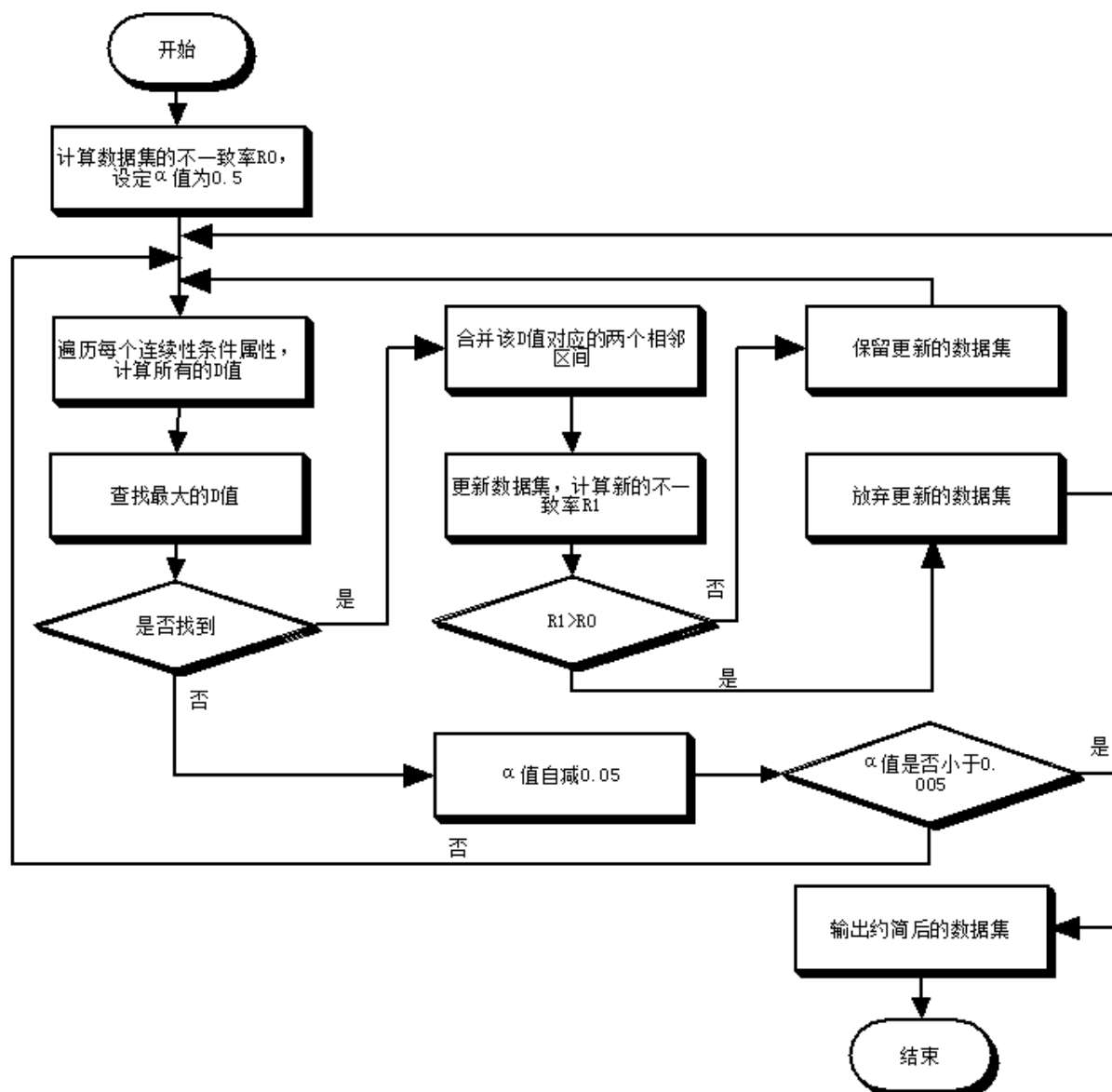


图 4.1 COR_CHI2 算法程序流程图

4.4 COR_CHI2 算法对随机森林算法性能优化实证分析

4.4.1 实证分析使用的数据集介绍

为不失一般性, 本文选取了 UCI 机器学习数据库的 Iris、Credit 和 Blood 数据集作为实验数据, 其变量如下表：

表 4.1 三种数据集的变量情况表

数据集	连续变量	离散变量	决策变量类别数	样本数	初始节点
Iris	4	0	3	150	123
Credit	6	9	2	670	1100
Blood	4	0	2	748	175

表 4.2 初始连续变量节点数分布表

数据集	变量 1	变量 2	变量 3	变量 4	变量 5	变量 6	合计数
Iris	35	23	43	22			123
Credit	340	213	131	23	164	229	1100
Blood	31	33	33	78			175

表 4.3 连续变量名称对应表

数据集	Iris	Credit	Blood
变量 1	Petal.Length	A2	Recency
变量 2	Petal.Width	A3	Frequency
变量 3	Sepal.Length	A8	Monetary
变量 4	Sepal.Width	A11	Time
变量 5		A14	
变量 6		A15	

4.4.3 COR_CHI2 算法对数据集的约简效果分析

三种算法均使用 JAVA 软件开发, 开发平台为 MyEclipse 7.0+JDK1.6, 程序部分源代码参见附录 B。在离散化过程中, 三种算法都要取得卡方分位点, 在实现时, 卡方分位点均通过 R 3.0.2 软件获得, 随机森林算法使用 R 3.0.2 软件的语言软件包 randomForest 4.6-6 来实现。

1. COR_CHI2 算法提升了对数据集的约简率

将 Iris、Credit 和 Blood 中的连续变量分别使用三种算法进行离散化, 数据约简效果如下表:

表 4.4 三种算法对 Iris 数据集数据约简效果表

算法名称	变量 1	变量 2	变量 3	变量 4	合计数	约简率(Ri)
初始数据集	35	23	43	22	123	--
ECHI2 算法	31	21	43	22	117	4.88%
MCHI2 算法	24	19	38	20	101	17.89%
COR_CHI2 算法	17	13	33	16	36	35.77%

表 4.5 三种算法对 Blood 数据集约简效果表

算法名称	变量 1	变量 2	变量 3	变量 4	合计数	约简率(Ri)
初始数据集	31	33	33	78	175	--
ECHI2 算法	30	33	33	73	169	3.43%
MCHI2 算法	30	33	33	73	169	3.43%
COR_CHI2 算法	28	26	27	72	153	12.57%

表 4.6 三种算法对 Credit 数据集约简效果表

算法名称	变量 1	变量 2	变量 3	变量 4	变量 5	变量 6	合计	约简率(Ri)
初始数据集	340	213	131	23	164	229	1100	--
ECHI2 算法	18	17	15	6	13	12	81	92.64%
MCHI2 算法	18	17	9	6	13	12	75	93.18%
COR_CHI2 算法	16	2	25	4	3	2	52	95.27%

从表 4.4-6 可以看出，在三个数据集中，不同的约简算法均有不错的表现，达到了连续变量离散化的目的。从约简效果看，而 COR_CHI2 算法的约简率明显高于 CHI2 系列算法，在 IRIS 数据集中 COR_CHI2 算法较 MCHI2 算法翻了一番；在 Blood 数据集中，COR_CHI2 算法较 CHI2 系列算法翻了 4 倍。因此，从信息约简的程度看，COR_CHI2 算法表现为突出，说明 COR_CHI2 算法可以大规模约简数据集中的冗余信息。

将三个表中的数据转化为散点图的形式，可得下图：

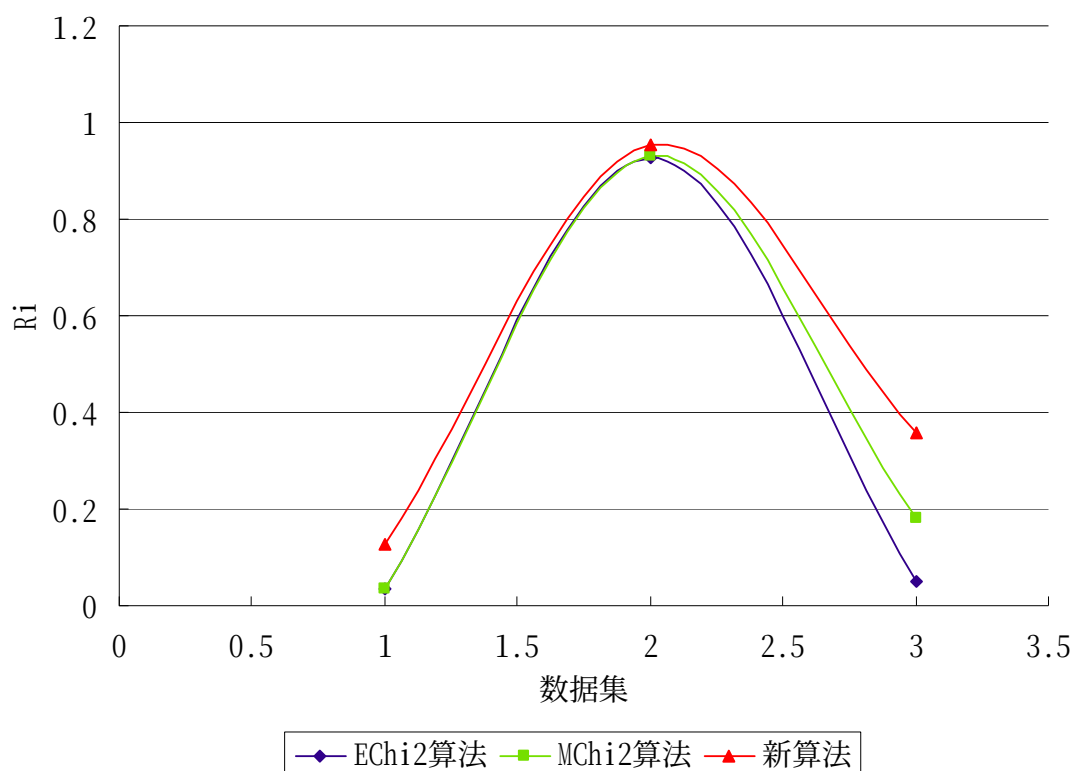


图 4.2 三种算法在三种数据集中平衡率的表现

从图上可以看出，对三个数据集，CHI2 系列算法的约简率基本差不多，而 COR_CHI2 算法的约简率一直在两者之上。

2. COR_CHI2 算法较好地保持了原有数据集的信息

从约简率看，COR_CHI2 算法占明显优势，但过量的信息约简可能数据集的中原有的信息流失，从而使数据集对分类算法性能带来不利的影响。为了全面衡量 COR_CHI2 算法的离散化性能，可以使用随机森林算法，观察数据集在约简前后，随

机森林算法在数据集上的分类性能的差异。

使用上述三个数据集，分别使用初始数据集和三种不同算法约简后的数据集，在保持随机森林算法参数一致的情况下，进行 100 次实验，取不同数据集上的随机森林的 OOB 估计的均值进行比较，分析 COR_CHI2 算法的离散化性能，实验数据如下：

表 4.7 使用随机森林对各种数据集处理后的 OOB 估计的均值

数据集	IRIS	Credit	Blood
初始数据集	0.0533	0.1300	0.2170
经 ECHI2 算法处理后	0.0513	0.1324	0.2128
经 MCHI2 算法处理后	0.0467	0.1328	0.2137
经 COR_CHI2 算法处理后	0.0457	0.1295	0.2120

从表 4.7 可以看出，在随机森林算法中，使用 COR_CHI2 算法对数据集进行约简后，算法的 OOB 估计值得到了降低，说明约简后的数据集性能不但没有下降，还使随机森林的精度得到了提升。从连续变量离散化的衡量标准来看，COR_CHI2 算法既提升了数据集约简率，也较好地保持了原有数据集的信息。

也可以采用两两配对 T 检验的显著性检验方法对两个算法的实验结果进行对比验证。不失一般性，将 iris 数据集上得到的 100 个配对数据，在 SPSS19.0 中进行两两配对 T 检验，检验结果如下表所示：

表 4.8 成对样本统计量信息表

配对序号	算法名称	均值	样本数量	标准差	均值的标准误
1	o	0.053293	100	0.0132389	0.0018538
	t	0.045727	100	0.0141705	0.0019843
2	e	0.051341	100	0.0133971	0.0018760
	t	0.045727	100	0.0141705	0.0019843
3	m	0.046723	100	0.0152014	0.0021286
	t	0.045727	100	0.0141705	0.0019843

表 4.9 成对样本检验结果表

配对序号	配对算法	均值	标准差	95% 置信区间		t	df	Sig. (双侧)
				下限	上限			
1	o - t	.0075666	.0189094	.0022482	.0128849	2.858	99	0.006
2	e - t	.0056139	.0214281	-.0004129	.0116407	1.871	99	0.067
3	m - t	.0009957	.0231817	-.0055242	.0075157	.307	99	0.760

在表 4.8-9 中, o 代表初始数据集, e 代表经 ECHI2 算法处理后的数据集, m 代表经 MCHI2 算法处理后的数据集, t 代表经 COR_CHI2 算法处理后的数据集, 从这个两表可以看出:在提升随机森林算法性能上看, COR_CHI2 算法产生的数据集, 较 ECHI2 算法产生的数据集和初始数据集都有显著的差异, 也就是说 COR_CHI2 算法既提升了数据集的约简率, 也较好地保持了原有数据集的信息。这样通过 COR_CHI2 算法的离散化处理, 使数据集的冗余信息得到了去除, 实现了连续变量离散化的主要目的。

4.4.4 COR_CHI2 算法约简后的数据集对随机森林算法运行效率的影响分析

从前面的对随机森林算法处理连续性数据时的采用的办法可以看出, 约简率越高, 连续变量的取值就会越少, 从而使算法在处理的过程中执行的效率更高。由于 COR_CHI2 算法可以有效地去除数据集中的冗余信息, 会使连续属性变量的取值减少, 从而可以较好地提升随机森林算法的运行效率, 这一点可以通过实证的方式进行进一步的分析。

继续使用上述三个数据集, 观察没有经过离散化算法处理过的初始数据集和经过三种算法处理的事的数据集, 对随机森林算法运行时间的要求, 来反应不同算法的性能。实验过程中, 考虑计算机系统运行的不稳定性, 观察指标为算法运行 100 次所需时间的平均值。

表 4.10 不同算法处理后的数据集在 RF 上运行的平均时间比较

(单位: 毫秒)

数据集	IRIS	Credit	Blood
初始数据集	81.40	69.78	75.86
经 ECHI2 算法处理后	83.44	67.88	74.38
经 MCHI2 算法处理后	82.41	68.10	74.22
经 COR_CHI2 算法处理后	82.29	52.49	73.18

从表 4.10 可以看出:

1. 经离散化处理后的数据集, 在随机森林算法中运行的时间均有所下降, 也就说明使用离散化处理算法可以提升随机森林算法的执行效率。

2. 在 IRIS 数据集和 Blood 数据集上, 由于数据的约简效果不明显, 因此随机森林算法的运行效率的提升也不够好, 但在 Credit 数据集上, 由于数据约简率很高, 因此随机森林算法的运行效率得到了较好的提升, 初始数据集经 COR_CHI2 算法处理后, 在随机森林算法上招待时间提升了 1.52 毫秒, 也就是说数据的约简率越高随机森林算法的运行效率就越好。

3. 从三种算法比较看, 由于 COR_CHI2 算法的约简率都超过 CHI2 系列算法, 因此使用 COR_CHI2 算法进行数据约简后的数据集, 使随机森林算法运行效率的提升的效果更明显, 也就是说 COR_CHI2 算法能较好地提升随机森林算法的性能。

综上所述, 本文提出的 COR_CHI2 算法, 提高了约简率, 在对约简后的数据集

在随机森林算法进行测试时，在随机森林算法中的执行时间，较 CHI2 系列算法具有明显的优势，较好地提升了随机森林算法的执行效率。

4.5 本章小结

针对随机森林算法中节点分裂时，C4.5 算法采用的二分离散化使算法效率下降的问题，在定义连续变量离散化的范畴、分析常用连续变量离散化方法，尤其是 CHI2 系列算法的基础上，针对 CHI2 系列算法没有考虑 χ^2 统计量和真实值之间存在偏差的问题，设计了一种改进的连续变量离散化算法，阐述了 COR_CHI2 算法的计算过程，并给出了算法的程序流程图。

在本章的第四部分，通过使用三种通用的 UCI 数据集，将 COR_CHI2 算法和没有解决偏差问题 CHI2 系列算法，在改善随机森林算法性能方面进行了比较分析。实验数据表明，和 CHI2 系列算法相比，COR_CHI2 算法能更有效地约简数据集中的冗余信息，而且能提升了数据集在随机森林算法上的执行效率。在对随机森林算法的执行效率要求很高的情况下，COR_CHI2 算法具有较好的推广价值。

5 随机森林算法分类性能的优化研究

从第二章的分析可以看出,随机森林算法的分类精度存在一定的优化空间,尤其是在有些应用和数据集上,本文拟在分析随机森林算法分类精度的影响因素出发,比较不同节点分裂算法的区别和联系,提出一种随机森林的节点分裂混合算法,进而达到提升随机森林算法分类性能的目的。

5.1 随机森林算法分类性能的影响因素分析

5.1.1 森林中决策树的数量

作为一种典型的组合分类器,森林中决策树的数量(简记为 $ntree$)对算法的性能有极大的影响。关于随机森林中树木的个数,文献 [90] 认为随机森林中决策树的个数 N ,对于随机森林算法的效率有着至关重要的影响。 $ntree$ 过大,则随机森林算法的运行效率会下降,而且决策树的数量过多,并不一定能对最终的分类结果产生影响;而 $ntree$ 过小,则随机森林的分类精度将有可能下降,或者出现决策树的过拟合现象。Wei Fan 在文献^[91]中对 N 的选取进行了研究,其结论表明当 $ntree=1000$ 时,随机森林将有很好的分类精度且可信度大于 99.7%。这两个文献的结论是否符合实际的结果,还需要进一步的验证。

1. 森林中决策树的数量对算法精度的影响

为了分析随机森林中决策树的数量对随机森林算法性能的影响程度,可以在其他参数不变的情况下仅观察森林中决策树的数量($ntree$)的变化对 OOB 估计的影响。训练集为 Blood 数据集,选择该数据集的原因是其分类精度不够高,使用 R 软件中的随机森林算法,随机特征变量 F 的值设为 3。使用随机森林算法对 Blood 数据集进行训练,以随机森林算法的 OOB 估计为观测指标。此处 OOB 估计取每个森林在形成过程中每棵树的 OOB 估计的平均值。由于 OOB 估计是随机森林算法泛化误差的无偏估计,该值越小说明随机森林算法的泛化误差越小,算法的分类精度就越高。在实验的过程中,分析在 $ntree$ 逐渐增大的情况下,对每个森林分别进行 1000 次实验,观察 OOB 估计的平均值变化情况,实验结果如下图所示:

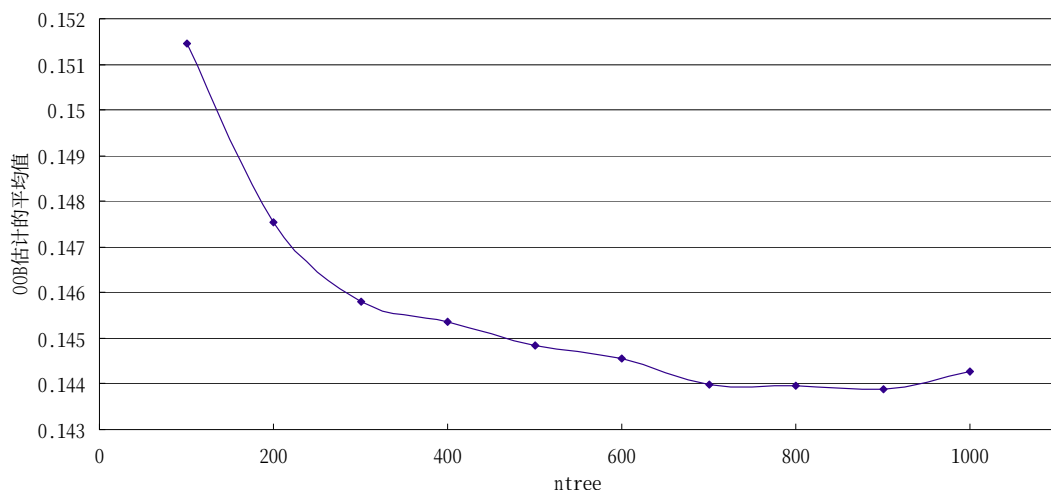


图 5.1 森林中决策树的数量和 OOB 估计之间的关系

从图 5.1 中可以看出

(1) 在决策树的数量为 100—700 时，OOB 估计随着森林中决策树的数量增加而逐渐减少，说明随机森林算法的分类精度随着森林中树木的个数增大而提高。

(2) 在决策树的数量为 700—1000 时，森林中决策树的数量继续增加时，OOB 估计基本不再变化，也就是说，当决策树的数量达到一定程度后，随机森林算法的分类精度趋向稳定。

2. 森林中决策树的数量对随机森林算法收敛性的影响

重复上面的实验，取每个森林形成过程中的每棵树的 OOB 估计的标准差为观测指标，对每个森林分别进行 1000 次实验，取观测指标的平均值，实验结果如下图所示：

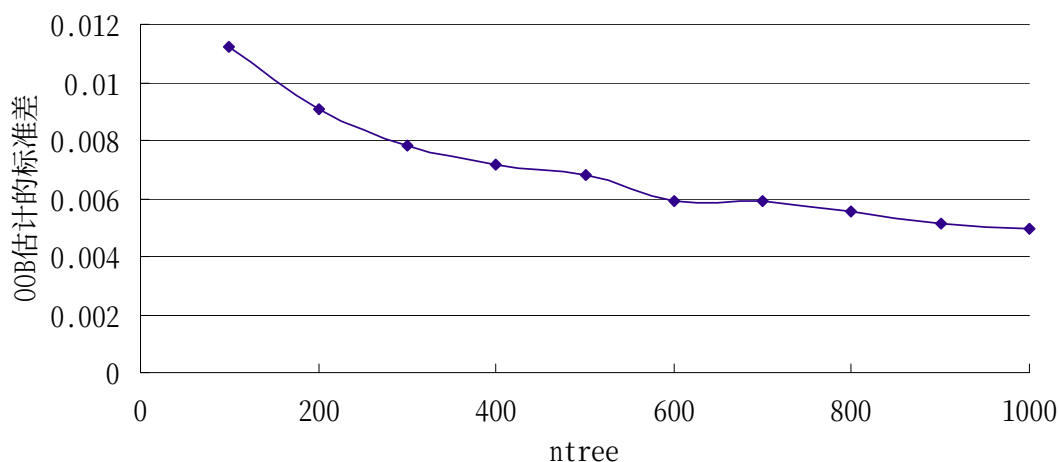


图 5.2 森林中决策树的数量对算法收敛性的影响

从图 5.2 中可以看出，OOB 估计的标准差随着森林中决策树的数量增大而逐渐

减小,说明 OOB 估计值随着 $ntree$ 的增大而越来越稳定,这也从另一个侧面说明了随机森林算法收敛的。在第二章随机森林算法的性质中可以看出,泛化误差从理论上证明了随机森林具有良好的收敛性,由于 OOB 估计是泛化误差的无偏估计,本实验从实证的角度也证明了随机森林具有良好的收敛性,即森林中决策树的数量的增加算法会越来越稳定。结合图 5.1 和图 5.2,可以看出,随机森林中决策树的数量对算法精度有较大的影响,且其数量的增大会使生成的随机森林的性能更加稳定。

5.1.2 随机特征变量 F 的值

F 对于单棵决策树的规模有着直接的影响,并且由于属性选择的随机性, F 过小则很有可能导致构建的每一个单棵决策树分类精度都不高,从而使得随机森林分类器分类精度不高; F 过大则随机森林规模增大,运行时间会更多。Breiman 研究了随机特征数与强度和关系数的关系以及随机特征数与泛化误差的关系,发现对于样本量较小的数据集(比如小于 1000),随着随机特征个数的增加,强度基本保持不变,但是相关系数会相应增加,测试集误差和 OOB 误差比较接近,都随着随机特征数的增加而增加,但 OOB 误差更加稳健。但对于大样本量(比如大于 4000),结果与小样本量不同,强度和关系数都随着随机特征数的增加而增加,而泛化误差随着随机特征变量的增加而略微减少。

简而言之,随机森林的强度和相关性都依赖于 F 的大小,如果 F 足够小,树的相关性趋向于减弱;另一方面,分类模型的强度随着随机特征变量数 F 的增加而提高,从而影响随机森林算法的分类精度。由于在每一个结点仅仅需要考察随机特征变量的一个子集,这种方法显著减少算法的运行时间。

1. 随机特征变量 F 的值对随机森林算法分类精度的影响

为了分析随机特征变量 F 的值对随机森林算法性能的影响程度,可以在其他参数不变的情况下仅观察 F 值的变化对 OOB 估计的影响。训练集为 Credit 数据集,选择 Credit 数据集的原因是它的样本属性有 15 个,便于观察 F 值的变化对分类精度的影响,使用 R 软件中的随机森林算法,森林中决策树的数量 $ntree$ 设为 1000。以 OOB 估计为观测指标,OOB 估计取每个森林在形成过程中每棵树的 OOB 估计的平均值,在随机特征变量 F 逐渐增大的情况下,对不同 F 值形成的森林分别进行 1000 次实验,观察 1000 次实验的 OOB 估计的平均值,实验结果如下图所示:

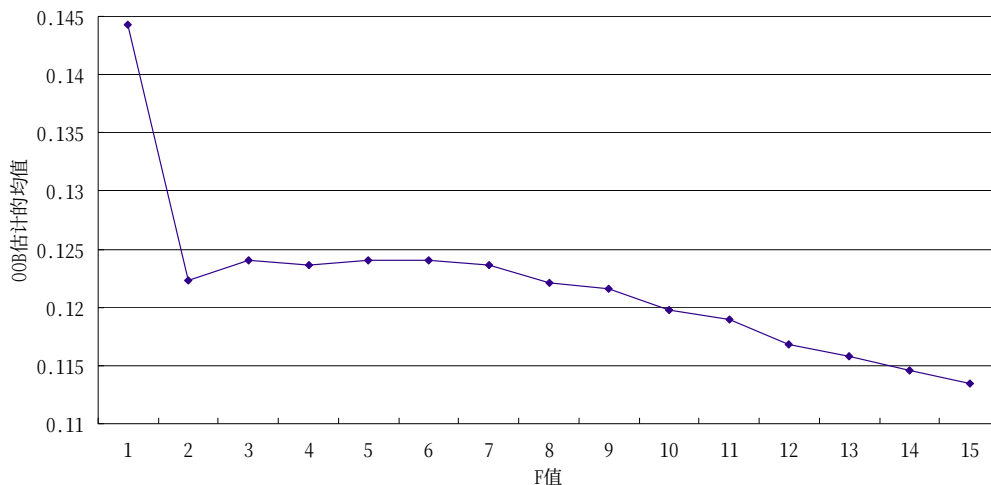


图 5.3 F 值对随机森林算法分类精度的影响

从图 5.3 中可以看出

(1) F 取值为 1 和 2 时，OOB 估计的变化很剧烈，说明 F 值对随机森林算法的性能影响很大。

(2) 在 2—15 之间，OOB 估计随着森林中 F 值增大而逐渐减少，说明 F 值是随机森算算法分类性能是正相关的。但从量的变化上看，OOB 估计总化变化在 0.01 之间，说明，F 值对随机森林算法分类性能的正相关关系是平缓的，也就是说随机森林算法的分类性能受 F 值的影响，但该值达到一定程度后基本变化不明显。

2.F 值对随机森林算法运行效率的影响

重复上面的实验，取每个森林形成过程中的所花费的运行时间为观测指标，对每个森林分别进行 1000 次实验，取观测指标的平均值，实验结果如下图所示：

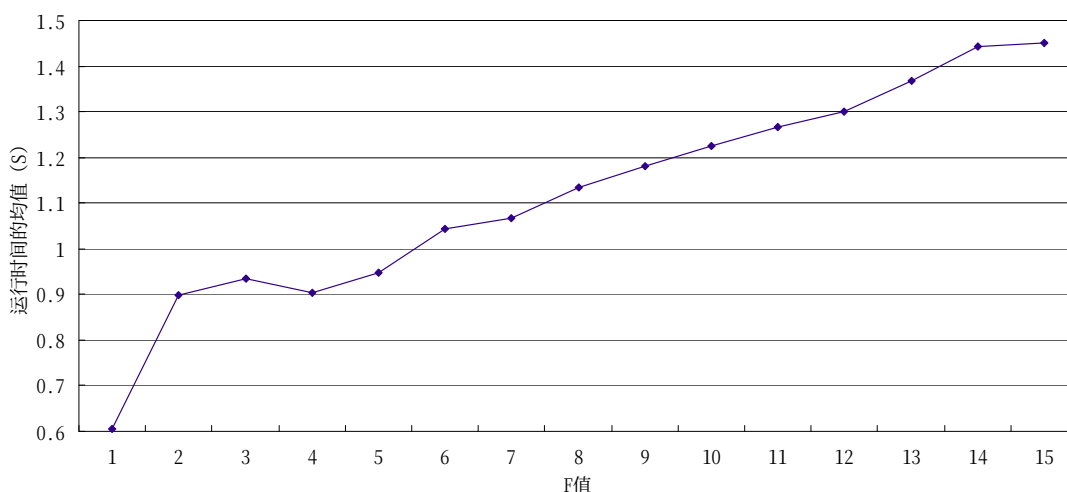


图 5.4 F 值对随机森林算法运行效率的影响

从图 5.4 中可以看出，算法平均运行时间随着 F 值的增加而逐渐加大，最大时间是最小时间的两倍，因此随着 F 值的增加参加属性选择的属性也随着增加，因此算法

在生成决策树时和进行决策时均需要耗用更多的时间，从而使得算法的运行效率下降。

综合图 5.3 和 5.4 可以看出，随机特征变量 F 的值达到一定程度后，将对算法的影响不太明显，但随着 F 值的增加会使决策树的运行效率下降，因此需要根据实际情况选择最佳的 F 值。Breiman 在文献中对 F 值取值进行了探讨，并通过实验验证在属性个数较少时， F 的值取 $\log_2 M + 1$ 较合适，其中 M 是数据集中样本属性个数。

5.1.3 不同的节点分裂算法比较

决策树的节点分裂算法很多，包括 CLS、ID3、C4.5、CART 等算法，这些方法的数学公式在第二章已经详细进行了介绍，此处不再重复。为了更好地了解这些算法的区别，首先比较一下这些算法在形成决策树的过程及结果的区别。

1. 不同算法形成决策树的过程及结果分析

考虑如下一个数据集：

表 5.1 数据集内容一览

序号	e1	e2	e3	e4	e5	e6	e7	e8	e9	Y
1	b	u	g	c	v	f	f	f	g	no
2	b	u	g	k	v	f	f	t	g	no
3	b	y	p	k	v	t	t	t	g	yes
4	b	u	g	m	v	f	f	t	s	no
5	b	u	g	m	v	t	f	t	s	no
6	b	u	g	cc	bb	f	f	f	g	no
7	a	u	g	x	h	t	f	f	g	yes
8	b	u	g	k	h	f	f	t	g	no
9	b	y	p	aa	v	f	f	t	g	no
10	b	u	g	c	v	f	f	t	g	no
11	b	u	g	w	v	t	t	t	g	no
12	b	u	g	m	v	t	t	t	g	yes
13	a	u	g	c	v	t	t	t	g	yes
14	b	u	g	m	v	f	f	t	s	no
15	b	u	g	e	bb	t	f	t	s	no
16	b	y	p	d	v	f	f	t	g	yes
17	b	y	p	i	bb	f	f	t	g	no
18	a	u	g	c	h	t	f	t	g	yes
19	a	y	p	j	ff	f	f	f	g	no
20	a	u	g	ff	ff	f	f	f	g	no
21	b	u	g	ff	ff	t	t	f	g	no
22	b	u	g	e	v	f	t	f	s	yes

23	b	u	g	aa	bb	f	f	t	g	no
24	b	u	g	i	bb	f	f	f	g	no
25	b	u	g	k	v	t	f	f	g	yes
26	b	u	g	i	bb	t	f	f	g	no
27	b	u	g	q	v	t	t	t	g	yes
28	b	y	p	w	v	t	t	f	g	yes
29	b	y	p	c	v	t	f	f	g	no
30	b	u	g	aa	v	f	t	t	g	no
31	b	u	g	x	h	t	f	t	g	no
32	b	u	g	w	v	t	f	t	g	yes
33	b	u	g	aa	v	t	f	t	g	no
34	b	y	p	i	h	t	t	t	g	no

表 5.1 是一个从 Credit 数据集中截取的一部分内容, 数据集中一共有 34 个样本, 每个样本有 9 个条件属性和 1 个目标属性。为了分析各种算法在节点分裂时的区别, 可以分析对该数据集的进行根节点节点分裂的计算过程。不同节点分裂算法在计算分裂规则时, 计算结果如下表所示:

表 5.2 三种节点分裂算法在根据节点分裂时各指标计算结果表

属性名称	信息增益(Gain)	信息增益率(GainRatio)	Gini 系数
e1	0.040604	0.067400	0.411359
e2	0.002641	0.003356	0.436086
e3	0.002641	0.003356	0.436086
e4	0.353627	0.101369	0.316585
e5	0.181407	0.112838	0.362353
e6	0.122972	0.123279	0.367647
e7	0.101462	0.116092	0.374020
e8	0.000172	0.000184	0.437611
e9	0.009320	0.015471	0.432454

从表 5.2 可以看出, 根据 ID3 算法的规则, 信息增益(Gain)指标越大则选择该属性, 因此 ID3 算法分裂时会选择 e4 属性; 根据 C4.5 算法的规则, 信息增益率(GainRatio)指标越大则选择该属性, 因此 C4.5 算法分裂时会选择 e6 属性; 根据 CART 算法的规则, Gini 系数指标越小则选择该属性, 因此 CART 算法分裂时会选择 e4 属性。由此可以看出三种算法在节点分裂选择属性时存在差别。另外, 根据这个数据集 CART 算法和 ID3 算法选择的属性一样, 并不说明这两个算法在分裂时会一直保持一致, 在有些数据集中, 也会出现不一致的情况。

2. 不同节点分裂算法对随机森林性能影响分析

根据上述分析可以看出, 不同分裂算法在进行节点分裂时, 选择的属性会出现不

致的现象,这势必会使最后得出的决策树的结构会出现较大的不同,从而使算法的性能受到影响。为了分析不同算法对随机森林性能的影响的量化信息,考虑到现有的随机森林算法一般都是采用其中一种方法,没有办法全部进行比较。另外,由于程序设计者的风格不同,算法代码结构也不尽相同,可比性也较差。因此需对上述 ID3、C4.5、CART 的算法进行编程实现,通过在相同的数据集、随机森林中其他参数不变的情况下,进行比较分析。

数据集分别使用 Blood、Haber 和 Credit,程序源码参见附录 C,对不同的算法进行 1000 次实验,每次实验时森林中决策树的数量为 1000, F 值取为 $\log_2 M + 1$,其中 M 是数据集中样本属性个数。取 1000 次实验的 OOB 平均值,其结果如下表所示:

表 5.3 三种算法在三个数据集上 OOB 估计比较

算法名称	Blood	Haber	Credit
CART	0.216730	0.243949	0.169573
C4.5	0.224584	0.264899	0.167324
ID3	0.227687	0.264862	0.167295

从表 5.3 中可以看出:

- (1) 在各个数据集上, C4.5 和 ID3 算法的性能基本相近。
- (2) 在各个数据集上, CART 算法的性能较 C4.5 和 ID3 算法均有大差异,但两者在不同的数据集上的性能时高时低,比如在 Blood 数据集上, CART 算法性能好些,而在 haber 数据集上, CART 算法的性能要差一些。虽然这三种算法有相同的理论基础,但因为采取的计算方式不同,而产生了差异。

为了深入研究不同节点分裂算法对随机森林性能影响,下面以 Blood 数据集为例,将 1000 次实验结果进行两两配对 T 检验,检查 CART 算法较 C4.5 和 ID3 算法在 OOB 估计的均值上的差异,实验结果如下:

表 5.4 成对样本统计量信息

配对序号	算法名称	均值	样本数量	标准差	均值的标准误
1	GART	0.216730	1000	0.0272346	0.0027235
	C4.5	0.224584	1000	0.0256366	0.0025637
2	GART	0.216730	1000	0.0272346	0.0027235
	ID3	0.227687	1000	0.0248568	0.0024857
3	C4.5	0.224584	1000	0.0256366	0.0025637
	ID3	0.227687	1000	0.0248568	0.0024857

表 5.5 成对样本检验结果表

配对序号	配对算法	均值	标准差	95% 置信区间		t	df	Sig. (双侧)
				下限	上限			
1	CART-C4.5	-.0078483	.0370060	-.0159711	-.0012855	-2.332	999	.022
2	CART-ID3	-.0109611	.0364152	-.0157967	-.0013455	-2.354	999	.021
3	C4.5-ID3	.0003172	.0332895	-.0065482	.0066626	.017	999	.986

从表 5.4—5 中可以看出：

(1) 使用 CART 算法进行随机森林分类时，随机森林算法的 OOB 估计比使用 C4.5 和 ID3 算法分别低出 0.78%和 1.09%，也就是说 CART 算法有利于随机森林算法泛化误差的提升。

(2) 从第 1 对和第 2 对检验的 Sig. (双侧)的值为 0.22, 0.21 可以看出，使用 CART 算法进行随机森林分类，和用 C4.5 和 ID3 算法有很大的差异。

(3) 从第 3 对检验的 Sig. (双侧)的值为 0.986 可以看出，使用 C4.5 和 ID3 算法进行节点分裂对随机森林算法性能的影响差异不大

综合上述三点，可以得出结论：采用不同的决策树节点分裂算法，会对随机森林算法的分类性能产生一定的影响。

5.2 随机森林的分类性能优化--节点分裂混合算法

为了提升随机森林算法的分类性能，从影响算法分类性能的因素看，森林中决策树的数量、随机特征变量 F 的值均基本没有优化的空间，这些可以通过实验找到适当的值，对节点分裂算法的优化在现有的研究中涉及甚少，本文拟在此方面做一些有益的探索。

从第二节的数据和统计检验的结果看，不同的节点分裂算法，在进行节点分裂时，存在一定的差异。对同一个数据集，因为选择的节点分裂算法不同，在节点分裂时选择的属性也会不同，从而使得对于同一个数据集，不同的节点分裂算法会得到一棵不同的决策树。正是由于这个差距，使得使用不同的节点分裂算法，得出的随机森林的分类精度会出现一定的差异，这一点在上一节的分析结论都可以很好地体现。从这个角度看，建立多个节点分裂算法的混合节点分裂算法，可以在同一棵树中，选择最优的属性进行节点分裂，从逻辑上看，应该有提升算法随机森林分类精度的可能性。本节拟将多个节点分裂算法进行线性组合，形成新的节点分裂规则，探寻一种新的提升

随机森林算法分类精度的思路。

5.2.1 节点分裂混合算法描述

由于 CLS 算法和 ID3 算法都是适用范围受限的算法，因此在讨论算法组合时，只考虑它们的升级版本 C4.5 算法和 CART 算法。

1. C4.5 算法与 CART 算法的区别与联系

单棵决策树节点分裂算法比较成熟的算法包括 C4.5 算法和 CART 算法，通过之前的研究发现，CART 与 C4.5 算法都是基于信息论的决策树算法，这为采用两者的线性组合打下了坚实的理论基础。

两者间的区别如下表所示：

表 5.6 CART 与 C4.5 算法比较

算法名称	节点分裂规则	连续性变量处理	处理类别数
CART	Gini 指数最小	二分离散化方法	二个
C4.5	信息增益率最大	二分离散化方法	二个或二个以上

2. 节点分裂混合算法的基本思想

由于 C4.5 算法与 CART 算法都是基于信息论的算法，因此两者的节点分裂原因存在某种程度的近似，因此可以考虑建立两者之间的线性组合，组建节点分裂混合算法。节点分裂混合算法模型如下：

$$\Phi(\alpha) = \beta_1 Gini_{split}(S) - \beta_2 GainRatio(A) \quad (5.1)$$

其中： $0 \leq \beta_i (i=1,2) \leq 1$ 不能二个同时为 0，也不能同时为 1，即边界点上，只能为 (1, 0)、(0, 1) 组合；

$Gini_{split}(S)$ 为 CART 算法分类的计算公式 (2.9)；

$GainRatio(A)$ 为 C4.5 算法分类的计算公式 (2.7)。

在节点分裂时，由于 CART 算法的分裂规则是 GINI 系数最小，C4.5 算法是信息增益率最大，因此两者达到最优时，相减的值最小则为节点分裂的最优规则，因此在节点分裂时时，应该选择 $\Phi(\alpha)$ 值最小的划分进行分裂。

5.2.2 节点分裂算法的程序流程

根据随机森林算法的构建过程，结合节点分裂混合算法的基本思想，使用 JAVA 语言实现了算法的程序设计，部分源代码详见附录 C，这里仅给出算法的程序流程：

Step1 根据初始数据集 DS，使用 bagging 抽样技术生成一个新的数据集，记为 BDS；

Step2 若 BDS 为非平衡数据，则进行使用 C_SMOTE 算法进行非平衡处理，否则，转 Step3；

Step3 若 BDS 中存在连续性属性变量，则使用 COR_CHI2 算法进行连续变量离散化处理，否则，转 Step4；

Step5 计算 BDS 中各个属性的 $\Phi(\alpha)$ 值；

Step6 比较各 $\Phi(\alpha)$ 值，把 $\Phi(\alpha)$ 值最小的属性作为分裂节点，创建节点；

Step7 根据结束条件确定是否结束，如果符合结束条件，则生成叶子结点，结束程序（如递归时应向上跳转），否则转 Step5；

Step8 根据分裂节点的属性，将数据集一分为二，分别记为 leftDS 和 rightDS，并在两个数据集中去除当前属性；

Step9 根据 leftDS 和 rightDS 分别递归调用 Step1- Step5 的步骤，即将 Step1 中的数据集 DS 改为 leftDS 或 rightDS，递归调用后续步骤。

在上述步骤中，第四步的程序结束条件主要包括三个：

1. 当前数据集中的所有样本均属于同一类
2. 当前数据集没有属性可用于划分时，则按照少数服从多数的原则将当前节点强制为叶子节点。
3. 当前数据集目标变量的类别取值中，正类和负类的比例达到某个给定的阈值时，应当根据少数服从多数原则将当前节点强制为叶子节点。这个阈值由用户根据实际应用的需要给定，一般在 0.9-1.0 之间。

5.2.3 混合算法分类性能实证分析

继续使用上述三个数据集，对上述混合算法进行 1000 次实验。由于 Blood 数据集中均存在连续性变量，因此需要使用 COR_CHI2 算法对初始数据集进行数据预处理。每次实验中森林数量为 1000，F 值取为 $\log_2 M + 1$ ，M 是数据集中样本属性个数。取 1000 次实验的 OOB 平均值，其结果如下表所示：

表 5.7 混合随机森林算法在三个数据集上 OOB 估计均值表

序号	β_1	β_2	Blood	Haber	Credit
1	1	0	0.214720	0.243949	0.169573
2	0.9	0.1	0.214369	0.247318	0.172951
3	0.8	0.2	0.214488	0.243288	0.166353
4	0.7	0.3	0.214359	0.24351	0.167094
5	0.6	0.4	0.214730	0.249703	0.169077
6	0.5	0.5	0.214967	0.242688	0.165122
7	0.4	0.6	0.215383	0.241223	0.164551
8	0.3	0.7	0.215795	0.241726	0.167283
9	0.2	0.8	0.216385	0.245289	0.168959
10	0.1	0.9	0.216743	0.250995	0.166944
11	0	1	0.218228	0.264899	0.167295

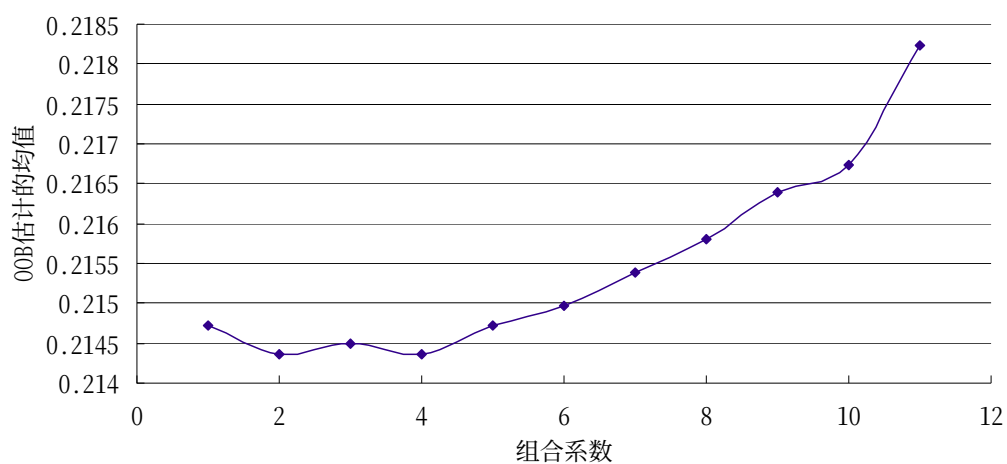


图 5.5 混合随机森林算法在 Blood 数据集上测试结果图

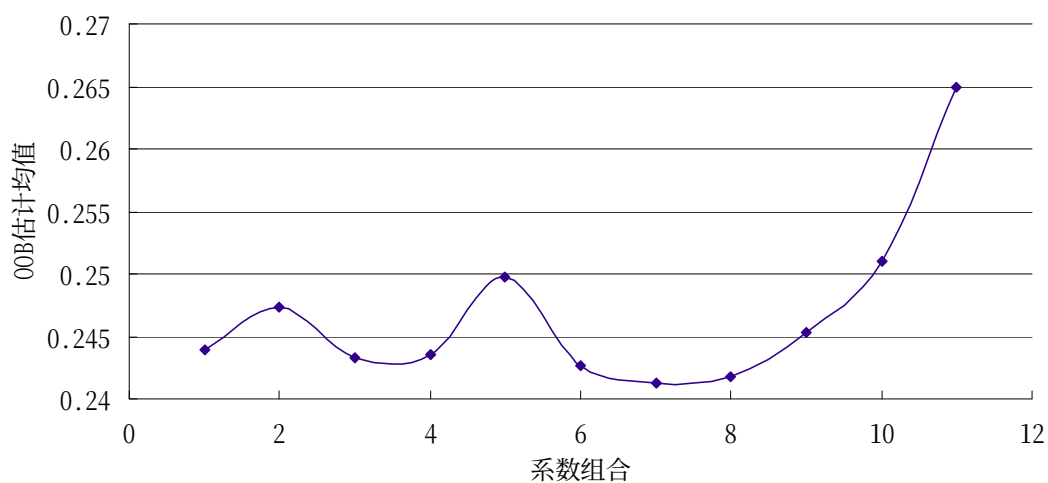


图 5.6 混合随机森林算法在 Haber 数据集上测试结果图

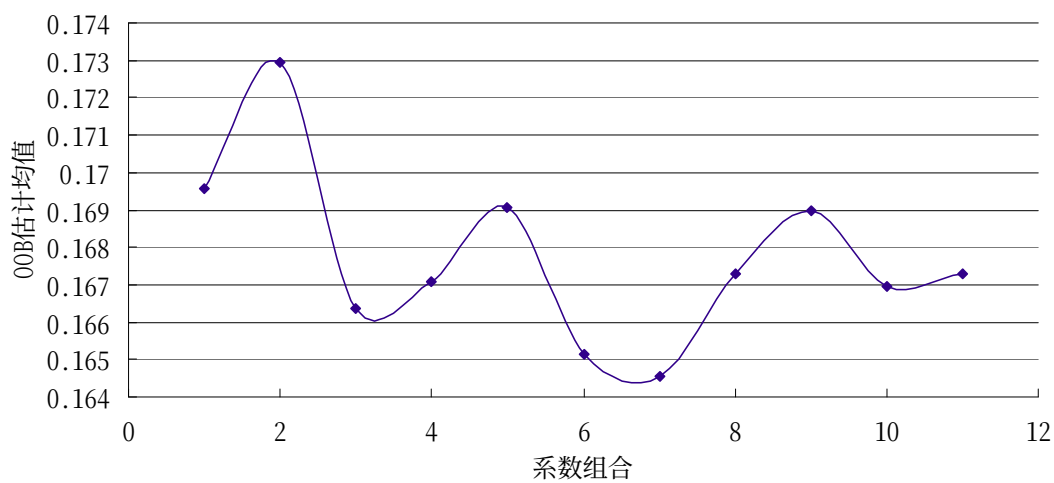


图 5.7 混合随机森林算法在 Credit 数据集上测试结果图

从表 5.7 和图 5.5—7 可以看出：

1. 使用混合随机森林算法后，算法的 OOB 估计值整体下降，说明算法的泛化误差减少，也就是说使用混合随机森林算法提升了分类精度；
2. 在图 5.5 和 5.6 中，C4.5 和 CART 算法性能相差较大，此时混合算法的分类精度变化不，而在图 5.7 中，由于 C4.5 和 CART 算法性能相差不是很大，因此混合算法的提升分类精度的幅度要高一些。
3. 对于不同的数据集，最佳的系数组合是不一定相同的，在 Blood 数据集中，当 β_1 的值为 0.7， β_2 的值为 0.3 时，混合算法的 OOB 估计最小，达到最佳状态；在 Credit 和 Haber 数据集中，当 β_1 的值为 0.4， β_2 的值为 0.6 时，混合算法的 OOB 估计为最小，达到最佳状态。
4. 混合算法的分类精度达到最佳时，均低于单独使用 C4.5 算法或 CART 算法的 OOB 估计，因此可以得出结论，使用混合算可以提升随机森林算法的分类精度，从而使算法可以满足达到某些特定领域的需求。

5.2.4 混合算法与 R 软件中随机森林算法的比较

为了全面考察混合算法的性能，可以将混合算法与 R 软件中随机森林算法性能进行比较，从而更好地证明混合算法的可靠性。为了比较两者的差异，首先要保证两者在基本参数配置上一致，两个算法的数据集均为 Blood 数据集，随机特征变量 F 均取为 3，森林中树木个数从 100 递增到 1000，每次增加 100 个。这样一共有 10 种组合，对每个组合进行 1000 次实验，对每个组合的 1000 次实验的 OOB 估计的均值和标准差进行比较。实验中使用的 R 软件中随机森林算法是 R 语言软件包 randomForest 4.6-6，简记为 RF4.6 算法。实验结果如下：

表 5.8 两种算法性能比较分析

组合	OOB 估计的均值		OOB 估计的标准差	
	混合算法	RF4.6 算法	混合算法	RF4.6 算法
1	0.214447	0.224723	0.0048994	0.002806
2	0.214455	0.222461	0.0045307	0.0019111
3	0.214446	0.221177	0.0042052	0.0016203
4	0.214436	0.220096	0.0039382	0.0013582
5	0.214406	0.21933	0.0038108	0.001275
6	0.214408	0.219058	0.0038283	0.0011797
7	0.21438	0.218516	0.0035903	0.0010309
8	0.214371	0.217952	0.0034168	0.0010173
9	0.214361	0.217568	0.003336	0.000924
10	0.214359	0.217493	0.0032593	0.0009287

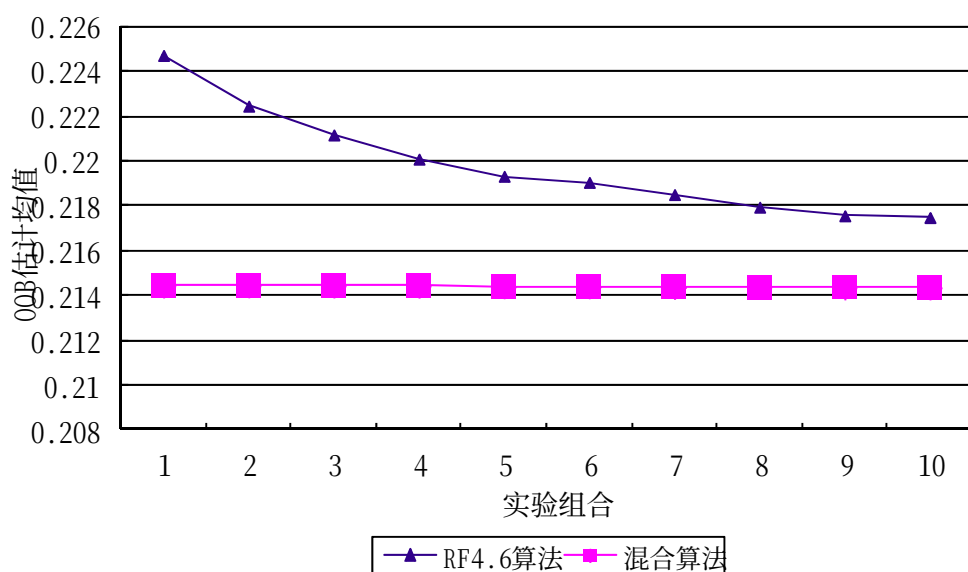


图 5.8 两种算法 OOB 估计的均值对比图

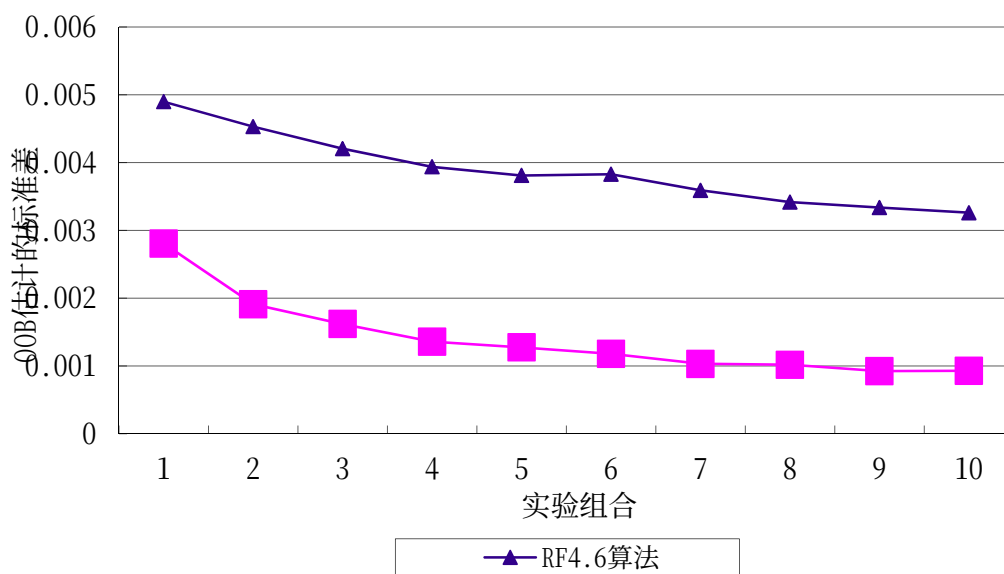


图 5.9 两种算法 OOB 估计的标准差对比图

从表 5.8、图 5.8—9 可以得出如下结论：

1. 森林中的树木个数对算法的性能均存在一定的影响,但混合算法受影响的程度相对较小。
2. 从 OOB 估计的均值看,混合算法始终在 RF4.6 算法的下方,因此在相同的条件下,混合算法具有比 RF4.6 算法更好的分类精度。
- 3.从 OOB 估计的标准差看,混合算法也一直在 RF4.6 算法的下方,说明混合算法的稳定性较 RF4.6 算法更好。
- 4.从图 5.9 还可以看出,随着森林中树木个数的增加,两种算法均表现出相同的

收敛性, 这和本章第二节分析相吻合, 也从另一个侧面表明了本混合算法可靠性。

5.3 混合算法稳定性分析

随机森林算法的稳定性是指在不同的因素水平下, 算法分类精度的差异程度, 差异越小, 则随机森林算法的稳定性越好。从本章第一节分析可以看出, 随机森林算法的分类性能主要受两方面的影响, 一是森林中决策树的数量, 二是随机特征变量 F 的取值。对于某一个特定的数据集, 当采用 Forestes-RI 方式产生最佳的随机特征变量 F 的值时, 该 F 值一般为 $\log_2 M + 1$, 也就是说该值基本是确定的, 因此森林中决策树的数量是影响随机森林算法稳定性的主要因素, 本文主要分析森林中决策树的数量对混合算法稳定性的影响程度, 找出当森林中决策树的数量达到多少时, 继续增加决策树的数量对算法的分类精度基本没有影响, 即混合算法达到稳定状态。为了研究本混合算法的稳定性, 本文通过引入随机森林算法稳定性估计的模型, 然后使用实验数据进行算法稳定性分析。

5.3.1 混合算法稳定性估计的模型分析

由于随机森林算法是一个多分类器, 算法最终的目标是对某个样本点进行分类, 每次分类的结果可以用一个示性函数表示

$$I(X, Y) = \begin{cases} 1 & \text{正确分类} \\ 0 & \text{错误分类} \end{cases} \quad (5.2)$$

其中:

X 为属性变量;

Y 为目标变量。

因此每次分类的过程, 其本质是在进行一次伯努利试验。设 p 为算法对样本点分类正确的概率, 则有 $P(1) = p, P(0) = 1 - p$ 。随机森林要对样本集进行分类, 样本集大小为 n , 则就是进行 n 重伯努利试验。从 X 表示随机森林算法对样本集进行正确分类的总次数, 则 X 是一个服从二项分布 $B(n, p)$ 的随机变量。 X 的概率分布如下:

$$P\{X = k\} = \binom{n}{k} p^k (1-p)^{n-k} \quad (5.3)$$

其中,

$X = k (k = 0, 1, 2, \dots, n)$, 为分类正确的样本数

n 为样本总数

统计意义下算法分类精度 P 应定义为

$$P = \frac{X}{n} \quad (5.4)$$

但在实验的过程中, 只能得到 X 的具体数值 C , 则通常意义下的算法分类精度 $\hat{p}$ 定义为

$$\hat{p} = \frac{C}{n} \quad (5.5)$$

可以证明, $\hat{p}$ 为 P 均值 p 的无偏估计。根据棣莫弗-拉普拉斯定理可知, 当 n 无穷大时, P 服从均值为 $\hat{p}$, 标准差为 $\frac{\hat{p}(1-\hat{p})}{n}$ 的正态分布, 即:

$$P \sim N\left[\hat{p}, \frac{\hat{p}(1-\hat{p})}{n}\right] \quad (5.6)$$

其中 $\hat{p}$ 为算法的分类精度观测值。

评价混合算法是否稳定, 其统计本质是检验多个正态总体 $P_i (i=1, 2, \dots, r)$ 的均值 p_i 是否相同, 即检验在同一因素下不同水平上分类精度的均值是否一致, 这样就可以使用方差分析的 F 统计量进行检验混合算法的稳定性。设某个影响算法稳定的因素水平为 r , 则在不同水平下对算法的分类精度进行测试, 重复测试 m 次, p_{ij} 为在因素水平 i 下, 第 j 次实验的算法的分类精度。进行 r 次实验后, 就可以取得 $r \times m$ 个算法分类精度的观察值 p_{ij} , 即取得 r 组样本数据, 然后通过基于 F 统计量的单因素方差分析, 就可以考察混合算法在某因素影响下的稳定程度。当在某因素的两种水平下, 算法的分类精度的均值是一致的, 也就说明该算法在达到这种水平后, 达到了一种稳定的状态。

5.3.2 混合算法稳定性估计

由于影响随机森林算法的主要因素是森林中树木的个数, 因此在进行检验时, 主要考察森林中树木个数的变化对混合算法分类精度的影响差异。从第四节的实验数据可以看出, 对于不同的数据集最佳系数是不同的, 为了研究方便, 继续考察 Blood 数据集, 因此取 β_1 的值为 0.7, β_2 的值为 0.3, 随机特征变量 F 为 3, 在这些因素保持不变的情况下, 观察对森林中树木的个数对算法稳定性的影响程度, 即进行单因素方差分析, 找出算法最稳定的状态。

从本章第二节的分析可以看出, 随着森林中树木个数的增加, 算法的分类精度越来越趋同, 考虑树木个数从 100 增加到 1000, 以 100 为增加基数, 共有 10 个水平, 观察每个水平两两比较下, 混合算法的稳定程度。使用单因素方差分析混合算法的步骤为:

1. 建立检验假设, 确定检验水准:

H_0 : 十组数据的 OOB 估计的总体均数相等, 即:

$$\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5 = \mu_6 = \mu_7 = \mu_8 = \mu_9 = \mu_{10} \quad (5.7)$$

H_1 : 十组数据的 OOB 估计的总体均数不等或不全相等, 各 μ_i 不等或不全相等

检验水准 α 设为 0.05

2. 在保持其他因素不变的情况下, 在森林中树木个数的 10 个水平下, 分别使用 Blood 数据集对混合算法进行 1000 次训练, 即每个水平独立地进行 1000 次森林的生

成，并取出每个森林的 OOB 估计。然后将十组数据输入到 SPSS 中进行单因素方差分析，统计结果如下：

表 5.9 十组数据在混合算法下 OOB 估计的描述性分析结果表

组别	N	均值	标准误	标准差	95% 置信区间		极小值	极大值
					下限	上限		
100	1000	0.214447	.0028060	.0000887	.214173	.214522	.2062	.2244
200	1000	0.214455	.0019111	.0000604	.214336	.214573	.2092	.2208
300	1000	0.214446	.0016203	.0000512	.214276	.214477	.2091	.2194
400	1000	0.214436	.0013582	.0000430	.214352	.214520	.2103	.2186
500	1000	0.214406	.0012750	.0000403	.214312	.214470	.2101	.2184
600	1000	0.214408	.0011797	.0000373	.214334	.214481	.2104	.2179
700	1000	0.21438	.0010309	.0000326	.214316	.214444	.2109	.2172
800	1000	0.214371	.0010173	.0000322	.214307	.214434	.2114	.2181
900	1000	0.214361	.0009240	.0000292	.214404	.214518	.2111	.2173
1000	1000	0.214359	.0009287	.0000294	.214371	.214487	.2108	.2173
总数	10000	0.214407	0.003882	.0014619	.0000139	.214383	.214437	.2062

表 5.10 十组数据的方差齐性检验表

Levene 统计量	df1	df2	显著性
278.260	10	10989	.000

表 5.11 十组数据的方差分析结果表

	平方和	df	均方	F	显著性
组间	.000	9990	.000	.726	.701
组内	.023	9999	.000		
总数	.024	9990			

(1) 从表 5.9 可以看出，随着森林中树木个数的增加，算法的 OOB 估计逐步减少，也就是说混合算法的分类精度逐步增加。

(2) 从表 5.10 的方差齐性检验可以看出，显著性很小，说明十组实验数据可以进行方差分析。

(3) 从表 5.11 的方差分析表可以看出，十组数据的均值差异的显著性为 0.701，

大于 α (0.05), 说明在置信水平 0.95 下不能否定零假设, 即拒绝 H_0 , 说明十组数据的均值存在显著的差异, 当然这种差异是某些组之间的差异, 不排除组与组之间没有显著差异的可能。也就是说使用混合算法生成的随机森林中树木的个数对算法的分类精度有显著差异, 即森林中树木的个数对混合算法的分类精度有重要的影响, 这和第一节使用 R 软件中的随机森林算法得出的结论是一样的。

综上所述, 随着随机森林中树木的数量变化, 混合算法的分类精度存在显著的差异, 也就是说当森林中树木个数从 100 增加到 1000 时, 从整个过程看, 混合算法的分类精度是递增的, 也就是说混合算法是不稳定的。由于随机森林算法具有收敛性, 因此森林中树木个数达到一定数量后, 算法可能会达到一种稳定状态, 这是后面章节需要进一步分析的内容。

5.3.3 混合算法稳定状态分析

从上面的分析可以看出, 森林中树木的个数对算法性能有显著的差异, 也就是说这个因素是影响混合算法稳定性的一个重要因素。由于森林中树木个数是由用户设定的, 那么设定一个什么样的数值才能使算法达到一个相对的稳定状态, 是一个值得研究的问题。

根据表 5.5 的数据, 绘制统计图, 分析混合算法的 OOB 估计的均值和标准差变化情况。统计图如下:

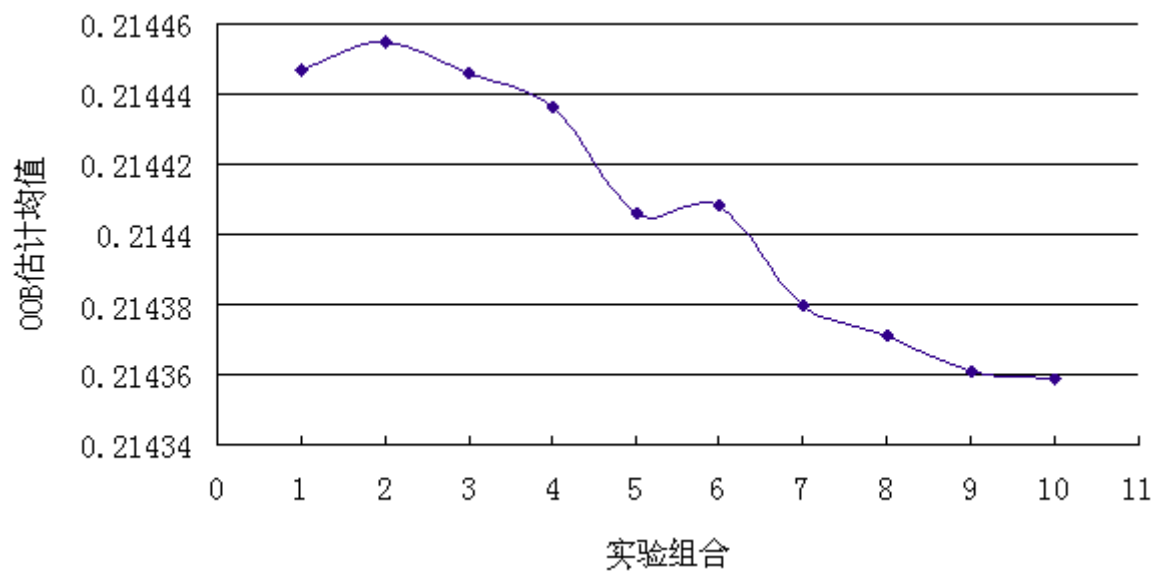


图 5.10 混合算法的 OOB 估计的均值变化趋势

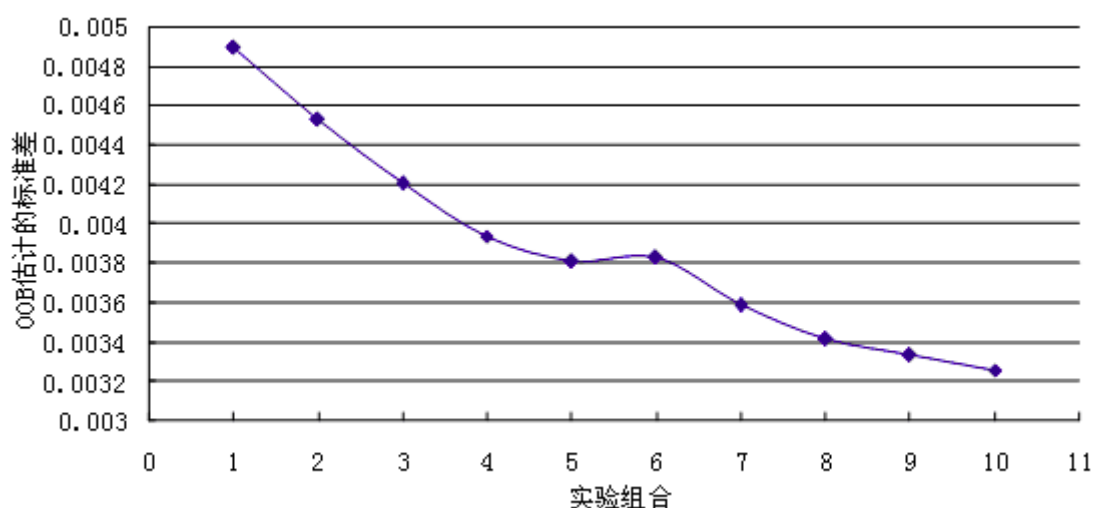


图 5.11 混合算法的 OOB 估计的标准差变化趋势

从图 5.10 可以看出算法性能随着森林中树木的个数增加而逐渐提升，但到 900 和 1000 后，趋向于稳定。从图 5.11 可以看出，混合算法的内在稳定性在逐步加大，这和第二节使用 R 软件中的算法进行实验得出的结论基本一致。那么，算法在什么状态下趋向于稳定，也就是森林中决策树的个数达到多少时，算法的分类精度基本不会发生太大的变化，可以通过分析十组数据的 OOB 估计值的均值变化情况，即可以对十组数据进行两两单因素方差分析，统计数据如下：

表 5.12 对五组数据进行两两方差分析结果

配对组别	平方和	df	均方	F	显著性
100-200	.000	1	.000	.997	.318
200-300	.000	1	.000	.983	.321
300-400	.000	1	.000	.805	.370
400-500	.000	1	.000	.585	.444
500-600	.000	1	.000	.091	.763
600-700	.000	1	.000	.307	.580
700-800	.000	1	.000	.043	.835
800-900	.000	1	.000	4.327	.038
900-1000	.000	1	.000	4.235	.040

从表 5.12 可以看出：

1. 混合算法生成的森林中，决策树的数量为小于 800 时，OOB 估计的均值都存在显著性差异，也就是说在这些水平中，混合算法的稳定性是比较差的。
2. 在决策树的数量为 800、900 和 1000 时，显著性分别为 0.038 和 0.040，均小于 α (0.05)，说明在置信水平 0.95 下应否定零假设。也就是说混合算法生成的

随机森林，在森林中树木为 800、900 或 1000 时，OOB 估计的均值没有显著差异。

3. 结合图 5.10 和表 5.12 看，可以得出结论：由混合算法产生的森林，算法的性能会随着森林中树木的个数增加而逐渐提升，在树木个数较小时，算法的性能不是很稳定，但达到一定的数量后，算法性能基本趋向于稳定。对于 Blood 数据集而言，森林中决策树的数量达到 800 时，该混合算法达到一种稳定状态，此时继续增加决策树的个数不会使算法的分类精度有太大的变化。

5.4 混合算法形成的森林中相关度及强度分析

5.4.1 随机森林中相关度及强度估计

将二分类随机森林算法应用到公式 (2.16) 中，则混合算法形成的森林的强度估计可简化为如下公式：

$$s = \frac{1}{n} \sum_{i=1}^n \left(Q(x_i, y) - Q(x_i, j) \right) \quad (5.6)$$

其中

$Q(x_i, y)$ 为在 OOB 数据集 $O_k(x)$ 上，对应决策树的正确分类的频率，即给定一个样本，森林中对其正确分类的决策树的个数和总的决策树的个数的比例。

$Q(x_i, j)$ 为在 OOB 数据集 $O_k(x)$ 上，对应决策树的错误分类的频率

由于 $Q(x_i, j) = 1 - Q(x_i, y)$

则有：

$$s = \frac{1}{n} \sum_{i=1}^n (2Q(x_i, y) - 1) \quad (5.9)$$

将二分类随机森林算法应用到公式 (2.17) 中，则混合算法形成的森林的平均相关度可以简化为：

$$\bar{\rho} = \frac{\text{var}(mr)}{sd(h(*))^2} = \frac{\frac{1}{n} \sum_{i=1}^n (2Q(x_i, y) - 1)^2 - s^2}{\left(\frac{1}{k} \sum_{u=1}^k \sqrt{1 + (2p_u - 1)^2} \right)^2} \quad (5.10)$$

其中 p_u 为每棵新决策树对 OOB 数据集 $O_k(x)$ 上所有样本分类正确的频率，即给定一个数据集，每棵决策树对数据集中的样本正确分类与总样本的比例。

5.4.2 混合算法形成的森林中强度分析

根据公式 (5.9) 可以计算随机森林的强度，下面通过实证的办法分析混合算法形成的森林的强度对混合算法的分类精度的影响程度。继续选用 Blood 数据集，随机特征变量 F 均取为 3，森林中树木个数为 1000，对混合森林中 11 种系数组合进行 1000

次实验，观察混合算法在不同系数组合下的强度和 OOB 估计的平均值，数据如下：

表 5.13 混合算法在不同系数组合下的强度和 OOB 估计均值对照表

序号	β_1	β_2	强度	OOB 估计
1	1	0	0.601603093	0.214720464
2	0.9	0.1	0.606684804	0.214369959
3	0.8	0.2	0.598675631	0.214488501
4	0.7	0.3	0.609844597	0.214359326
5	0.6	0.4	0.606570765	0.214730129
6	0.5	0.5	0.607102841	0.214967455
7	0.4	0.6	0.602397276	0.215383309
8	0.3	0.7	0.598657283	0.215795346
9	0.2	0.8	0.607857742	0.216385135
10	0.1	0.9	0.604072392	0.216743832
11	0	1	0.557025893	0.218228957

对上述数据在 SPSS 中进行 Pearson 相关性检验，检验结果如下：

表 5.14 强度和 OOB 估计的 Pearson 相关性检验结果表

		强度	OOB 估计
强度	Pearson 相关性	1	-.740**
	显著性（双侧）		.009
	N	11	11
OOB 估计	Pearson 相关性	-.740**	1
	显著性（双侧）	.009	
	N	11	11

**：在 .01 水平（双侧）上显著相关。

从表 5.14 可以看出，Pearson 检验的显著性为 0.009，小于 0.01，则说明混合算法形成的森林的平均强度和 OOB 估计显著相关，且呈负相关关系。也就是说，随着森林中树木的强度提升，算法的 OOB 估计会下降，而算法的分类精度也随之上升，这和第二章中随机森林算法性质的理论分析基本一致。在系数为 0.7 和 0.3 时，森林中的强度达到最佳状态，而此时算法的分类精度也表现最好，这也从另一个方面说明了该混合算法具有一定的优越性。

5.4.3 混合算法形成的森林中相关度分析

根据公式 (5.10) 可以计算随机森林的相关度，下面通过实证的办法分析混合算法形成的森林的相关度对混合算法的分类精度的影响程度。继续选用 Blood 数据集，随机特征变量 F 均取为 3，森林中树木个数为 1000，对混合森林中 11 种系数组合进

行 1000 次实验, 观察混合算法在不同系数组合下的相关度和 OOB 估计的平均值, 数据如下:

表 5.15 混合算法在不同系数组合下的相关度和 OOB 估计均值对照表

序号	β_1	β_2	相关度	OOB 估计
1	1	0	0.293534	0.214720464
2	0.9	0.1	0.301788	0.214369959
3	0.8	0.2	0.308867	0.214488501
4	0.7	0.3	0.305265	0.214359326
5	0.6	0.4	0.299158	0.214730129
6	0.5	0.5	0.298538	0.214967455
7	0.4	0.6	0.301858	0.215383309
8	0.3	0.7	0.303691	0.215795346
9	0.2	0.8	0.295115	0.216385135
10	0.1	0.9	0.298056	0.216743832
11	0	1	0.514368	0.218228957

对上述数据在 SPSS 中进行 Pearson 相关性检验, 检验结果如下:

表 5.16 相关度和 OOB 估计的 Pearson 相关性检验结果表

		相关度	OOB 估计
强度	Pearson 相关性	1	.728*
	显著性 (双侧)		.011
	N	11	11
OOB 估计	Pearson 相关性	.728*	1
	显著性 (双侧)	.011	
	N	11	11

在 0.05 水平 (双侧) 上显著相关。

从表 5.16 可以看出, Pearson 检验的显著性为 0.013, 小于 0.05, 则说明混合算法形成的森林的平均相关度和 OOB 估计显著相关, 且呈正相关关系。也就是说, 随着森林中树木相关度的提升, 算法的 OOB 估计也会上升, 而算法的分类精度也随之下降, 这和第二章中随机森林算法性质的理论分析基本一致。在系数为 0.7 和 0.3 时, 森林中的相关度最小, 而此时算法的分类精度也表现最好, 这和前面的分析基本一致。

5.5 本章小结

本章通过详细分析影响随机森林分类性能主要因素, 针对不同的节点分裂算法对

随机森林分类性能有不同的影响效果的实际情况, 提出一个将多个节点分裂算法组成线性函数的随机森林混合算法, 通过修改各分裂函数在线性函数中的系数值, 观察在不同的系数组合的情况下, 随机森林分类性能的表现。实验数据显示, 在某些系数组合的情况下, 混合节点分裂算法的分类性能比单一节点分裂算法的分类性能要更好一些。

本文提出的混合随机森林算法, 可以提高算法的分类精度, 在某些对分类精度要求较严格的领域可以使算法得到应用。由于 CART 算法只能生成二叉树, 因此本混合算法生成的决策树也只能是二向分叉, 这使得算法在应用时或许存在的问题, 需要在今后的研究中进一步拓展, 以期取得更好的成果。

6 优化后的随机森林算法对优质股票池的选择

在对随机森林算法完成优化后,可以将该优化后的算法应用到具体的应用中,在现有的文献中,随机森林已经应用在很多领域,但在量化投资领域,由于该领域对算法的精度要求严格和连续变量较多等实际情况的存在,使得随机森林算法在该领域的应用涉及较少,本文拟将优化后的随机森林算法应用到量化投资领域,解决股票投资中选择优质股票的实际问题。

股票投资一般分为三个环节,首先是根据宏观环境选取某个行业,其次是选取行业中较好的股票,即产生优质股票池,第三是在优质股票池中选择几只股票进行投资组合。选择优质股票是股票投资中的关键环节,是进行投资组合(Portfolio)的重要前提。所谓的优质股票是指那些具有优异的成长性、丰厚的投资回报以及良好的抗风险能力的股票,这些股票的价格在市场中表现为:在熊市中,其下跌幅度低于行业平均水平;在牛市时,其上涨超过行业平均水平。所有的投资者和机构都希望投资于优质股票。然而,由于股票价格受多种因素的影响,表现出“没有规律”的随机游动特性,选择优质股票成为可望而不可及的事情。

从技术上看,股票的选取可以归结影响股票价格的多因素分析问题,其中每一个因素可以看作一个维度的指标,这样股票的价格就由多个维度的指标体系决定,从众多的股票中选取优质的股票,就归结为大量信息的统计分析,其本质是一个多维空间的分类问题。

该分类问题涉及到两个方面,一是影响股票价格的维度选择,即指标体系的选择,另一个是分类算法的选择,即选择合适的分类算法。本章拟以价值成长投资策略(GARP)为基础,采用此策略下的选股模型指标体系,使用优化后的随机森林算法,构建量化选股模型,实现从大量的股票中,选择一部分优质股票,形成优质股票池,从而有利于投资人进行有针对性的投资组合。

6.1 量化投资中选股研究综述

由于选股问题其本质是分类问题,因此人们在选择数量化工具进行股票选择时,纷纷选择数据挖掘工具中的分类算法进行股票选择。

6.1.1 朴素贝叶斯分类方法在选股分析中的应用

钱颖能等人以分析上海证券交易所股票的年度财务报告为出发点,建立贝叶斯分类选股模型,该模型选择股票的时间跨度为2002年至2004年,通过该模型选出了一批优质股,通过数据显示,该批优质股相对于市场指数能够获取显著的超额收益。

左辉等人以证券分析师所推荐股票各方面总体特征为出发点,选择2007年1月到2007年10月间推荐人次为5次及以上的股票,通过贝叶斯分类方法选出具有这

类特征的股票，进而根据这些股票的特征去判断股票的优劣。

6.1.2 决策树分类方法在选股分析中的应用

在 Eric H,Keith L,Chee K 在论文中，提出了 EPS-Price、Price-MOM 等指标构建了决策树模型，对美国科技股 1993 至 1999 年的数据进行分析，得出比较好的收益。使用他们提出的决策树方法并参考其指标，对我国国内科技股板块从 2012 年到 2013 年的数据进行检验，结果表明模型在效果并不显著，原因是模型中这些指标主要为美国投资经理的经验选择，在中国市场上应用还需要一个本土化过程。

赵永进等(2005)^[87]从股票分析的基本面和技术面着手，把判定树分类 ID3 算法应用到股票财务数据的分析上，选取有代表性的财务指标，并对样本数据进行测试，开创数据挖掘分类算法在量化选股模型在中国股票市场的先河，投资者利用测试结果可以对上市公司的经营情况和获利能力进行分析。但该文章仅仅是提供了一种分析手段，投资者不能通过模型得到诸如可否继续持有股票等指导性意见，没有达到量化选股的最终要求。

国信证券工程师焦健等(2010)^[86]针对 Eric H,Keith L,Chee K(2000)等人提出了 EPS-Price、Price-MOM 等六项指标进行定义调整，提出了基于市销率、市现率，ROA 变化率等适合中国股票市场的六项指标，并在此指标的基础上，使用 CART 决策树算法，对科技板块 152 只股票数据进行训练和测试，通过五分法转换和剪枝操作，得出模型预测效果显著，多头明显跑赢行业平均利润率。由于决策树算法在处理噪音数据效果较差，且容易过拟合等特点，使得模型的应用行业有一定的局限性。

6.1.3 神经网络分类方法在选股分析中的应用

Quah^[102]比较多层感知器、自适应神经模糊推理以及径向基神经网络等三种神经网络方法在股票选择中的应用，都取得了较好的结果。

Kim 和 Han^[103]利用遗传算法确定人工神经网络的连接权重，并对离散化后的数据进行建模预测股票价格，实验表明，该方法能降低参数空间的维数，进而提高预测性能。

张建军等^[105](2010)采用科威特第纳尔指数，通过建立一个三层（输入层、隐含层和输出层）结构的 BP 神经网络，对股市整体走势进行预测，成功地利用数据挖掘中神经网络技术对我国股票行情波动趋势进行研究，预测未来股市的行情走势及其波动情况，但其模型对个股的走势预测效果欠佳。

此外，还有众多学者把人工神经网络技术应用的股票价格预测领域，都表现出出色的性能^{[104][105][107][108]}。

6.1.4 支持向量机分类方法在选股分析中的应用

李云飞(2008)^[90]在其论文中，提出基于支持向量机的股票选择模型，模型采用最小二乘法，核函数选用径向基核函数。在模型训练完成后，使用交叉验证的实证方法，实证结果支持了支持向量机模型的分类精度和泛化能力均优于传统神经网络的理

论推断,但由于支持向量机算法在多分类选择时,性能显著低于随机森林算法,因此模型还具有提升的空间。

Wei Huang, Yoshiteru Nakamori, Shou yang Wang(2005)给出来预测股市周运动方向的支持向量机方法,并用日经指数和 BP 神经网络方法做了对比,得出了支持向量机方法更好的结论^[91]。

Francis 和 Lijuan Cao(2001)用支持向量机预测金融时间序列,利用芝加哥的期货数据实证得出支持向量机优于多层 BP 神经网络方法。

6.1.5 使用优化后的随机森林算法进行选股分析的原因

综上所述,选择股票其本质是分类问题,人们已经使用了数据挖掘中经典的分类算法进行选股分析,但将随机森林算法应用到此领域在国内文献中见之甚少,而随机森林算法在其他领域的应用都得到了比较好的效果,因此本文拟在此方面做积极的探索,也属于该领域的创新。另外,使用优化后的随机森林算法进行优质股票池的选择还有以下几点考虑。

一是随机森林算法的抗干扰性和股票数据的特点相适应。从前文对随机森林算法的介绍可以看出,随机森林算法因其具有训练集选取随机和属性选择随机两个随机性的特点,使得算法具有很好的容错性和鲁棒性,这和当前股票市场异常情况和干扰项较多的情况相适应。

二是优化后的随机森林算法可以较好地处理连续性属性数据。在股票投资领域,涉及到了大量的连续性数据,如何才能更好地处理这些连续性数据,使得算法的性能更好,也是该领域的一个难题,本文的优化后的随机森林算法可以较好地解决这个问题。

三是优化后的随机森林算法具有较好的分类精度。从研究综述上可以看出,大部分的选择策略都集中在单分类器分析上,很少使用多分类器进行研究,从性能上讲,多分类器在一定程度上较单分类器具有更显著的分类精度,这一点在文献^[47]中已经得到了实验数据的支持。而且优化后的随机森林算法具有很好的预测准确率和稳定性,虽然混合算法在算法分类精度方面提升幅度不大,但在对分类精度的要求比较高的量化投资领域,分类精度的小幅提升却可以带来投资决策的巨大成功。

基于以上几点情况,本文选择随机森林算法作为选股模型的量化工具,相信该优化算法应用到股票投资将会有不错的表现。

6.2 股票数据预处理方法简介

由于股票数据的特殊性,在进行模型选股时,需要对数据进行预处理,本节主要介绍一下需要用到的数据预处理方法。

6.2.1 数据标准化处理

在进行股票数据分析时,有些数据需要进行标准化,这样有利于数据在同一个标

准上进行比较和分析。在统计分析时,数据标准化的方法是很多的,最常用的方法包括三种,一是 min—max 标准化;二是 z-score 标准化;三是按小数定标标准化。

1.min-max 标准化

该标准化方法首先要找出数据序列的最大值(记为 maxA)和最小值(记为 minA)然后将原始数据 x 通过标准化公式转换为 x' , 该标准化公式是:

$$x' = \frac{x - \min A}{\max A - \min A} \quad (6.1)$$

从这个公式可以看出, x' 的取值在 $[0, 1]$, 它是通过原始数据 x 的线性变换而得出来的, 这样转换后的数据就可以在一个相同的区间中进行比较。

2.z-score 标准化

该标准化方法先计算出数据序列的均值(mean)和标准差(sd), 然后将原始数据 x 使用 z-score 标准化公式转换为 x' , 其标准化公式是:

$$x' = \frac{x - \text{mean}}{sd} \quad (6.2)$$

这种标准化的方法用得比较普遍, 也具有较好的统计属性。

3.小数定标标准化

该方法通过对数据统一去除个位数来达到标准化的目的, 其计算公式如下:

$$x' = \frac{x}{10 \times j} \quad (6.3)$$

从这个公式可以看出, 该方法是通过给定一个最小整数, 让原始数据都除去这个最小整数和 10 的乘积, 这样做最后的结果是所有的数都缩小了一个固定的比例, 从而使数据回到相同的量纲上来进行比较。该方法在确定 j 值是存在一定的不确定因素, 因此很少被使用。

6.2.2 数据噪声处理

在时间序列分析中, 去除数据噪声有很多方法, 主要包括滑动平均法、傅立叶变换、卡尔曼滤波方法、小波分析和趋势滤波等方法。

1. 滑动平均法

滑动平均法又叫移动平均法。它是一种最简单平滑去噪技术, 它的核心思想是: 时间序列的噪声一般表现为随机波动的形式, 而去噪后的数据应是时间序列稳定的趋势数据, 因此根据时间序列资料、逐项推移, 依次计算包含一定项数的序时平均值, 用这个平均值的序列反应原序列的长期趋势, 这个长期趋势就是去除噪声后的数据本身的一种变化发展规律。

移动平均法得到去噪后数据的公式如下:

$$F_{t+1} = \frac{1}{n} \sum_{i=t-n+1}^t x_i \quad (6.4)$$

其中: n 是移动时间的长度,即参与计算移动平均值时所使用的历史数据的数目; F_{t+1} 是 $t+1$ 时刻根据前 n 个历史数据平均后的预期数据。

2.傅立叶变换,是一种传统的去噪方法,该方法去噪基本假定是,在不同频率的信号中低频信号是有用的信号,而高频信号则为噪声。傅立叶变换去噪的基本原理是:根据傅立叶转换函数,将时间序列的数据转换成多个频率信号的组合,然后将某个频率水平以上的傅立叶系数全部设为零,然后对余下的频率信号进行傅里叶逆变换,再将这些信号转换成去除噪声的时间序列数据,从而实现数据去噪的目的。

(1) 设原始信号为 $f(t)$, 其傅里叶变换公式为:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{i\omega t} dt \quad (6.5)$$

(2) 傅里叶逆变换公式:

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{-i\omega t} d\omega \quad (6.6)$$

$F(\omega)$ 确定了 $f(t)$ 在整个时间域上的频谱特性。

傅里叶变换在去噪时也有一个难以克服的问题,就是要求处理的信号必须是平稳的。这主要是傅里叶变换是把信号完全转换成频域进行分析,这样做会产生两个问题,一是为了计算某一点频率的频谱需要计算过去和未来的所有时间信号,二是完全丢失了信号的时域信息。因此傅里叶变换在处理股票数据这种非平稳信号时,效果就不是很好。

3.小波去噪

小波(wavelet)是一种小区域的波,它有二个重要的特点,一是长度短,二是叠加后其平均值是零。小波分析的主要思想是通过引入一系列的小波函数,将原有的函数进行分解,也就表现为信号的一种叠加。运用小波变换进行数据噪声消除是小波分析的一个非常重要的应用之一。

小波去噪方法包括三个基本的步骤:一是对含噪声信号进行小波变换,即对信号进行分解;二是对变换得到的小波系数进行某种处理,以去除其中包含的噪声;三是对处理后的小波系数进行小波逆变换,得到去噪后的信号。

(1) 小波分解

小波分解是指将一个复杂时频信号用一系列不同频率、不同延迟的简单标准信号相叠加的形式表现出来。通过一层层的分解,可以将原始信号分解成许多不同频率的频率成分,形成如图 6.1 所示的小波树。

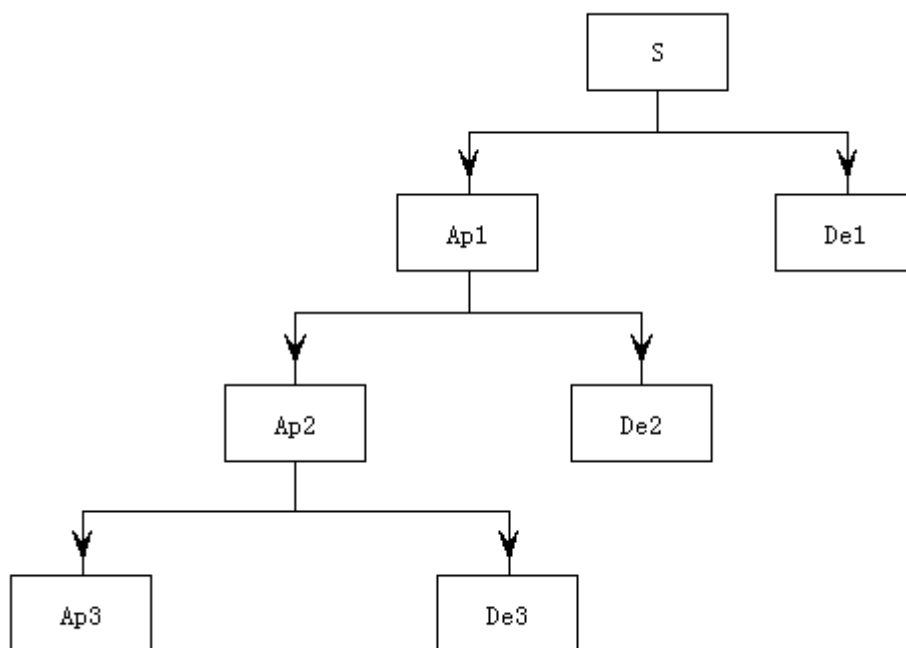


图 6.1 小波信号分解树示意图

图 6.1 中 S 表示原始信号, Ap 表示信号的低频成分, 或称逼近信号, 而 D 表示信号的高频成分或称细节信号, 数字则表示分解的层数。可以看到原始信号首先分解成一个低频成分 $Ap1$ 和一个高频成分 $De1$, 而 $Ap1$ 又可以继续分解成一个低频成分 $Ap2$ 和一个高频成分 $De2$, 接着低频成分可以继续分解下去。小波分析的理论认为信号的低频成分是最重要的, 它蕴含着信号最主要的特征, 而高频成分则给出信号的细微差别。例如, 对人说话的声音进行去除噪声处理时, 如果去除声音数据中的高频部分, 只保留低频部分, 那么就会分辨不出是谁在说话, 但是仍然能听懂所说的内容; 而如果去除低频部分只保留高频成分的话, 则只会听到一些无意义的声音。

小波分解是小波分析的最重要的一步, 信息经过分解后组成了很多的小的部分, 这为去除噪声提供了可能。小波变换的基本思想是用 2 个或 2 个以上的函数(小波基)去逼近原函数。

(2) 去除噪声

在使用小波变换将信号进行分解后, 接着就是使用某种规则, 去除一部分符合条件的信号, 被去除的部分就是数据中的噪声。使用的规则不同, 去噪的方法也不同。一是强制消噪处理该方法, 该方法利用 DWT 函数把小波分解结构中的高频部分全变成零, 即把高频部分全部消除, 再对信号进行重构。此方法简单, 消噪后信号也比较平滑。二是默认阈值消噪处理, 利用 `ddencmp` 函数产生信号默认阈值, 然后利用 `wden` 函数进行消噪处理。三是给定软或硬阈值消噪处理在实际消噪处理过程中, 阈值可通过经验公式获得, 而且这种阈值比默认阈值更具有可信度。

(3) 小波重构

重构的原理与分解相反,是将最大尺度的低频成分与各层高频成分进行组合,从而重构出原始信号,也就是将 Ap_3 , De_3 , De_2 和 De_1 进行叠加而得到 S , 该过程是分解过程的逆过程。通过小波的分解和重构两个过程,可以方便地将股票数据进行拆分和还原,从而达到提取重要信息,提高数据分析质量的目的。

4. 趋势滤波

一个含噪声的一维数据的模型可表示成如下的形式:

$$s(i) = f(i) + \lambda e(i), i = 0, \dots, n-1 \quad (6.7)$$

其中: $f(i)$ 为真实数据;

$e(i)$ 为噪声;

$s(i)$ 为含噪声的数据。

真实数据一般是一些比较平稳的数据,而噪声 $e(i)$ 数据通常表现为高频数据。去除噪声的本质思想是通过某种函数找到真实数据 $f(i)$ 。

趋势滤波方法去噪时,假设 $f(i)$ 为趋势项, $e(i)$ 为波动项,算法的目的就是估计一个尽量光滑的 $f(i)$, 同时保证 $e(i)$ 尽量较小。文献中有很多趋势滤波的算法,例如 Hodrick-Prescott(H-P) 滤波^[96]、L1 趋势滤波^[97]、移动平均滤波^[98]、指数滤波^[99]、光滑样条^[100]等。在这些方法中, H-P 滤波因其简洁的表达和计算,广泛地应用在很多领域。

Hodrick 和 Prescott(1997)^[96]提出了 H-P 滤波,即求得一个序列 x_t 最小化(6.7)式的目标函数,该序列就是真实数据。目标函数由两部分组成,一部分是波动项 z_t 的平方和,另一部分是趋势项 x_t 二次差分的平方和,规则化参数 λ 控制了两者的权重。目标函数的公式如下:

$$\min_x \left\{ \sum_{t=1}^n (y_t - x_t)^2 + \lambda \sum_{t=2}^{n-1} (x_{t-1} - 2x_t + x_{t+1})^2 \right\} \quad (6.8)$$

6.3 选股模型指标体系的简介

确定选股模型的指标体系,首先需要解决股票投资领域一个争论不止的问题——使用什么样的投资策略,因为不同的投资策略使用的指标体系是不一样的。

6.3.1 传统的价值型投资策略

1. 传统的价值型投资策略的思想

该策略倡导一种理性的投资思路和选股标准——价值投资,其思想是通过对股票基本面的分析,寻找具有更好价值因子的公司,且交易价格低于内在价值的公司,以期获得超过市场平均收益额,价值因子包括市盈率,市净率,市销率。其代表理论有格雷厄姆(Benjamin Graham)的价值投资理论。

价值投资理论认为,任何一种金融工具都是有一个内在的稳定的投资基础,这个

基础就是金融工具自身的价值，股票也一样，股票的投资基础就是其内在的价值。进行股票价值投资时，应用某种指标去衡量这个内在的价值，当取得这个内在价值的量化指标后，就可以在股票市场价值低于或高于内在价值时，进行股票的投资，即进行股票交易。

价值投资理论最重要的两个概念：内在价值与安全边际。内在价值的概念由来已久，从股票开始产生它就产生了，它就是多方面的因素决定的股票的价值，这些因素包括公司的投资收益、净资产、经营管理效率和市场对企业的未来预期等因素，是上市公司预期的现金流的折现值。内在价值由于受多方面的影响，因此它在一定时间跨度内是稳态的，而股票的价格在市场上表现是变化不定的，股票的内在价值和市场价格之间的差距就是“安全边际”。在价值投资理论中，安全边际是其核心观点，一般来说安全边际越大越好，也就是说股票的市场价格越偏离其内在的价值，则这支股票越应该投资。

价值投资的核心过程：先评估某一股票的内在价值，并将其与股票当前交易价格相比较。如果股票当前交易价格低于股票内在价值，并能获得足够的安全边际，就买入该证券，进而投资获利。可见，价值投资的核心是强调安全边际。

2. 传统的价值投资的理论缺陷

(1)传统价值投资理论过于强调股票的内在价值，而忽视股票的成长性，这会在投资中错过投资机会。传统价值投资通过市盈率或市净率等指标来衡量股票的交易时机，而这两个指标在成长型股票是很难体现出来，因此这使得价值投资者在进行投资时，经常会避开正在成长中的股票，因为这些股票的市盈率或市净率等指标在数据是低估的。

(2)传统价值投资理论主张集中投资，反对分散投资，提升了投资者的投资风险。价值投资理论主张集中资金投资一个存在安全边际最大的股票，因此会增加投资的风险，一旦观测指标出现偏差，或者出现虚假的投资信息，在这种理论指导下的投资风险就较大。

(3)传统价值投资理论的投资决策可操作性较差。传统的价值投资理论是建立在稳固基础理论的基础上的，特别关注股票的内在价值，强调安全边际。其对股票内在价值的判断，通常遵循威廉姆斯“股票内在价值等于公司证券持有者未来年份得到的分红和利息的现值”的投资理论，通过对预期的股票投资回报率的贴现值来衡量内在价值，该方法缺乏可操作性，换言之，传统的价值投资理论关于“安全边际”的判断方法缺乏可操作性使得依据传统的价值投资理论的投资决策也缺乏可操作性。

(4)传统价值投资理论在中国股票市场的适用性存在的问题。传统的价值投资理论认为价值投资的盈利方式是长期持有一只安全边际较高的股票，通过股票的红利稳定地获取收益。而在现今的中国股市派发红利的习惯还没有养成，一些公司很少分红，一些公司只是把分红当作一种任务，更不要提稳定的股利政策。在投资人的眼

中，获得短期的收益比长期收益更具有可行性。另外历年的基金年报普遍显示，现今的股市盈利方式主要是依靠股票买卖差价来实现。按照传统的价值投资的观点，我国股市并不具备价值投资的长远市场环境。

6.3.2 积极的成长型投资策略

1. 成长型投资策略的思想

投资中，选择成长型投资策略的投资人一般都购买那些具有成长性的股票，这些股票一般表现为销售收入高于市场水平，而且企业的盈余率也比较高，这些股票还有一个明显的特点就是企业正处于快速发展阶段。这些投资人是想通过企业的快速成长，从而使其股票市场价格也能较快地涨上去，这样就可以通过股票的差价得到可观的利润。成长因子如净利润增长率，ROE 增长率，ROA 增长率等。

成长型投资者关注于市盈率的分母部分，也就是每股收益及与其相关的经济因素。这些投资者努力寻找那些每股收益将会快速增长的公司，并且假定市盈率在未来一段时间内会保持不变，这个假设就意味着股价会像每股收益一样以相同的速度上涨。成长型投资者关注公司当前和未来的经营和利润增长前景，尤其偏好各种看起来很美的概念和故事，但并不过多在意当前的估值水平。这一点正好和价值型投资策略相反。

2. 成长型投资策略的主要问题

成长型投资策略只关心股票的预期成长性，而不关注其现在的价格，但人们在对未来的预期一般都充满更多的想象，因此一般的预期都很不是准确的。而且由于中国股票市场的不确定性太多，很多的政策性因素会极大地影响股票价格的走势，因此从整体上看，成长型投资策略在实施的过程中存在偏差的可能性要高于价值成长策略。

6.3.3 价值成长投资策略（GARP）

就在人们为选择价值型还是成长型投资策略争论不休时，投资大师彼得林奇（Peter Lynch）将格雷厄姆（Benjamin Graham）的价值投资策略和成长型投资策略彻底融合，他利用该方法在 1977 至 1990 的十三年间创造了年平均收益率高达 29% 的传奇业绩，实现基金投资业绩同业排名第一。该策略是上述两个理论的综合，其思想是挑选出从一定程度上价值被低估，同时又具有比较好的持续增长能力的潜力股。其特点是能够充分利用两者的优势，弥补单一价值投资和成长投资的不足。以获取稳健的投资收益。

GARP 策略主要考查上市公司的两个重要的指标，一是市净率，二是资产收益率。首先选出一个基本面较好的低市净率的股票组合，这是衡量上市公司内在价值的价值型思想。然后，在此基础上，考察上市公司的净资产收益率，这是衡量上市公司盈利能力和未来成长性的成长性指标，通过低市净率和高净资产收益率的结合，最终构建 GARP 组合，这样在股票市场发生价值与成长的风格轮动时，GARP 策略可以兼顾到两个方面，所以预期可以获得比较稳定的收益。

价值成长投资策略^[114,115]以组合投资为投资原则,首先筛选基本面比较好,又具有比较低的市净率的数量较多的股票,在得到这些股票后,在这些股票中再挑选一些行业龙头的股票进行组合,通过购买多只股票进行投资。这样做的原因有二个,一是这样选择的股票安全边际相对较高,另一个就是多个股票的组合,可以降低投资的风险,这样投资者可以获得比较稳健的收益。

6.3.4 以价值成长投资策略为指导的选股模型指标体系的构建

1. 影响因子构建

从上的 GARP 策略的投资思想可以看出,该策略倾向于使用多个指标来衡量股票的可投资性,且这些指标包括股票的基本面、股票的价值和股票的成长性等三个方面,因此本模型在构建影响因子时,也以此思想为指导进行。首先,从国信证券工程师焦健等人提出的六因子量化选股模型出发,根据价值成长投资策略的思想,适当加入其他因子进行组合,使指标体系中同时包含价值因子和成长因子。

国信证券工程师焦健等人提出的六因子量化选股模型,由市净率、市盈率、ROA、前一月股票收益率、EPS 一致预期变化率、EPS 一致预期六个指标构建。前二个指标都是价值因子,是衡量市场及股票是否合理的常用指标,在本模型中给予保留。后四个指标体现了成长因子的概念,由于前一月股票收益率是在模型中作为目标变量,因此就余下 ROA、EPS 一致预期变化率和 EPS 一致预期三个指标。这样模型的影响因子共有五个指标,其具体的概念如下:

(1) 市净率。它是股票市场价格与该股票每股净资产的比值。该指标是股票价格的风向标。由于净资产是企业的自有资产,也就是所有者权益,这部分资产是企业赖以生存的基础,如果一个企业的净资产过小,就会出现破产清算的危险。该指标可以较好地反应企业的预期价值。股票的市场价格是市场对企业的估价,而企业的净资产是企业实际的价值,这个比值是预期和实际的比较,如果两者相差甚远,则比值会过小,这个时候投资该股票的风险会更小,反之则风险更大。该指标是一个典型的价值指标。

(2) 市盈率。它是股票市场价格与该股票每年每股盈利额的比值。这个指标是进行股票投资最基本也是最重要的指标之一。这个指标可以反应市场对这支股票的预期,如果这个指标很高,说明市场对这个股票对应的企业的盈利能力有很高的预期,这是一个典型的成长型指标,一般认为该比率小于 30,且大于 20 时,应该比较正常,如果超出 30,说明投资人对股票的预期过高,投资该股票的就存在一定的风险;如果这个指标低于 20,或者低得更多,说明该股票的市场价格过低,投资的风险相对较小,就可以进行认购。

(3) 资产收益率(Return on Assets, ROA),它是企业的税前利润和平均总资产的比值。简单地说就是企业的 1 元钱的资产能换回多少收益,该指标反应的是企业的总资产的获利能力,它将规模不同的企业放到同一个标准上来衡量,有利于投资人透

过现象看到企业真正的本质的东西。这个比值越大,说明企业总资产的回报率就比较高,说明企业在节约支出,增加企业主营业务收入方面有较好的表现;这个指标越小,说明企业资产的利用效率很差,遇到这种状况的企业应深入分析企业存在资产利用率低的原因,根据原因去查找问题,针对问题进行纠正企业的经营管理策略。

(4) EPS 一致预期。该指标通过对各卖方分析师的调查(研究报告,电话,Email 等方式),获得上市公司盈利预期数据,然后取平均值,这个平均数据称之为一致预期数据。该项数据的核心目标是通过专家调查的方式,取得市场对公司未来盈利的预期水平,使用数量化的形式反应市场预期。

国内目前提供一致预期数据服务的有三家:朝阳永续、大智慧和 591gu。朝阳永续是最早推出一致预期数据服务的公司,产品最专业。起初朝阳永续的一致预期数据产品主要是面向买方机构,价格比较昂贵。

大智慧在 2011 年 11 月份也推出了一致预期数据,该公司强调这个指标的科学性和权威性,一是信息全面、数据精准、力促信息对称,立足于最全最新的机构研究成果;二是通过以严谨的四因素衰变矩阵模型处理数据,从而真实反映市场的共同预期;三是应用价值投资理念挖掘投资价值,进行多维数据监控,发现预期突变机会,提供相对估值排行等功能,为投资者发现价格洼地。

591gu.com 提供的一致预期数据功能较朝阳永续和大智慧简单些,主要有个股一致预期 EPS 与股价走势对比,未来两年一致预期 EPS、PE 和 PB 数据,未来两年一致预期 PE 排行榜、一周内一致预期 EPS 调增排行榜等。但 591gu 的一致预期数据不收费,可免费使用。

鉴于朝阳永续在数据方面的权威性,本研究的一致预期数据均来源于该公司。

(5) EPS 一致预期变化率。该指标是截止到月底对该年 EPS 的预测和截止到上月底对该年 EPS 的预测的比值,这项指标反应市场预期的变化情况,该项指标也通过朝阳永续公司提供。

由于随机森林算法可以处理的属性变量不受限制,且优化后的随机森林算法可以较好地处理连续性数据,考虑上述五个指标不能全面反映企业的财务和金融状态,以根据 GARP 策为指导,本研究拟增加以下 4 个指标:

(1)净资产收益率(ROE),又称股东权益报酬率,是报告期净利润与报告期末净资产的比值。它是判断上市公司盈利能力的一项重要指标,一直受到证券市场参与各方的极大关注。如果公司净资产收益率呈上升趋势,那么肯定是企业的盈利模式发生变化,主要有以下几大因素:产品毛利上升,产品结构向高毛利发展,资产周转率上升,财务杠杆加大,而这些因素都是不能持久了,所以说净资产收益率是挑选一个可持续高速增长企业很好的指标,而这正是巴式风格最理想的投资指标。

(2)存货周转率,它是衡量和评价企业购入存货、投入生产、销售收回各环节

管理状况的综合性指标。该指标越高，说明企业经营效率高，存货周转的速度很快，企业的销售能力就越强，从而可以减少营运资金在存货占用的资金，节省成本，是企业快速发展的标志。反之，该指标越低，说明企业的存货使用效率低下，企业的销售效率不高，存货的周转速度很差，说明企业的发展速度有停滞的危险。因此这个指标是挑选成长型股票必须具备的一个指标。

(3)资产负债率，它是企业负债总额占企业资产总额的百分比。这个指标反映了在企业的全部资产中由债权人提供的资产所占比重的大小，反映了债权人向企业提供信贷资金的风险程度。由于该指标是企业偿债能力的体现，在不同的经济形势下，此比率需进行相应的调整，是企业经营策略的一种反映，体现企业管理层的财务管理能力，是企业价值因子的一个重要指标。

(4)流通市值，它是公司的股票在证券市场里的交易价格乘以总股本的值，反映了市场在某个时期对公司的看法，是公司价值的一种市场体现。一般来说流通市值是围绕公司价值上下波动的。当然这种波动是随机的，不对称的，它往往受到市场预期的影响，容易出现极端化的倾向。流通市场指标是价值投资的核心，如果市场过低低估股票市值，则该股票就是值得投资的股票。

根据以上分析，本文使用的选股指标体系如下：

表 6.1 选股指标体系一览

序号	指标名称	简称	含义	因子维度	反映情况
1	市净率	PB	每股股价/每股净资产	价值因子	投资风险
2	市盈率	PE	(月底收盘价×汇率×总股本)/净利润(归属于母公司股东)	价值因子	盈利能力
3	资产负债率	DTA	负债总额/资产总额	价值因子	偿债能力
4	流通市值	MV	月底 A 股流通市值	价值因子	股票规模
5	总资产净利率	ROA	公司净利润/平均资产总额	成长因子	盈利能力
6	存货周转率	IT	销货成本/平均存货余额	成长因子	营运能力
7	净资产收益率	ROE	净利润/平均股东权益	成长因子	盈利能力
8	EPS 一致预期	EPS	截止到月底对该年 EPS 的预测	成长因子	市场预期
9	EPS 一致预期变化率	EPSR	截止到月底对该年 EPS 的预测/截止到上月底对该年 EPS 的预测-1	成长因子	市场预期

以上九个指标既反应了企业的营运能力、盈利能力、成长能力和市场预期，又反应了成长因子和价值因子的内容，使体系充分体现了价值成长投资策略（GARP）的主要思想。体系中有一项重要内容就是加入了 EPS 预期，这个一般的研究文献中没有涉及，加上这项指标的原因是基于股票价格受人们的预期影响很大，如果在指标体系中仅反映公司过去的情况的指标，则选股指标体系的局限性是很大的。

2. 目标变量构建

根据价值成长投资策略的长期性特点, 兼顾模型稳定性, 使用移动平均的统计方法, 将目标变量构建为股票前 12 个月的月平均涨跌幅均值 $\bar{R}$ 的函数。使用移动平均统计方法的原因是由于股票价格存在一定的递延性。其计算步骤分为三步:

首先, 计算每只股票前 12 个月的月平均涨跌幅 $\bar{R}$, $\bar{R}$ 使用几何平均法构建, 公式如下:

$$\bar{R} = \left(\sqrt[12]{\prod_{i=1}^{12} (R_i + 1)} \right) - 1 \quad i=1, 2, \dots, 12 \quad (6.9)$$

其中: R_i 为该股票前第 $i-1$ 个月的月涨跌幅;

$R_i = (P_{\text{月末}} - P_{\text{月初}}) / P_{\text{月初}}$, $P_{\text{月初}}$ 为月初股票价格, $P_{\text{月末}}$ 为月末股票价格。

其次, 将该行业全体股票的 $\bar{R}$ 加权平均, 计算行业前 12 个月的月平均涨跌幅 $\bar{hR}$, 公式如下:

$$\bar{hR} = \frac{1}{12} \sum_{j=1}^{12} \bar{R}_j \quad j=1, 2, \dots, 12 \quad (6.10)$$

最后, 将每只股票的 $\bar{R}$, 与全体样本平均值的 $\bar{hR}$ 比较, 构建目标变量, 公式如下:

$$Y_j = \begin{cases} 1\text{类} & \text{当 } \bar{R}_j \geq \bar{hR} \text{ 时} \\ -1\text{类} & \text{当 } \bar{R}_j < \bar{hR} \text{ 时} \end{cases} \quad j=1, 2, \dots, 12 \quad (6.11)$$

其中: $\bar{R}_j$ 为第 j 支股票的前 12 个月的平均月涨跌幅;

$\bar{hR}$ 为该行业的前 12 个月的月平均涨跌幅。

6.4 优化后的随机森林算法对优质股票池的选择

6.4.1 样本的选择

样本选取区间为 2012 年 1 月至 2013 年 2 月, 财务和股票数据来自 Wind 资讯, EPS 数据来自朝阳永续信息技术有限公司, 样本个数共计 4406 个, 月度样本选取时, 剔除了 ST 股和数据缺失的股票, 每期股票样本范围在 363 支股票左右, 因每个月的股票数据缺失情况不一, 因此每个月的样本数据会稍有不同。

在 363 支股票中, 有 202 支股票属于计算机、通信和其他电子设备制造业, 另 161 支股票属于批发和零售业, 该行业分类为 2012 年证监会新制定的行业分类。从实时性和可比性来说, 选择 2012 年为起点较宜, 且行业分类也更为合适。

选择这 363 支股票进行分析, 是基于以下三点考虑:

1. 进行选股时, 所选股票均为同一行业, 这样可以剔除行业差异, 在指标选择时, 可以不考虑行业因素;
2. 通过两个行业的对比, 有利于分析算法的普适性值;

3. 所选行业的股票均超过 100 支，股票数量较大，有利于统计观察。其他制造业分类中，股票数量较少，不利于统计观察。

6.4.2 指标数据的财报匹配

模型中有些指标的数据来源于上市公司的财务报告，上市公司财报公布时间如下表：

表 6.2 上市公司财报公布时间

序号	财报分类	公布截止期
1	年报	4 月 30 日
2	一季度报	5 月 31 日
3	半年报	8 月 31 日
4	三季度报	10 月 30 日

在进行指标计算时，月度样本数据应用的财报数据如下表：

表 6.3 样本数据与财报匹配规则表

序号	月度时间	财报分类	序号	月度时间	财报分类
1	1 月	上年三季度报	7	7 月	当年一季度报
2	2 月	上年三季度报	8	8 月	当年半年报
3	3 月	上年三季度报	9	9 月	当年半年报
4	4 月	上年年报	10	10 月	当年三季度报
5	5 月	当年一季度报	11	11 月	当年三季度报
6	6 月	当年一季度报	12	12 月	当年三季度报

6.4.3 数据预处理

在上述模型的 9 个指标中，仅流通市值（MV）不是比值，因此对其进行 z-score 标准化，公式为：

$$SMV_i = \frac{MV_i - \overline{MV}}{\delta_{mv}} \quad i=1,2,3,\dots,n \quad (6.12)$$

其中： MV_i 为第 i 支股票的流通市值；

$\overline{MV}$ 为样本股票流通市值的均值；

δ_{mv} 为样本股票流通市值的标准差。

由于受股票市场中各种偶然因素的影响，股票数据存在许多的噪声。这些噪声严重地影响了随机森林算法的使用效果，因此必须对模型中某些指标，选用合适的去噪方法进行处理。由于股票数据本身具有非平稳性、非线性和噪声较多的特点，数据的波动很大。本文尝试使用小波变换算法对模型中的股票价格和总市值数据进行预处理，该算法使用 R 软件编程实现，在去除噪声时，调用 wavelets 数据包，使用 HAAR 小波进行分解，使用 DWT 函数进行强制消噪处理，对股票价格和总市值数据进行处理。

由于 9 个变量的均为连续性变量，但优化后的随机森林算法处理变量时，需要对变量进行级别的划分，即连续变量离散化，因此需要对 9 个变量进行离散化处理。离散化处理方法在第四章已经有了详细的介绍，使用其提出的 COR_CHI2 算法进行数据的离散化处理。

6.4.4 当月实际表现分析

在进行数据预处理后，分行业对第五章提出的优化后的随机森林算法进行训练和预测，设置森林中树木个数为 1000，进行 1000 次实验，实验数据分析如下：

1. 模型在两个行业中分类正确率分析

分别使用 2012 年 1 月至 2013 年 1 月，计算机、通信和其他电子设备制造业的 2335 个训练样本，和批发和零售业的 1708 个训练样本，对优化后的随机森林算法进行训练，使用训练得到的随机森林分别对两个行业 2013 年 2 月股票进行分类预测，分类准确率如下表所示：

表 6.4 2013 年 2 月两行业分类正确率对比表

	计算机、通信和其他电子设备制造业	批发和零售业
分类为上涨股票实际上涨的股票个数①	132	98
分类为上涨股票实际下跌的股票个数②	16	20
分类为下跌股票实际上涨的股票个数③	23	15
分类为下跌股票实际下跌的股票个数④	31	28
正确分类个数⑤=①+④	163	126
错误分类个数⑥=②+③	39	35
全体样本个数⑦=⑤+⑥	202	161
模型分类正确率⑧=⑤/⑦	80.69%	78.26%

从表 6.4 可以看出：

(1) 该随机森林算法的分类精度在两个行业中分类正确率均在 75% 以上，在计算机、通信和其他电子设备制造业中的表现更佳，分类精度达到了 80%。这就说明优化后的随机森林算法可以较好地进行优质股票池的选择。

(2) 从两个行业来看，优化后的随机森林算法的分类精度存在一些差异，相差 2.43 个百分点，这个差异应该和数据集的大小和行业本身的特点有关，具体原因需要进一步探讨。

2. 该算法筛选出的优质股票在当月的实际表现分析

为了研究方便，仅以计算机、通信和其他电子设备制造业为例，分类使用优化后的随机森林算法筛选出的股票的实际涨跌情况和行业的涨跌情况进行对比分析，从而检验算法的适用性。2013 年 2 月份使用优化后的随机森林算法筛选出的优质股票池涨跌幅如下表所示：

表 6.5 2013 年 2 月优质股票池的实际涨跌情况分析表

指 标		该行业股票实际涨跌情况	分类为优质股票实际涨跌情况	分类为劣质有股票实际涨跌情况
股票个数		202	148	54
均值		-.3924	.1590	-2.1142
中值		-.5224	-.1473	-2.3036
标准差		2.31322	2.19406	1.77980
极小值		-5.79	-4.87	-5.79
极大值		7.56	7.56	2.45
百分位数	25	-1.9913	-1.3980	-3.6474
	50	-.5224	-.1473	-2.3036
	75	.9853	1.3600	-.7049

由表 6.5 可知, 2013 年 2 月计算机、通信和其他电子设备制造业行业当月平均涨跌幅为-0.3924%, 整个行业下跌, 分类为优质的 148 支股票中, 当月平均涨跌幅为 0.1590%, 跑赢行业平均水平 0.6%, 且最大值也在该类股票中, 最小值不在该类股票中, 说明该选股模型在 2013 年 2 月份有较好的表现。

6.5 本章小结

本章在对现有选择优质股票的模型和使用的分类算法入手, 通过比较分析价值策略和成长策略, 分析了价值成长投资策略 (GARP) 理念的选股模型指标体系, 选取了 2012 年 1 月至 2013 年 2 月间 360 多支股票的 4406 个样本数据, 在对数据进行离散化和平稳化预处理后, 使用第五章得到的随机森林优化算法实现了较高正确率的股票池选择。通过分析优选后的股票池在行业平均收益、最值方面的实际表现, 验证了该量化选股模型性能优异。

选择合适的数据挖掘算法进行分类选股, 是量化投资的一个重要内容, 使用随机森林算法进行股票选择是股票投资领域的有益探索。通过本章的研究, 使随机森林算法得到更好的推广, 使机构和投资人获得了一个较好的投资分析工具。当然本文在选股时, 局限于某个行业, 这主要是考虑到行业因素是股票选择时另一个很重要的前提条件, 模型中加入行业因素和对小行业股票的分类选择是后续研究的重要内容。

7 总结与展望

随机森林算法作为一种多分类器的典型代表, 由于其较好地克服了单分类器容易过拟合和局部最优解的问题, 具有较好的分类效果和较佳的容错性, 被成功地应用在众多的领域, 并被很多研究人员进行了深入的研究和优化。

7.1 本文的创新点

本文主要对随机森林算法处理非平衡数据和连续性数据时存在的问题, 以及算法的分类效果的提升等方面做了大量的优化研究, 并将该优化后的算法应用到了量化投资的股票选择领域。

1.在非平衡数据集的优化方面, 本文提出了一种新的解决非平衡问题的算法, 该算法较好地解决了数据集的非平衡问题, 较显著地提升了随机森林算法在非平衡数据集上的分类性能。

2.在连续变量离散化优化方面, 本文提出了一种新的连续变量离散化的算法, 该算法可以有效地提升连续变量信息约简程度, 使得随机森林算法在进行节点分裂时运行效率更高, 从而提升了随机森林算法的运行效率。

3.本文对随机森林算法在节点分裂规则选择上做了一些有益的探索, 设计了一种基于线性组合的节点分裂混合随机森林算法, 该混合算法通过提升森林中决策树的平均强度, 降低森林中决策树的平均相关度, 使得随机森林算法的分类精度得到了一定程度的提升。并建立统计模型估计了混合算法的稳定状态, 为用户提供了一种较好的随机森林参数设置参照依据。

4.在价值成长投资策略的选股指标体系的基础上, 通过小波分析和 COR_CHI2 算法进行数据预处理, 使用节点分裂混合算法形成的随机森林成功地实现了优质股票池的选择, 可为投资者进行有针对性的投资组合提供统计支持。在量化投资领域, 只有几个大的证券公司在决策树进行研究, 使用随机森林算法进行选择优质股票的做法为数不多, 因此在国内的研究中较为前沿。

7.2 今后的研究思路

虽然优化后的随机森林算法的性能均有了一定程度的提升, 但是综合随机森算法现有的优化思路看, 该算法优化在以下几个方面还可以继续进行研究与分析:

1.在连续变量离散化优化方面, 由于卡方拟合只是一种近似, 能否在近似的过程中加入一些权重, 这些权重的计算根据目标变量或属性变量的分布进行, 从而更好了使这种近似接近真实的数据集。

2.在对随机森林分类性能优化方面, 由于时间关系, 本文只是做了节点分裂的线性函数, 是否有其他函数形式能更好地提升算法的性能, 这是一个值得探讨的问题。

3.通过引入更好的统计分析方法, 和随机森林算法进行组合, 这是随机森林算法

优化的一个重要方向,限于本人对现有统计方法掌握得不够的原因,本文没有在此方面进行进一步的探讨,如果能在某个特定的应用引入一种新的算法,将该算法和随机森林算法进行组合,从而衍生出一种新的随机森林算法将是一个不错的研究方向。

4.在比较 bagging 和 boosting 方法时,本文还没有进行充分的论述,可以从随机森林初始数据集抽样方式的选取上进行进一步的探讨,比如引入 boosting 的权重思想,用于产生训练集,从而提升随机森林算法的性能。

5.随着大数据时代的来临,传统的随机森林算法必定会成为过去,随机森林算法一定要紧跟时代的步伐,在大数据背景下对随机森林算法进行全新的优化是势必未来的随机森林算法优化领域热门话题,如何实现随机森林算法的分布式、并发式计算,将是一个重要的研究方向。

8 参考文献和注释

- [1] T. G. Dienerich. Machine Learning Research for Current Directions[J] AI Magazine. 1997, 18(4):97~136
- [2] R.E. Sehapire. The Strength of Weak Learnability[J]. Machine Learning. 1990, 5(2): 197~227
- [3] L. Breiman. Bagging Predictors. [J] Machine Learning, 1996, 24(2):123~140
- [4] T.K. Ho. Random Decision Forest [J]. In Proceedings of the 3rd International Conference on Document Analysis and Recognition. Montreal, Canada, 1995, 8:278~282
- [5] T. K. Ho, The Random Subspace Method for Constructing Decision Forests. [J] IEEE Transactions on Pattern Analysis and Machine Intelligence, 1998, 20(8):832~844.
- [6] L. Breiman, Random Forests[J]. Machine Learning, 2001, 45(1):5~32
- [7] Parkhurst D F, Brenner K P, Dufour A P, Wymer L J. Indicator Bacteria at Five Swimming Beaches—Analysis Using Random Forests[J]. Water Research, 2005, 39(7)
- [8] Smith A, Sterba-Boatwright B, Mott J. Novel Application of a Statistical Technique, Random Forests, in a Bacterial Source Tracking Study[J]. Water Research, 2010, 44(14)
- [9] Perdiguerro-Alonso D, Montero F E, A Kostadinova, Raga J A, Barrett J. Random Forests. a Novel Approach for Discrimination of Fish Populations Using Parasites as Biological Tags[J]. International Journal for Parasitology, 2008, 38(12)
- [10] Gislason P O, Benediktsson J A, Sveinsson J R. Random Forests for Land Cover Classification[J]. Pattern Recognition Letters, 2006, 27(4)
- [11] Jan P, Bernard D B, Niko E C V, Roeland S, Sven D, Piet D B, Willy H. Random Forests as a Tool for Ecohydrological Distribution Modelling[J]. Ecological Modelling, 2007, 207(2/4).
- [12] Lee S L A, Kouzania A Z, Hu E J. Random Forest Based Lung Nodule Classification Aided by Clustering[J]. Computerized Medical Imaging and Graphics, 2010, 34(7)
- [13] Diaz-Uriate R, Andres S A D. Gene Selection and Classification of Microarray Data Using Random Forest [J]. BMC Bioinformatics, 2006, 7(3)
- [14] Chen X W, Liu M. Prediction of Protein-protein Interactions Using Random Decision Forest Framework [J]. Bioinformatics, 2006, 21(24)
- [15] Pal M. Random Forest Classifier for Remote Sensing Classification [J]. Remote Sens, 2005, 26(1)

- [16] Ham J, Chen Y C, Crawford M P, Ghosh J. Investigation of the Random Forest Framework for Classification of hyperspectral Data[J]. IEEE Trans. Geosci. Remote Sens, 2005, 43(3).
- [17] Gislason P O, Benediktsson J A, Sveinsson J R. Random Forests for Land Cover Classification[J]. Pattern Recogn. Lett, 2006, 27(4)
- [18] Xu P, Jelinek F. Random Forests and the Data Sparseness Problem in Language Modeling [J]. Computer Speech & Language, 2007, 21(1)
- [19] Auret L, Aldrich C. Change Point Detection in Time Series Data with Random Forests[J]. Control Engineering Practice, 2010, 18(8)
- [20] S.Q.Guo, C.Gao, J.Yao, L.Xie. An Intrusion Detection Model Based on Improved Random Forests Algorithm[J]. Journal of Software. 2005, 16(8):1490-1498
- [21] 张华伟. 基于层次分类和集成学习的文本分类技术研究[D]. 江西师范大学, 2007, 5
- [22] 吕飒丽, 汪强虎, 李霞, 郭政. 基于决策森林特征基因的两种识别方法[J]. 生物信息学. 2004, 2(3):20~23
- [23] W. Fan, H.X.Wang, P.S.Yu, S.Ma. Is Random Model Better On Its Accuracy and Efficiency. In Proceedings of the Third IEEE International Conference on Data Mining. IBM Thomas J. Watson Res. Center, Hawthorne, NY, USA, 2003, 12:51~58
- [24] 李霞, 张田文, 饶绍奇等. 特征基因挖掘的决策森林方法[J]. 哈尔滨工业大学学报. 2004, 36(4):480~483
- [25] Gall J, Lempitsky V. Class-Specific Hough forests for object detection[C]//Proceedings of IEEE Conference on Computer Vision and Pattern Recognition. Los Alamitos: IEEE Computer Society Press, 2009: 1022~1029
- [26] Gall J, Yao A, Razavi N, et al. Hough forests for object detection, tracking, and action Recognition [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2011, 33(11):2188~2202
- [27] Criminisi A, Shotton J, Robertson D, et al. Regression forests for efficient anatomy detection and localization in CT studies [M]. Lecture Notes in Computer Science. Heidelberg: Springer, 2011, 6533:106-117
- [28] Ishwaran H, Kogalur U B, Blackstone E H, Lauer M S. Random Survival Forests [J]. The Annals of Applied Statistics, 2008, 2
- [29] Ishwaran H, Udaya B, Kogalur. Consistency of Random Survival Forests[J]. Statistics and Probability Letters, 2010, 13~14.
- [30] Nicolai, Meinshausen. Quantile Regression Forests [J]. Journal of Machine

Learning Research, 2006, 7(6).13~14.

[31] 马景义, 吴喜之, 谢邦昌.拟自适应分类随机森林算法[J] 数理统计与管理 2010, 9.29(5):805-811

[32] A. Prinzie, D.V.D.Poel.Random Forests for Multiclass Classification:Random MultiNomial Logit. Expert Systems with Applications. 2008, 34(3):1721~1732

[33] 吴琼, 李运田, 郑献卫, 面向非平衡训练集分类的随机森林算法优化.工业控制计算机 2013.26(7):89~90

[34] 杜均 代价敏感学习及其应用[D].中国地质大学(武汉),2009

[35] 雍凯.随机森林的特征选择和模型优化算法研究[D]. 哈尔滨工业大学, 2008,12

[36] 孙丽丽.基于属性组合的随机森林[D]. 河北大学,2011.5

[37] L. Breiman, Bagging predictors.Mach.Learn, 1996, 24 (2): 123~140

[38] Schapire R E.The Strength of Weak Learn ability[J]. Machine Learning, 1990:5:197~227

[39] Freund Y.Boosting a Weak Algorithm by Majority[J]. Information and Computation, 1995 ; 121 2):256~285

[40] Dietterich T.An Experimental Comparison of Three Methods for Constructing Ensembles of Decision Trees:Bagging, Boosting and Randomization [J]. Machine Learning, 2000, 40(2)

[41] Breiman L.Randomizing Outputs to Increase Prediction Accuracy[J]. Machine Learning, 2000, 40(3)

[42] D.S.Yeung , W.W.Y. Ng , D.Wang , E.C.C.Tsang ,X.Z.Wang.Localized Generalization Error and Its Application to Architecture Selection for Radial Basis Function Nerual Network[J]. IEEE Trans.on Neural Networks. 2007, 18:1294~1305

[43] <http://www.cs.waikato.ac.nz/Jml/weka/>

[44] G.Izmirlian.Application of the random forest classification algorithm to a seldi-tof proteomics study in the setting of a cancer prevention trial[J]. Annals of the New York Academy of Sciences, 2004,1020: 154~174

[45] D.R.Cutler et al.Random forests for classification in ecology[J]. Ecology, 2007,88, 2783~2792

[46] Weiss G M.Mining withRarity.A Unifying Framework[J] . SIGKDD Explorations, 2004, 6(1) : 7~19

[47] 黄衍, 查伟雄.随机森林与支持向量机分类性能比较[J].北京:软件, 2012.33(6):107~110

- [48] 李建更, 高志坤. 随机森林针对小样本数据类权重设置[J]. 计算机工程与应用, 2009, 45(26):131~134
- [49] Weiss G M. Mining with Rarity : A Unifying Framework[J]. SIGKDD Explorations, 2004, 6(1): 7-19
- [50] Fan W, Stolfo J S, Zhang J X. AdaCost: misclassification cost Sensitive boosting [C]. Proceedings of the 16th International Conference on Machine Learning, 1999:97~105.
- [51] Domingos P. MetaCost: a general method for making classifiers cost sensitive [C]. proceedings of the 5th International Conference on Knowledge Discovery and Data Mining, 1999:155~164
- [52] Kotsiantis S B, Pintelas P E. Mixture of Expert Agents for Handling Imbalanced Data Sets [J]. Annals of Mathematics Computing & Teleinformatics, 2003, 1(1):46~55.
- [53] Estabrook A, Taeho J, Japkowicz N. A multiple resampling method for learning from imbalanced data sets[J]. Computational Intelligence, 2004, 20(1):18~36.
- [54] HAN J W, KAMBER M. Data Mining Concepts and Techniques[M]. Morgan Kaufmann Publishers, 2000
- [55] METHA M, AGRAWAL R, RISSANEN J. SLIQ: A Fast Scalable Classifier for Data Mining[J]. Lecture Notes in Computer Sci. Proc. of the 5th Int. Conf. on Extending Database Tech.[C], 1996:18~33.
- [56] 陈荣鑫. R 软件的数据挖掘应用[J]. 重庆工商大学学报(自然科学版), 2011, 28(6): 602-607
- [57] Chawla NV, Bowyer KW, Hall LO. SMOTE: Synthetic Minority Over-sampling Technique[J]. Journal of Artificial Intelligence Research, 2002(16):321~357.
- [58] Han H, Wang W Y, Mao B H. Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning [C]. Proc of International Conference on Intelligent Computing(ICIC'05), 2005:878~887.
- [59] 王春玉. 非平衡数据集分类方法研究及其在电信行业中的应用[D]. 浙江大学 2011.6
- [60] 薛薇. 非平衡数据集的改进 SMOTE 再抽样算法统计研究[J]. 2012.6, 29(6):95~98
- [61] 何晓群. 多元统计分析 [M]. 中国人民大学出版社. 2008.9:73
- [62] Dougherty J. Kohavi R. and Sahami M. Supervised and Unsupervised Discretization of continuous feature. Machine learning: Proc. 12th Int'l Conf, 1995, 194~202.

- [63] Liu Huan,Hussain Farhad,Tan Chew Lim et al.Discret ization: An Enabling Technique[J]. Data Mining and Knowledge Discovery , 2002, 6(4), 393~423
- [64] LiuH,HussainF.Diseretization:An Enabling Teehnique[J]. DataMiningand Knowledge Diseovery, 2002:393~423.
- [65] 李慧. 基于粗糙集理论的连续属性离散化算法研究[D]. 辽宁师范大学, 2010, 5.
- [66] Kerber R. ChiMerge : discretization of numeric attributes[C].Proceedings Ninth National Conference On Artificial Intelli-gence. AAAI Press, 1992, 123~128
- [67] Liu H.Setiono R.Feature selection via discretization [J]. IEEE Transactions on Knowledge and Data Engineering, 1997, 9(4) : 1642~645
- [68] Tay EH, Shen L. A modified chi2 algorithm for discretization [J]. IEEE Transactions on Knowledge and Data Engineering.2002, 14(3) : 666~670
- [69] Chao-TonSu, Jyh-HwaHsu.An extended chi2 algorithm for discretization of real value attributes[J]. IEEE Transactions on Knowledge and Data Engineering,2005, 17(3) : 437~441
- [70] 杜荣骞.生物统计学[M].高等教育出版社, 2003, 4
- [71] J.Y.Ching, A.K.C.Wong and K.C.C.Chan.Class-Dependent Discretization for InductiveLearning from Continuous and Mixed-Mode Data[J].IEEE Trans.Pattern Analysis and MachineIntelligence, 1995, 17 (7) : 641~651
- [72] L.A. Kurgan and K.J.Cios.CAIM Discretization Algorithm[J]. IEEE Transactions onKnowledge and Data Engineering, 2004, 16(2): 145~153
- [73] Cheng-Jung Tai, Chien-I. Lee, Wei-Pang Yang. A discretization algorithm based on Class-Attributes Contingency Coefficient[J]. Information Sciences , 2008,178:714~731
- [74] Chien-I Lee, Cheng-Jung Tsai, Ya-Ru Yang, Wei-Pang Yang.A Top-Down and Greedy Method for Discretization of Continuous Attributes. Proceedings of the Fourth International Conference on Fuzzy Systems and Knowledge Discovery. 2007,1:472~476
- [75] 白根柱, 裴志利, 王建, 孔英, 刘丽莎. 基于粗糙集理论和信息熵的属性离散化方法[J]. 计算机应用研究, 2008, 25(6):1701~1703
- [76] 贾俊平, 何晓群, 金勇进. 统计学[M]. 中国人民大学出版社(第四版), 2011, 6
- [77] 杨振海, 程维虎, 张军舰. 拟合优度检验[M]. 科学出版社, 2011, 3
- [78] 闫德勤, 张丽平. 连续属性离散化 IntegralChi2 算法[J]. 小型微型计算机系统, 2008, 4(4):691~693
- [79] 杨秋洁. 基于 IV 属性选择的随机森林模型研究[D], 合肥工业大学, 2010,

- [80] M.Dash, H.Liu.Feature selection for Classification[J]. Intelligent Data Analysis, 1997, 1(3):131~156.
- [81] G.Izmirlian.Application of the random forest classification algorithm to a seldi-tof proteomics study in the setting of a cancer prevention trial[J]. Annals of the New York Academy of Sciences, 2004:154~174
- [82] D. R. Cutler et al.Random forests for classification in ecology[J].Ecology, 2007:2783~2792
- [83] Brian Van Essen,Chris Macaraeg, Maya Gokhale and Ryan Prenger. Accelerating a random forest classifier: multi-core, GP-GPU? [J].2012 IEEE 20th International Symposium on Field-Programmable Custom Computing Machines,2012:232~239
- [84] Ruggieri S.Efficient C4.5.IEEE Transactions on Knowledge and Data Engineering[J]. 2002, 14(2):438~444
- [85] 栾丽华, 吉根林.决策树分类技术研究[J]. 计算机工程, 2004, 30(9):94~96
- [86] 焦健, 赵学昂, 葛新元.CART 决策树的行业选股方法[R]. 深圳:国信证券经济研究所, 2010
- [87] 赵永进.基于数据挖掘的股票分析与预测研究[D].河南:郑州大学, 2005.
- [88] 张建军.基于数据挖掘的股票数据分析[D].山东:中国石油大学(华东), 2010
- [89] 桑雨, 闫德勤, 梁宏霞, 李克秋.对 Chi2 系列算法的改进方法[J].小型微型计算机系统,2009, 3(3):524~529
- [90] 李云飞.基于人工智能方法的股票价值投资研究[D].黑龙江:哈尔滨工业大学, 2008
- [91] Wang Shouyang, YuLean, KKLai.Crude oil Price Forecasting with TEIEI ethodology [J]. Journal of System Science and ComPlexity, 2005, 8(2):145~166
- [92] 方匡南, 吴见彬, 朱建平, 谢邦昌.随机森林方法研究综述[J]. 统计与信息论坛,2011,3(26):32~38
- [93] 孙媯.基于数据挖掘的股票分析和预测模型的设计与应用. 北京邮电大学软件[D],2011,8
- [94] 陈光兴.张一明.浅谈价值成长投资策略在中国股市的适用性[J]. 经营管理者, 2010(24):8~8
- [95] 严高剑.胡浩.马坚.李颖.GARP 投资策略-成长与价值并重[R]. 中信证券研究所, 2008.

- [96] HODRICK R J, PRESCOTT E C. Postwar US Business Cycles: An Empirical Investigation[J]. Journal of Money, Credit, and Banking, 1997: 1~16.
- [97] KIM S J, KOH K, BOYD S.L1 Trend Filtering[J].Siam Review, 2009, 51(2) : 339~360.
- [98] OSBORN D R. Moving Average Detrending and the Analysis of Business Cycles[J]. Oxford Bulletin of Economics and Statistics, 1995, 57(4) : 547~558
- [99] LUCAS R E. Two Illustrations of the Quantity Theory of Money[J]. The American Economic Review, 1980, 70(5):1005~1014
- [100] REINSCH C H. Smoothing by Spline Functions[J]. Numerische Mathematik, 1967, 10(3):177~183
- [101] Cohen G, Hilario M. One-class Support Vector Machines with a Conformal Kernel-A Case Study in Handling Class Imbalance[C]. Proc.of SSPR&SPR, 2004, 2004:850~858.
- [102] T. S. Quah.DJIA stock selection assisted by neural network[J]. Expert Systems with Applications, 2008, 35:50~58
- [103] Kim, K, Han, I.Genetic Algorithm Approach to Feature Discretization in Artificial Neural Networks for the Prediction of Stock Price Index[J]. Expert Systems with Applications, 2000, 19:125~132
- [104] Zhang G, Patuwo, B.E Hu, M.Y. Forecasting with Artificial Neural Networks: the State of the Art [J].International Journal of Forecasting, 1998, 14:35~62
- [105] Kamijo K, Tanigawa T.Stock Price Pattern Recognition: A Recurrent Neural Network Approach [J]. Proceedings of the International Joint Conference on Neural Networks, 1990, 215~221
- [106] Vladimir N, Vapnik.Statistical Learning Theory[C], Publishing House of Electronics Industry, 2004
- [107] Yoon Y, Swales G.Predicting Stock Price Performance: A Neural Network Approach [J].Proceedings of the 24th Annual Hawaii International Conference on System Sciences,1991, 156~162
- [108] Trippi R.R., DeSieno D.Trading Equity Index Futures with a Neural Network [J]. Journal of Portfolio Management, 1992, 19:309~317

9 附 录

附录 A 非平衡数据处理 C_SMOTE 算法部分源代码

```
/**
 * 按 C_SMOTE 方法合成
 *
 * @param path 文件存放路径
 * @param filename0 文件名
 * @param k 近邻点的个数
 * @param m 产生的人造样本个数
 * @return
 */
public ArrayList orSmoteCenter(String path, String filename0, int k, int m, String center) {
.....
    for (int i = 0; i < oldMyVector.size(); i++) {
        // 复制初始类, 用于移除当前类
.....
        String single = (String) oldMyVector.get(i);
        tempMyVector.remove(i);
        // 求 K 个近邻
        String[] singlek = this.getNear(k, single, tempMyVector);
        // 合成新类, 并加入到负类集合中
        for (int j = 0; j < rm.length; j++) {
            lastMyVector.add(this.compoundSingle(single, center));
        }
    }
.....
}

/**
 * 计算数据集的重心
 * @param tempMyVector
 * @return
 */
public String calCenter(ArrayList tempMyVector){
    String center = "";
    String singlex = "";
```

```

double[] t = new double[((String)tempMyVector.get(0)).split(",").length];

for (int i = 0; i < tempMyVector.size(); i++) {
    singlex = (String)tempMyVector.get(i);
    String[] tmp1 = singlex.split(",");
    for(int j=0;j<tmp1.length;j++){
        t[j] += Double.parseDouble(tmp1[j]);
    }
}

for(int j=0;j<t.length;j++){
    center += ""+(t[j]/tempMyVector.size())+", ";
}

return center.substring(1,center.length()-1);
}

/**
 * 按照 SMOTE 方法合成新的负类
 * @param singlex
 * @param singley
 * @return
 */

public String compoundSingle(String singlex,String singley){
    String singlep = "";
    String[] tmp1 = singlex.split(",");
    String[] tmp2 = singley.split(",");
    double t1,t2,t3;

    for(int i=0;i<tmp1.length;i++){
        t1 = Double.parseDouble(tmp1[i]);
        t2 = Double.parseDouble(tmp2[i]);
        t3 = t1 + Math.random()*(t2-t1);
        singlep += t3+", ";
    }

    return singlep.substring(1,singlep.length()-1);
}

```

附录 B 连续变量离散化矫正算法部分代码：

```
import java.io.*;
import java.util.*;
import org.rosuda.REngine.Rserve.RConnection;//为连接 R 软件的 JAR 包

/**
 * 通过卡方分布，取得相邻区间的 D 值
 * 如果自由度为 1，需要做连续性矫正
 * 如果各组内的理论次数小于 5，则应把它与其相邻的一组或几组合并，直到理论次数大于
5 为止。
 */

public double calculateByCorrection(ArrayList MyVector1, ArrayList MyVector2, double p,
RConnection c) {
    ArrayList allMyVector = new ArrayList();
    discret discret1 = new discret();
    int N = MyVector1.size() + MyVector2.size();// 两个区间的样本总数
    int numAtt = (((String) MyVector1.get(0)).split(",")).length;// 属性个数
    int[] ri = { MyVector1.size(), MyVector2.size() };// i 区间中样本数
    int i = 0;
    String[] a = new String[N];
    int j = 0;
    for (i = 0; i < MyVector1.size(); i++) {
        allMyVector.add(MyVector1.get(i));
        String[] temp = ((String) MyVector1.get(i)).split(",");
        a[j] = temp[temp.length - 1];
        j++;
    }
    for (i = 0; i < MyVector2.size(); i++) {
        allMyVector.add(MyVector2.get(i));
        String[] temp1 = ((String) MyVector2.get(i)).split(",");
        a[j] = temp1[temp1.length - 1];
        j++;
    }
    String[] a1 = discret1.sortBySet(a);// 进行排序，并求类别
    int[] cj = new int[a1.length];// j 类样本个数
```

```

for (i = 0; i < cj.length; i++) { // 给 CJ 赋值
    cij[i] = discret1.getNumSame(a1[i], numAtt - 1, allMyVector);
}
int[][] aij = new int[2][a1.length];
double[][] eij = new double[2][a1.length];
int k = 0;
for (k = 0; k < a1.length; k++) { // 按列
    aij[0][k] = discret1.getNumSame(a1[k], numAtt - 1, MyVector1);
    eij[0][k] = ri[0] * (cj[k] * 1.0 / N);
}
for (k = 0; k < a1.length; k++) { // 按列
    aij[1][k] = discret1.getNumSame(a1[k], numAtt - 1, MyVector2);
    eij[1][k] = ri[1] * (cj[k] * 1.0 / N);
}
// 计算卡方值
double x1 = 0.0;
for (i = 0; i < 2; i++) { // 按行,只有 2 个区间
    for (k = 0; k < a1.length; k++) { // 按列
        if(eij[i][k]<5.0){ //如果理论值小于 5, 则要合并
            .....
        }
        if((a1.length-1)<2){ // 自由度小于 1
            //进行矫正
            x1 += ((Math.abs(aij[i][k] - eij[i][k])-0.5) * 1.0 *(Math.abs(aij[i][k] - eij[i][k])-0.5))/ eij[i][k];
        }else{
            x1 += ((aij[i][k] - eij[i][k]) * 1.0 * (aij[i][k] - eij[i][k]))/ eij[i][k];
        }
    }
}
double x2 = Double.parseDouble((c.eval("qchisq(" + p + "," + (a1.length - 1) + ")")).asString()); // 调用 R, 计算卡方值
.....
return D;
}

```

附录 C 合随机森林算法部分源代码

```
/**
 * 自助法重采样,独立抽样 k 次, 生成 k 个相互独立的自助样本集,同时生成 OOB 训练集
 * @param k 森林中决策树的个数
 * @param myVector 初始数据集
 * @param F 特征值
 * @param giniCoff CART 算法的系数
 * @param gainCoff C4.5 算法的系数
 * @param rate 叶子节点数据集正负样本比例
 * @return
 */
public data_Node[] bootstrap(int k, tagExampleSet[] myVector, int F,
                             double giniCoff,double gainCoff,double rate) {

    data_Node[] arrDN = new data_Node[k];
    tagExampleSet[] tmpMyVector = new tagExampleSet[myVector.length];
    tagExampleSet[] oobtmpMyVector = new tagExampleSet[myVector.length];
    int a = 0;
    int m = 0;
    int treeDeep = 0;//树的深度
    for (int i = 0; i < k; i++) {
        for (m = 0; m < myVector.length; m++) { // 重置 OOB 样本集
            oobtmpMyVector[m] = new tagExampleSet();// 实例化
            oobtmpMyVector[m].setExample(myVector[m].getExample());
        }
        for (int j = 0; j < myVector.length; j++) { // 独立抽样
            tmpMyVector[j] = new tagExampleSet();// 实例化
            a = (int) (Math.random() * myVector.length);
            if (a == 0) {
                a++;
            }
            if (j == 0) {
                tmpMyVector[j].setExample(myVector[0].getExample());
                oobtmpMyVector[0].setExample(null);// 去掉抽取的样本
            } else {
                tmpMyVector[j].setExample(myVector[a].getExample());
            }
        }
    }
}
```

```

        oobtmpMyVector[a].setExample(null);// 去掉抽取的样本
    }
}
// 计算 OOB
ArrayList<tagExampleSet> listExample = new ArrayList<tagExampleSet>();

for (m = 0; m < myVector.length; m++) { // 重置 OOB 样本集
    if (oobtmpMyVector[m].getExample() != null) { // 没有被抽取
listExample.add(oobtmpMyVector[m]);
        }
    }
    Iterator it = listExample.iterator();
    int tempSign = 1;
    tagExampleSet[] oobMyVector = new tagExampleSet[listExample.size() + 1];
    while (it.hasNext()) {
        oobMyVector[tempSign] = (tagExampleSet) it.next();
        tempSign++;
    }
    oobMyVector[0] = new tagExampleSet();
    oobMyVector[0].setExample(tmpMyVector[0].getExample()); // 给第一行赋上标题
    arrDN[i] = new data_Node();
    arrDN[i] = arrDN[i].buildTree(arrDN[i], tmpMyVector, F, giniCoff,
gainCoff, treeDeep, rate);
    treeDeep = 0;
    if (arrDN[i] != null) {
        arrDN[i].setOob(arrDN[i].testClass(arrDN[i], oobMyVector));
    } else {
        arrDN[i].setOob(0.0);
    }
}
return arrDN;
}

/**
 * 递归建立决策树
 * @param rootNode 根节点
 * @param myVector 初始数据集
 * @param F 特征值

```

```

* @param giniCoff CART 算法的系数
* @param gainCoff C4.5 算法的系数
* @param rate 叶子节点数据集正负样本比例
* @param treeDeep 树的深度
* @return 根节点
*/

public data_Node buildTree(data_Node rootNode, tagExampleSet[] myVector, int F, double
giniCoff,double gainCoff,int treeDeep,double rate) {
    rootNode = new data_Node();

    tagNode minTageNode = OldOrNewChoose_Property(myVector,F,giniCoff, gainCoff);
    rootNode.setLeavenode(1);// 为非叶子节点
    rootNode.setProper_name(minTageNode.getP_name());// 节点对应的属性名称
    rootNode.setProper_value(minTageNode.getDivRule());// 节点规则
    // 按照划分规则将原测试集分为左右 2 个测试集
    int place = -1;
    ArrayList<tagExampleSet> arrLeftVector = new ArrayList<tagExampleSet>();
    ArrayList<tagExampleSet> arrRightVector = new ArrayList<tagExampleSet>();
    //生成按找到的节点分裂左右子集
    for (int i = 0; i < myVector[0].getExample().length; i++) { // 取每个属性
        if (myVector[0].getExample()[i] == minTageNode.getP_name()) {
            place = i;
            for (int j = 0; j < myVector.length; j++) { // 取每个样本
                tagExampleSet tSet = new tagExampleSet();
                tSet.setExample(myVector[j].getExample());
                if (j == 0) { // 加入属性名称行
                    arrLeftVector.add(tSet);
                    tagExampleSet tSet1 = new tagExampleSet();
                    tSet1.setExample(myVector[j].getExample());
                    arrRightVector.add(tSet1);
                } else {
                    if
(minTageNode.getDivRule().contains(myVector[j].getExample()[i])) { // 如果属性值在规则中，则将
该样本加入到左子集中， 否则加入右子集
                        arrLeftVector.add(tSet);
                    } else {
                        arrRightVector.add(tSet);
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}
int tempSign = 0;

//生成左子集，去除对应属性
tagExampleSet[] leftVector = new tagExampleSet[arrLeftVector.size()];
Iterator it = arrLeftVector.iterator();
while (it.hasNext()) {
.....

}
//生成右子集，去除对应属性
tagExampleSet[] rightVector = new tagExampleSet[arrRightVector.size()];
Iterator it1 = arrRightVector.iterator();
tempSign = 0;
while (it1.hasNext()) {
.....
}
int yesNum = 0;
int noNum = 0;
HashSet<String> hs1 = new HashSet<String>();

for (int k = 1; k < leftVector.length; k++) {
    String ans1 = leftVector[k].getExample()[leftVector[0].getExample().length - 1];
    if(ans1.equals("yes")){
        yesNum++;
    }else{
        noNum++;
    }
    hs1.add(ans1);
}
double mk = 0.0;

if(yesNum>noNum){
    mk = (double)yesNum/(double)(noNum+yesNum);
}else{

```

```

        mk = (double)noNum/(double)(noNum+yesNum);
    }
    if (leftVector.length <= 5 || leftVector[0].getExample().length <= 1 || hs1.size() == 1 ||
mk>rate){
        //样本个数太少，样本属性长度为空，所有样本为同一类别，或叶子数据集超过某个比例，
        则为叶子节点，算法结束
        .....//设置叶子节点
    }else{
        rootNode.setLeftNode(buildTree(rootNode.LeftNode,    leftVector,    F,giniCoff,
gainCoff,treeDeep,rate));// 递归建立左子树
    }
    yesNum = 0;
    noNum = 0;
    HashSet<String> hs2 = new HashSet<String>();
    for (int k = 1; k < rightVector.length; k++) {
        String ans2 = rightVector[k].getExample()[rightVector[0].getExample().length - 1];
        if(ans2.equals("yes")){
            yesNum++;
        }else{
            noNum++;
        }
        hs2.add(ans2);
    }
    double mk2 = 0.0;

    if(yesNum>noNum){
        mk2 = (double)yesNum/(double)(noNum+yesNum);
    }else{
        mk2 = (double)noNum/(double)(noNum+yesNum);
    }
    if (rightVector.length <= 5 || rightVector[0].getExample().length <= 1||hs2.size() == 1||
mk2>rate) {
        // 样本个数太少，样本属性长度为空，所有样本为同一类别，或叶子数据集超过某个比例，则为
        叶子节点，算法结束
        .....//设置叶子节点
    } else {
        rootNode.setRightNode(buildTree(rootNode.RightNode,rightVector,
        F,giniCoff,

```

```

gainCoff,treeDeep,rate));// 递归建立右子树
        }
    //}
    return rootNode;
}
/**
 * 节点分裂混合算法
 * @param myVector 实始数据集
 * @param F 随机特征值
 * @param giniCoff CART 算法系数
 * @param gainCoff C4.5 算法系数
 * @return 待分裂属性节点
 */
public tagNode OldOrNewChoose_Property(tagExampleSet[] myVector, int F, double
giniCoff,double gainCoff) {
    tagNode minTageNode = new tagNode();
    ArrayList<String> varValue = new ArrayList<String>();
    double yesNum1, noNum1;
    double yesNum2, noNum2;
    double D, minD;
    double total, total1, total2;
    double infoD,infoaD,gain,bothmix,splitinfo;
    // 在属性个数范围内, 生成随机数序列, 数量为 F 个不重复的随机数
    int[] nums = getRandom(F, myVector[0].getExample().length - 1);
    for (int i = 0; i < F; i++) { // 取 F 个属性
        HashMap<String, result> h1 = new HashMap<String, result>();
        result re1 = null;
        for (int j = 1; j < myVector.length; j++) { // 取每个样本对应属性的值
            // 取得每个属性值, 及其对应的结果值 的个数
            if (h1.containsKey(myVector[j].getExample()[nums[i]])) {
                re1 = (result) h1.get(new String(myVector[j].getExample()[nums[i]]));
                result re2 = new result();
                re2.setName(re1.getName());
                re2.setNoNum(re1.getNoNum());
                re2.setYesNum(re1.getYesNum());
                if      (myVector[j].getExample()[myVector[0].getExample().length
1].equals("yes")) {

```

```

        re2.setYesNum(re1.getYesNum() + 1);
    } else {
        re2.setNoNum(re1.getNoNum() + 1);
    }
    h1.put(myVector[j].getExample()[nums[i]], re2);

    } else {
        result re3 = new result();
        re3.setName(myVector[j].getExample()[nums[i]]);
        varValue.add(myVector[j].getExample()[nums[i]]);
        if (myVector[j].getExample()[myVector[0].getExample().length
1.equals("yes"))) {
            re3.setYesNum(re3.getYesNum() + 1);
        } else {
            re3.setNoNum(re3.getNoNum() + 1);
        }
        h1.put(myVector[j].getExample()[nums[i]], re3);
    }
}

// 得到该属性所有集合的划分
String[] tempVarValue = new String[varValue.size()];
int tmpSign = 0;
Iterator<String> it1 = varValue.iterator();
while (it1.hasNext()) {
    tempVarValue[tmpSign] = (String) it1.next();
    tmpSign++;
}

String[] kk = TestSort.sort1(tempVarValue);
D = 0;

    minD = 0;
    tagNode taNode1 = new tagNode();
    result re2 = null;
    for (int mm = 0; mm < kk.length / 2; mm++) {
        //根据集合划分情况计算各个子集中正负样本的个数
        .....

        total = yesNum1 + noNum1 + yesNum2 + noNum2;
        total1 = yesNum1 + noNum1;

```

```

        total2 = yesNum2 + noNum2;
// 计算每对集合的 Gini 系数
        D = (total1 / total) * (1 - (yesNum1 / total1) * (yesNum1 / total1) - (noNum1 /
total1) * (noNum1 / total1));
        D += (total2 / total) * (1 - (yesNum2 / total2) * (yesNum2 / total2) - (noNum2 /
total2) * (noNum2 / total2));
        //计算信息益增
        //整个数据集的熵
        infoD = DecisionTreeUtil.info((int)(yesNum1+yesNum2), (int)total) +
DecisionTreeUtil.info((int)(noNum1+noNum2), (int)total);

        infoaD = 0.0;
        splitinfo = 0.0;
        for(int zi=0;zi<arrYesNum.length;zi++){
//求根据此属性的取值对数据集划分的熵
            .....
        }
        gain = infoD-infoaD;
        gainRatio = gain/splitinfo;
        //将信息增益率和 GINI 两者混合
        bothmix = giniCoff*D - gainCoff*gain;
        // 把该属性最小的 GINI 记录下来
        if (mm == 0) {
            minD = bothmix;
//设置最小节点相应属性
            .....
        } else {
            if (bothmix < minD) {
                minD = bothmix;
                //设置最小节点相应属性
                .....
            }
        }
    }
    }
    }
    varValue.removeAll(varValue);
    // 比较最小 GINI 系数，即最佳分裂点
    if (i == 0) {

```

```

        minTageNode.setSign(1);
        minTageNode.setDivRule(taNode1.getDivRule());
        minTageNode.setP_name(taNode1.getP_name());
        minTageNode.setD_value(taNode1.getD_value());
    } else {
        if (taNode1.getD_value() < minTageNode.getD_value()) {
            minTageNode.setSign(1);
            minTageNode.setDivRule(taNode1.getDivRule());
            minTageNode.setP_name(taNode1.getP_name());
            minTageNode.setD_value(taNode1.getD_value());
        }
    }
}
return minTageNode;
}

```

10 在学期间发表的学术论文和研究成果

1. 基于多因素方差分析的遗传算法参数设定 The Fourth International Conference on Genetic and Evolutionary Computing 2010.12
2. 一种随机森林的混合算法 统计与决策 2014.2.400(4):7-9
3. 使用随机森林算法实现优质股票的选择 首都经济贸易大学学报 2014.3
4. 随机森林算法性能影响因素分析 数据分析 2014.4

11 后 记

四年的博士研究生生活即将结束,在此特向对我进行过无私帮助和关心的老师和同学们表示最真挚的谢意!

首先对我的博士生导师纪宏教授、谢邦昌教授致以诚挚的谢意,感谢他们多年来对我的关心、鼓励和帮助。这篇论文从选题到确定研究方法,到最后的修改定稿,纪老师和谢老师都给了我悉心的指导和帮助。四年来,两位导师渊博的知识、开拓的思想、严谨的治学态度、实事求是的研究精神、强烈的创新意识以及诲人不倦的为师品格是我一生宝贵的精神财富,激励我不断努力。

其次要是感谢马立平老师、刘黎明老师等统计学院的老师们,在上学期对我的无私帮忙和指导,在你们的亲切教导和启发下,我开拓了眼界,汲取到丰富的知识,不管是科研能力还是实践能力都得到了极大的提高。

感谢已经毕业的黄恒君师弟、卢媛师妹、王薇师妹、贾辉师妹,感谢霍晨师兄、白涛珍师弟、韩跃师弟、秦磊师弟以及研究小组的全体成员,在课题的研究过程中一起讨论,并给了我许多建设性的意见。感谢首都经济贸易大学 2010 级博士班的全体同学,因为有了大家的相识相知,才有充满欢声笑语的博士生活,才有了这么多丰富美好的回忆。

感谢中国大陆领先的金融数据、信息和软件服务企业 Wind 资讯公司的业务员及数据服务人员,是您们的无私援助,使我能取得完整的财务及股票数据!感谢朝阳永续信息技术有限公司的业务员及数据服务人员,是您们的无私援助,使我能取得完整的上市公司盈利预测数据!

特别的,感谢我的父母及我的家人,在我学习期间为我做出的牺牲和付出。



首都经济贸易大学
Capital University of Economics and Business

博士学位论文

Dissertation for Degree of Doctor